UNIDAD 1: INTELIGENCIA ARTIFICIAL Y AGENTES INTELIGENTES

¿QUÉ ES LA INTELIGENCIA ARTIFICIAL?

La inteligencia es la capacidad de entender, asimilar, elaborar información y utilizarla para resolver problemas. Para inteligencia artificial, las definiciones de los autores varían en torno a dos dimensiones principales: razonamiento-conducta y eficiencia humana.

Procesos mentales	**Sistemas que piensan como humanos**	**Sistemas que piensan racionalmente**
	«El nuevo y excitante esfuerzo de hacer que los computadores piensen… máquinas con mentes, en el más amplio sentido literal». (Haugeland, 1985) «[La automatización de] actividades que vinculamos con procesos de pensamiento humano, actividades como la toma de decisiones, resolución de problemas, aprendizaje…» (Bellman, 1978)	«El estudio de las facultades mentales mediante el uso de modelos computacionales». (Charniak y McDermott, 1985) «El estudio de los cálculos que hacen posible percibir, razonar y actuar». (Winston, 1992)
Conducta	**Sistemas que actúan como humanos**	**Sistemas que actúan racionalmente**
	«El arte de desarrollar máquinas con capacidad para realizar funciones que cuando son realizadas por personas requieren de inteligencia». (Kurzweil, 1990) «El estudio de cómo lograr que los computadores realicen tareas que, por el momento, los humanos hacen mejor». (Rich y Knight, 1991)	«La Inteligencia Computacional es el estudio del diseño de agentes inteligentes». (Poole *et al.*, 1998) «IA… está relacionada con conductas inteligentes en artefactos». (Nilsson, 1998)

Las definiciones que aparecen en la parte superior se refieren a procesos mentales y al razonamiento, mientras que las de la parte inferior aluden a la conducta. Las definiciones de la izquierda miden el éxito en términos de la fidelidad en la forma de actuar de los humanos, mientras que las de la derecha toman como referencia un concepto ideal de inteligencia, que llamaremos racionalidad. Un sistema es racional si hace «lo correcto», en función de su conocimiento

¿Por qué es interesante la IA?

- *Sentido de la identidad:* El hombre se ha aplicado a sí mismo el nombre científico de homo sapiens (hombre sabio o "capaz de conocer") como una valoración de la trascendencia de nuestras habilidades mentales tanto para la vida cotidiana como en nuestro propio sentido de la identidad.
- *Comprensión de entidades inteligentes:* Los esfuerzos en el campo de la IA se enfocan en lograr la compresión de entidades inteligentes. Por eso, una de las razones de su estudio es aprender más sobre nosotros mismos. A diferencia de la filosofía y de la psicología, que también se ocupan de la inteligencia, los esfuerzos de la IA están encaminados tanto a la comprensión de entidades inteligentes como su construcción. Otra razón por la que se estudia la IA es que las entidades inteligentes así construidas son interesantes y útiles por derecho propio. No obstante las fases tempranas en las que aún se encuentra la IA, ha sido posible crear diversos productos, importantes y sorprendentes.

- *Es un problema complejo pero la solución existe:* El problema que aborda la IA es uno de los más complejos, pero a diferencia de la investigación en torno al desplazamiento a mayor velocidad que la luz o de un dispositivo antigravitatorio, el investigador del campo de la IA cuenta con pruebas contundentes de que tal búsqueda es factible. Todo lo que este investigador tiene que hacer es mirarse en un espejo para tener ante sí un ejemplo de sistema inteligente.
- *El estudio de la inteligencia es una de las disciplinas más antiguas:* Por más de 2000 años los filósofos se han preguntado cómo se ve, aprende, recuerda y razona. La llegada de las computadoras a principio de la década de los cincuenta permitió pasar de la especulación, a nivel de charlas de café, en torno a estas facultades mentales a su abordaje mediante una auténtica disciplina teórica y experimental.

FUNDAMENTOS DE LA INTELIGENCIA ARTIFICIAL

- *Comportamiento inteligente:* Se refiere a la capacidad de un sistema para resolver problemas y realizar tareas de manera efectiva, eficiente y adaptativa.
- *Comportamiento racional:* Se refiere a la capacidad de un sistema para tomar decisiones que maximicen un objetivo o meta específica en función de la información disponible.

Corrientes

Desde los inicios de la IA se plantearon dos enfoques diferentes, uno por parte de los investigadores del Carnegie-Mellon y otro por los investigadores del MIT. Estos caminos se fueron alejando hasta llegar a plantearse un distanciamiento que caracterizó las investigaciones en IA de los veinte años siguientes y que fue sin lugar a dudas muy perjudicial por el atraso que provocó en algunas líneas de investigación.

Corriente conexionista - subsimbólica- ascendente.

La escuela liderada por los investigadores de la Universidad de Carnegie Mellon proponía desarrollar modelos del **comportamiento humano** a partir de elementos que simularan en todo lo posible al cerebro con un **enfoque cognitivo**. Así iniciaron una corriente denominada "de **aproximaciones subsimbólicas**" que sigue un diseño de modelo "ascendente", comenzando en el nivel más bajo y operando hacia niveles superiores. Según esta corriente, para concebir máquinas inteligentes hay que seguir muchos de los **pasos evolutivos** que acompañaron el desarrollo de la inteligencia en los seres vivos, o sea hay que comenzar por replicar la capacidad de procesamiento de señales y subir por la cadena evolutiva en pasos sucesivos, construyendo **progresivos niveles superiores de inteligencia** que constituirán las bases sólidas para los pasos siguientes. Al amparo de esta propuesta se desarrollaron los **modelos conexionistas**, **redes neuronales** inspiradas en los modelos biológicos.

Corriente tradicional - simbólica - descendente.

La segunda escuela, liderada por Minsky y McCarthy del MIT orientó su actividad a obtener **comportamientos "inteligentes"** sin procurar que la estructura de los componentes tuviese semejanza con sus equivalentes biológicos y llegó a su mayor auge en los primeros 20 años de la IA. Esta escuela iniciada en el MIT fue denominada "tradicional", "**de procesamiento de símbolos**" o "de diseño descendente" y se apoyó en **representar el conocimiento mediante sentencias declarativas y en la deducción de sus consecuencias** a partir de la aplicación de **reglas de inferencia.** Sin embargo, poco a poco se fue comprobando que muchas de las soluciones propuestas eran apropiadas para modelos pequeños, pero completamente **ineficientes o inaplicables** cuando se pretendía resolver problemas de tamaño real. Los investigadores se encontraron con que sus sistemas sucumbían ante la creciente longitud y complejidad de su programación.

Disciplinas que han contribuido con ideas, puntos de vista y técnicas al desarrollo de la IA

- Los filósofos (desde el año 400 a.C.) facilitaron el poder imaginar la IA, al concebir la idea de que la mente es de alguna manera como una máquina que funciona a partir del conocimiento codificado en un lenguaje interno, y al considerar que el pensamiento servía para seleccionar la acción a llevar a cabo. Modelo de cómo funciona la mente.
- Las matemáticas proporcionaron las herramientas para manipular tanto las aseveraciones de certeza lógicas, como las inciertas de tipo probabilístico. Asimismo, prepararon el terreno para un entendimiento de lo que es el cálculo y el razonamiento con algoritmos.
- Los economistas formalizaron el problema de la toma de decisiones para maximizar los resultados esperados. Se maximiza el razonamiento racional
- Los psicólogos adoptaron la idea de que los humanos y los animales podían considerarse máquinas de procesamiento de información.
- Los lingüistas demostraron que el uso del lenguaje se ajusta a ese modelo.
- Los informáticos proporcionaron los artefactos que hicieron posible la aplicación de la IA. Los programas de IA tienden a ser extensos y no podrían funcionar sin los grandes avances en velocidad y memoria aportados por la industria informática.
- La teoría de control se centra en el diseño de dispositivos que actúan de forma óptima con base en la retroalimentación que reciben del entorno en el que están inmersos. Inicialmente, las herramientas matemáticas de la teoría de control eran bastante diferentes a las técnicas que utilizaba la IA, pero ambos campos se están acercando.

Antecedentes de la Inteligencia Artificial

Hay muchas actividades divididas en muchos subcampos. Algunas aplicaciones son:

- *Planificación autónoma:* A un centenar de millones de millas de la Tierra, el programa de la NASA **Agente Remoto** se convirtió en el primer programa de planificación autónoma a

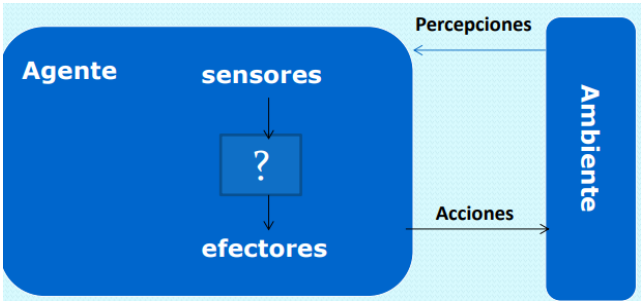
bordo que controlaba la planificación de las operaciones de una nave espacial desde abordo (Jonsson et al., 2000). El Agente Remoto generaba planes a partir de objetivos generales especificados desde tierra, y monitorizaba las operaciones de la nave espacial según se ejecutaban los planes (detección, diagnóstico y recuperación de problemas según ocurrían).

- *Juegos:* Deep Blue de IBM fue el primer sistema que derrotó a un campeón mundial en una partida de ajedrez cuando superó a Garry Kasparov por un resultado de 3.5 a 2.5 en una partida de exhibición (Goodman y Keene, 1997). Kasparov dijo que había percibido un «nuevo tipo de inteligencia» al otro lado del tablero. La revista Newsweek describió la partida como «La partida final». El valor de las acciones de IBM se incrementó en 18 billones de dólares.
- *Control autónomo:* El sistema de visión por computador ALVINN (Autonomous Land Vehicle In a Neural Network) fue entrenado para dirigir un coche de forma que siguiese una línea. Se instaló en una furgoneta controlada por computador en el NAVLAB de Carnegie Mellon University que se utilizó para dirigir al vehículo por Estados Unidos. El NAVLAB posee videocámaras que transmiten imágenes de la carretera a ALVINN, que posteriormente calcula la mejor dirección a seguir, basándose en las experiencias acumuladas en los viajes de entrenamiento. Es el antecesor del auto de google.
- *Diagnosis:* Los programas de diagnóstico médico basados en el análisis probabilístico han llegado a alcanzar niveles similares a los de médicos expertos en algunas áreas de la medicina. Heckerman (1991) describe un caso en el que un destacado experto en la patología de los nodos linfáticos se burló del diagnóstico generado por un programa en un caso especialmente difícil. El creador del programa le sugirió que le preguntase al computador cómo había generado el diagnóstico. La máquina indicó los factores más importantes en los que había basado su decisión y explicó la ligera interacción existente entre varios de los síntomas en este caso. Eventualmente, el experto aceptó el diagnóstico del programa.
- *Planificación logística (guerra):* Durante la crisis del Golfo Pérsico de 1991, las fuerzas de Estados Unidos desarrollaron la herramienta Dynamic Analysis and Replanning Tool para automatizar la planificación y organización logística del transporte. Lo que incluía hasta 50.000 vehículos, carga y personal a la vez, teniendo en cuenta puntos de partida, destinos, rutas y la resolución de conflictos entre otros parámetros. Las técnicas de planificación de IA permitieron que se generará un plan en cuestión de horas que podría haber llevado semanas con otros métodos.
- *Robótica:* Muchos cirujanos utilizan hoy en día asistentes robot en operaciones de microcirugía. HipNav (DiGioia et al., 1996) es un sistema que utiliza técnicas de visión por computador para crear un modelo tridimensional de la anatomía interna del paciente y después utiliza un control robotizado para guiar el implante de prótesis de cadera.
- *Procesamiento de lenguaje y resolución de problemas:* PROVER B desarrollado en la Universidad de Duke, (Littman et al., 1999) es un programa informático que resuelve

crucigramas mejor que la mayoría de los humanos, utilizando restricciones en programas de relleno de palabras, una gran base de datos de crucigramas, y varias fuentes de información como diccionarios y bases de datos online, que incluyen la lista de películas y los actores que intervienen en ellas, entre otras cosas.

AGENTES INTELIGENTES

Un agente es cualquier cosa capaz de percibir su medio ambiente con la ayuda de sensores y actuar en ese medio mediante actuadores. Un agente tomará una decisión en un momento dado dependiendo de la secuencia completa de percepciones hasta ese instante. La función del agente que describe su comportamiento se puede presentar en forma de tabla.



¿Cómo deben actuar los agentes inteligentes?

Los agentes inteligentes son racionales o sea hacen lo correcto.

Acción correcta = acción que lleva al agente a obtener el mejor desempeño.

Para evaluar el desempeño es necesario decidir ***cómo y cuándo*** medir dicho desempeño.

La racionalidad de un Agente Inteligente dependerá de los siguientes factores:

- La medida del rendimiento que define el criterio de éxito
- El conocimiento del medio en el que habita que ha acumulado el agente
- Las acciones que el agente puede llevar a cabo
- La secuencia de percepciones del agente hasta ese momento

Un **agente es racional** si en cada posible secuencia de percepciones (percepción se utiliza en este contexto para indicar que el agente puede recibir entradas en cualquier instante), emprende aquella acción que maximice su medida de rendimiento, basándose en las evidencias aportadas por la secuencia de percepciones y en el conocimiento que el agente mantiene almacenado. Es aquel que hace lo correcto, es decir, aquello que permite al agente obtener un mejor resultado. Se necesita determinar una forma de medir el éxito. En general, un agente tomará una decisión en un momento dado dependiendo de la secuencia completa de percepciones hasta ese instante.

Un agente debe ser **autónomo**, es decir, debe saber aprender a determinar cómo tiene que compensar el conocimiento incompleto o parcial inicial. Después de las suficientes experiencias interaccionando con el entorno, el comportamiento del agente racional será efectivamente independiente del conocimiento inicial.

Aprendizaje: La configuración inicial del agente puede reflejar un conocimiento preliminar del entorno, pero a medida que el agente adquiere experiencia éste puede modificarse y aumentar.

La naturaleza del entorno: En la discusión de la racionalidad de un agente aspiradora simple, hubo que especificar las medidas de rendimiento, el entorno, y los actuadores y sensores del agente. Todo ello forma lo que se llama el entorno de trabajo, para cuya denominación se utiliza el acrónimo **REAS (Rendimiento, Entorno, Actuadores, Sensores)**. En el diseño de un agente, el primer paso debe ser siempre especificar el entorno de trabajo de la forma más completa posible

Tipo de agente	Medidas de rendimiento	Entorno	Actuadores	Sensores
Sistema de diagnóstico médico	Pacientes sanos, reducir costes, demandas	Pacientes, hospital, personal	Visualizar preguntas, pruebas, diagnósticos, tratamientos, casos	Teclado para la entrada de síntomas, conclusiones, respuestas de pacientes
Sistema de análisis de imágenes de satélites	Categorización de imagen correcta	Conexión con el satélite en órbita	Visualizar la categorización de una escena	Matriz de pixels de colores
Robot para la selección de componentes	Porcentaje de componentes clasificados en los cubos correctos	Cinta transportadora con componentes, cubos	Brazo y mano articulados	Cámara, sensor angular
Controlador de una refinería	Maximizar la pureza, producción y seguridad	Refinería, operadores	Válvulas, bombas, calentadores, monitores	Temperatura, presión, sensores químicos
Tutor de inglés interactivo	Maximizar la puntuación de los estudiantes en los exámenes	Conjunto de estudiantes, agencia examinadora	Visualizar los ejercicios, sugerencias, correcciones	Teclado de entrada

Entorno de trabajo de los Agentes Inteligentes

El entorno de trabajo de los agentes inteligentes se refiere al contexto o espacio en el que un agente inteligente opera y toma decisiones. Este entorno puede ser físico o virtual, y su diseño y características influyen en cómo el agente interactúa con él y realiza sus tareas.

Propiedades de los entornos de trabajo:

- **Totalmente observable vs Parcialmente observable:**
  - *Totalmente observable:* Si los sensores del agente le proporcionan acceso al estado completo del medio en cada momento, entonces se dice que el entorno de trabajo es totalmente observable. Los sensores del agente le proporcionan acceso al estado completo del entorno; ejemplo: los sensores detectan todos los aspectos relevantes a la toma de decisiones.
  - *Parcialmente observable:* no es totalmente observable debido al ruido y a sensores poco exactos o que no reciben la información de alguna parte del sistema. Por ejemplo, un agente aspiradora con sólo un sensor de suciedad local no puede saber si hay suciedad en la otra cuadrícula, y un taxi automatizado no pude saber qué están pensando otros conductores.
- **Determinista vs Estocástico:** Es determinista si el siguiente estado del medio está totalmente determinado por el estado actual y la acción ejecutada por el agente, en caso contrario es estocástico.
  - *Determinista* sería el juego de ajedrez: el siguiente estado del tablero (medio) está completamente determinado por el estado actual de las piezas y la acción (movimiento)

realizada por un jugador (agente). Cada movimiento sigue reglas estrictas, y no hay elementos de azar o incertidumbre sobre el resultado de un movimiento.

- *Estocástico:* más bien con el azar, con no depender de algo previo. El agente taxi es claramente estocástico en este sentido, ya que no se puede predecir el comportamiento del tráfico exactamente; más aún, una rueda se puede reventar y un motor se puede gripar sin previo aviso

- **Episódico vs Secuencial:**
  - *Entorno episódico:* cuando la experiencia del agente se divide en episodios atómicos independientes, donde cada episodio consiste en la percepción del agente y la realización de una única acción posterior.
  - *Entorno secuencial:* no existe dicha división y una decisión presente puede afectar a decisiones futuras.

    Por ejemplo, un agente que tenga que seleccionar partes defectuosas en una cadena de montaje basa sus decisiones en la parte que está evaluando en cada momento, sin tener en cuenta decisiones previas; más aún, a la decisión presente no le afecta el que la próxima fase sea defectuosa. En entornos secuenciales, por otro lado, la decisión presente puede afectar a decisiones futuras. El ajedrez y el taxista son secuenciales: en ambos casos, las acciones que se realizan a corto plazo pueden tener consecuencias a largo plazo.
- **Estático vs Dinámico:**
  - *Estático:* el entorno no cambia mientras el agente está deliberando
  - *Dinámico:* el entorno sí cambia

    El taxista es claramente dinámico: tanto los otros coches como el taxi se están moviendo mientras el algoritmo que guía la conducción indica qué es lo próximo a hacer. El ajedrez, cuando se juega con un reloj, es semideterminista. Los crucigramas son estáticos.
- **Discreto vs Continuo:** La distinción entre discreto y continuo se puede aplicar al estado del medio, a la forma en que se maneja el tiempo y a las percepciones y acciones del agente.
  - *Discreto:* el entorno tiene un número finito de estados distintos
  - *Continuo:* no es posible enumerar los estados

    Por ejemplo, un medio con estados discretos como el del juego del ajedrez tiene un número finito de estados distintos. El ajedrez tiene un conjunto discreto de percepciones y acciones. El taxista conduciendo define un estado continuo y un problema de tiempo continuo: la velocidad y la ubicación del taxi y de los otros vehículos pasan por un rango de valores continuos de forma suave a lo largo de tiempo. Las conducción del taxista es también continua (ángulo de dirección, etc.).

- **Individual vs Multiagente:** La distinción entre el entorno de un agente individual y el de un sistema multiagente puede parecer simple, pero hay que analizar cada caso en particular, por ejemplo un agente resolviendo un crucigrama es el caso de un agente individual pero un agente que juega al ajedrez es un entorno multiagente.
    - *Individual:* un solo agente resolviendo un problema
    - *Multiagente:* varios compitiendo o cooperando

      Por ejemplo, un agente resolviendo un crucigrama por sí mismo está claramente en un entorno de agente individual, mientras que un agente que juega al ajedrez está en un entorno con dos agentes.

Entornos de trabajo	Observable	Determinista	Episódico	Estático	Discreto	Agentes
Taxi circulando	Parcialmente	Estocástico	Secuencial	Dinámico	Continuo	Multi
Diagnóstico médico	Parcialmente	Estocástico	Secuencial	Dinámico	Continuo	Individual
Análisis de imagen	Totalmente	Determinista	Episódico	Semi	Continuo	Individual
Robot clasificador	Parcialmente	Estocástico	Episódico	Dinámico	Continuo	Individual
Controlador de refinería	Parcialmente	Estocástico	Secuencial	Dinámico	Continuo	Individual
Tutor interactivo de inglés	Parcialmente	Estocástico	Secuencial	Dinámico	Discreto	Multi

Estructura de los agentes inteligentes:

Agente = arquitectura + programa

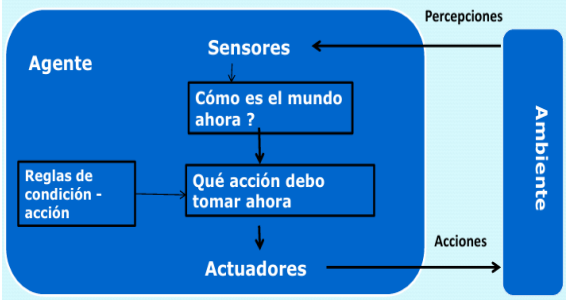
El programa debe ser apropiado para la arquitectura, si el programa tiene que recomendar acciones como caminar, la arquitectura tiene que tener piernas. La arquitectura puede ser un PC, un coche robotizado con varias computadoras, cámaras y sensores, etc. Los programas reciben la percepción actual como entrada de los sensores y devuelven una acción de los actuadores.

```
función AGENTE-DIRIGIDO-MEDIANTE TABLA(percepción) devuelve una acción
    variables estáticas: percepciones, una secuencia, vacía inicialmente
                         tabla, una tabla de acciones, indexada por las secuencias de
                         percepciones, totalmente definida inicialmente

    añadir la percepción al final de las percepciones
    acción ← CONSULTA(percepciones, tabla)
    devolver acción
```

Tipos de programas de los agentes inteligentes:

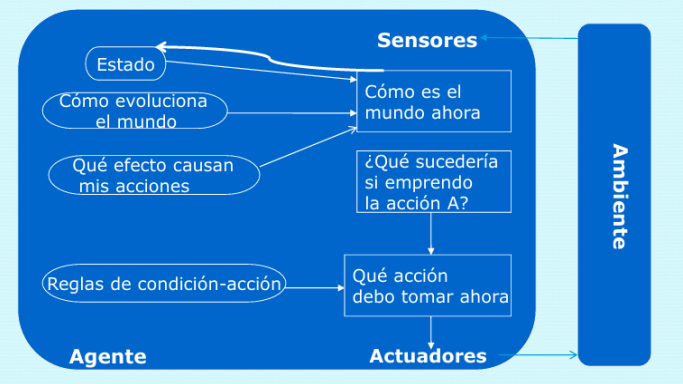
- ***Agentes reactivos simples:*** estos agentes seleccionan las acciones sobre la base de las percepciones actuales, ignorando el resto de las percepciones históricas. Tienen reglas o conexiones como si… entonces…



```
función AGENTE-REACTIVO-SIMPLE(percepción) devuelve una acción
    estático: reglas, un conjunto de reglas condición-acción

    estado ← INTERPRETAR-ENTRADA(percepción)
    regla ← REGLA-COINCIDENCIA(estado, reglas)
    acción ← REGLA-ACCIÓN[regla]
    devolver acción
```

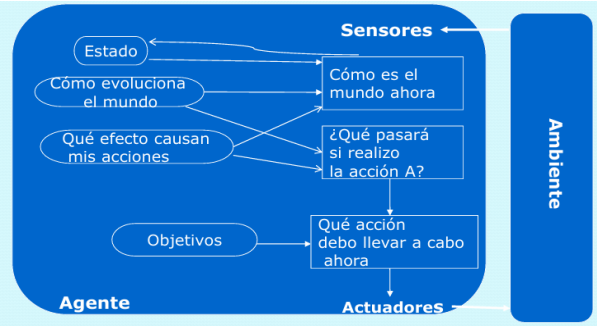
- **Agentes reactivos basados en modelos:** La forma más efectiva que tienen los agentes de manejar la visibilidad parcial es almacenando información de las partes del mundo que no pueden ver, es decir, el agente debe mantener algún tipo de estado interno que dependa de la historia percibida, que permita reflejar por lo menos alguno de los aspectos no observables. La actualización de la información de estado interno según pasa el tiempo requiere codificar dos tipos de conocimiento en el programa del agente. *Primero* se necesita alguna información acerca de cómo evoluciona el mundo independientemente del agente. *Segundo, se necesita* más información sobre cómo afectan al mundo las acciones del agente. Este conocimiento acerca de cómo funciona el mundo se denomina modelo del mundo, un agente que utilice este modelo es un agente basado en modelos.



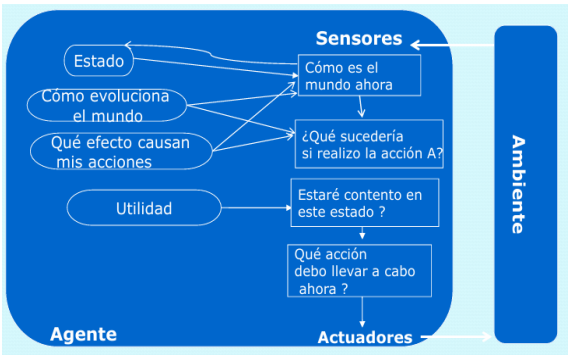

```
función AGENTE-REACTIVO-CON-ESTADO(percepción) devuelve una acción
    estático: estado, una descripción actual del estado del mundo
              reglas, un conjunto de reglas condición-acción
              acción, la acción más reciente, inicialmente ninguna

    estado ← ACTUALIZAR-ESTADO(estado, acción, percepción)
    regla ← REGLA-COINCIDENCIA(estado, reglas)
    acción ← REGLA-ACCIÓN[regla]
    devolver acción
```

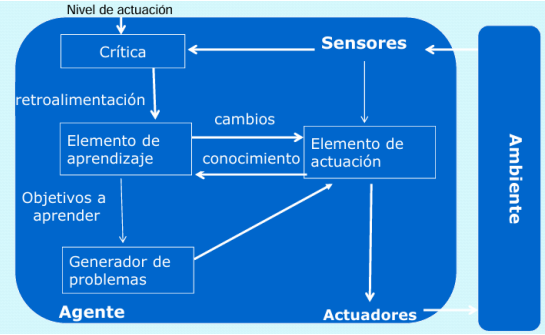
- **Agentes basados en objetivos:** El conocimiento sobre el estado actual del mundo no es siempre suficiente para decidir que hacer, es decir que además de la descripción del estado actual, el agente necesita información sobre su meta, que describa situaciones que son deseables.



- **Agentes basados en utilidad:** Estos agentes utilizan una función de utilidad que proyecta un estado en un número real que representa un nivel de felicidad. La definición completa de una función de utilidad permite tomar decisiones racionales en dos tipos de casos en los que las metas son inadecuadas. *Primero* cuando haya objetivos conflictivos y solo se puedan alcanzar algunos de ellos, la función de utilidad determina el equilibrio adecuado. *Segundo*, cuando haya varios objetivos por los que se pueda guiar el agente, y ninguno de ellos se pueda alcanzar con certeza, la utilidad proporciona un mecanismo para ponderar la probabilidad de éxito en función de la importancia de los objetivos.



- **Agentes que aprenden:** Un agente que aprende posee dos componentes principales, un elemento de aprendizaje responsable de hacer mejoras y un elemento de actuación responsable de la selección de acciones externas. El elemento de aprendizaje se realimenta de las críticas sobre la actuación del agente y determina cómo se debe modificar el elemento de actuación para proporcionar mejores resultados en el futuro. El generador de problemas es responsable de sugerir acciones que lo guiarán hacia experiencias nuevas e informativas. La observación de los estados sucesivos del entorno puede permitir que el agente aprenda cómo evoluciona el mundo y la observación de los resultados de sus acciones puede permitir que el agente aprenda qué hacen sus acciones. El aprendizaje en el campo de los agentes inteligentes puede definirse como el proceso de modificación de cada componente del agente, lo cual permite a cada componente comportarse más en consonancia con la información que se recibe, lo que por tanto permite mejorar el nivel medio de actuación del agente.



Definiciones y enfoques de la IA: La inteligencia (general) es "la capacidad de entender, asimilar, elaborar información y utilizarla para resolver problemas"

Para Inteligencia Artificial las definiciones de los autores actuales varían en torno a dos dimensiones principales:

- Razonamiento-conducta
- Eficiencia humano-ideal

"La capacidad de realizar tareas que, por el momento, los humanos hacen mejor" - Elaine Rich y Kevin Knight

Breve historia y algunos hitos

- En 1936 Alan Turing crea la Máquina de Turing, la base de las computadoras.
- En 1943, Warren McCulloch y Walter Pitts crean la primera neurona artificial.
- Alan Turing, consciente de los alcances de su máquina, define en 1950 el Test de Turing. Una prueba para determinar si una máquina es inteligente (como un humano).
- En 1956, John McCarthy convocó el Dartmouth workshop, una reunión de dos meses de duración a la que asistieron cerca de 10 científicos que trabajaban en temas relacionados con "máquinas pensantes". Estuvieron, entre otros, Marvin Minsky, Nathaniel Rochester y Claude Shannon. En esta reunión se acuñó el nombre "Inteligencia Artificial".
- La primera década estuvo marcada por un optimismo exagerado. Se rápidamente resolvieron problemas como la demostración de teoremas y se logró que las máquinas superarán a los humanos en algunos juegos simples.
- Etapa de depresión (AI winter). Cuando se intentaron atacar problemas del dominio del sentido común o aquellos que necesitan de conocimiento general, como los relacionados con el lenguaje, los esfuerzos no fueron suficientes. Esta situación llevó a un desánimo generalizado y a la pérdida de inversiones.
- Después de la etapa de depresión, ya en los 80's, la IA resurgió de la mano de los Sistemas Expertos. Desde ese momento el área no deja de crecer. Los sistemas de visión artificial y los de reconocimiento de patrones han provisto innumerables soluciones en los últimos años. La cantidad de datos disponibles, los recursos computacionales y los avances como el de las redes

neuronales profundas (deep learning) han formado un círculo virtuoso donde tanto las empresas como la comunidad científica invierten cada vez más recursos.

Presente

Universidades y empresas, muchas veces en conjunto, no dejan de romper barreras. Las tareas que son realizadas mejor por los humanos que por las máquinas se reducen vertiginosamente. Las empresas de la imagen de arriba son un ejemplo. Autos que se manejan solos, máquinas que responden preguntas de conocimiento general mejor que una persona, reconocimiento de objetos con precisión superior a la humana, etc.

Otras razones (más mundanas) para hacer IA

En los últimos años se incrementó fuertemente la demanda laboral para los perfiles de data scientist.

¿Qué pasará en el futuro? Si bien nadie podría pronosticar con toda precisión lo que se podría esperar en el futuro, es evidente que las computadoras que posean inteligencia a nivel humano (o superior) tendrán repercusiones muy importantes en nuestra vida diaria así como en el devenir de la civilización.

El caso extremo es la singularidad tecnológica. Un punto en el tiempo donde, para algunos pensadores, la inteligencia artificial superará a la humana.

UNIDAD 2: APRENDIZAJE AUTOMÁTICO

Cuando el diseñador desconoce las leyes que gobiernan el ambiente, el único camino es el aprendizaje automático.

Reconocimiento de patrones

¿Qué es el Reconocimiento de Patrones? El reconocimiento está considerado como una función básica del ser humano. Por otra parte, cuando hablamos de un patrón decimos que el mismo es la descripción de un objeto. Se puede aplicar en un agente inteligente o como un fin en sí mismo.

Sería fácil enumerar ejemplos de nuestros permanentes actos de reconocimiento de objetos, una voz o un rostro familiares, gestos, formas, colores, etc. Un ser humano es un sofisticado sistema de información, y esto es debido en parte a su elevada capacidad de reconocimiento de objetos.

Podemos distinguir dos categorías de reconocimiento: *reconocimiento de objetos concretos y reconocimiento de objetos abstractos*. En el caso de objetos concretos se trata de reconocimiento perceptual, como por ejemplo una forma (patrón espacial) o una secuencia (patrón temporal). En el caso de objetos abstractos se trata de reconocimiento conceptual, tal como un viejo argumento o la solución de un problema.

El Reconocimiento de Patrones que vamos a desarrollar está asociado al Reconocimiento de objetos concretos (Perceptual). Cuando una persona percibe un patrón, realiza una inferencia inductiva y asocia esta percepción con algunos conceptos generales derivados de su experiencia pasada. De este modo el problema del reconocimiento puede ser concebido como el de discriminar, clasificar o categorizar la información de entrada, no entre patrones individuales sino entre poblaciones, por medio de la búsqueda de características o atributos invariantes entre los miembros de una población. En lo que sigue, se tratan aspectos relacionados con la informática e ingeniería sobre el diseño de sistemas de reconocimiento autónomos.

Aplicaciones: El Reconocimiento de Patrones provee una teoría general cuyas aplicaciones son múltiples. Algunas son:

Tarea de clasificación	Datos de entrada	Salida
Reconocimiento de Caracteres Reconocimiento del habla Reconocimiento de voz Predicción del clima Diagnóstico médico Predicción de Acciones	Señales ópticas o líneas Ondas acústicas Voz Mapas climáticos Síntomas Gráficos financieros	Nombre del carácter Nombre de la palabra Nombre de quien habla Pronóstico climatológico Enfermedad Predicción de alzas y bajas

El diseño de un sistema de reconocimiento automático involucra por lo general las tareas siguientes:

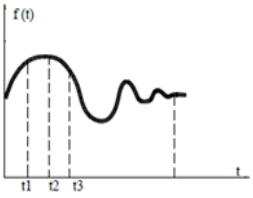
1. **Sensado:** se refiere a la representación de la información obtenida mediante algún tipo de sensor sobre los objetos a ser reconocidos. Cada cantidad medida describe una característica del objeto. Es el acto de medir u obtener los valores de ciertos atributos del objeto a ser reconocido.

   Esta grilla puede representarse mediante un vector patrón como sigue:



$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \quad con \quad x_i = 0 \quad o \quad x_i = 1$$

   Donde por ejemplo se asigna el valor 1 si el elemento forma parte del carácter y 0 en caso contrario.



Función continua de una variable t (una señal acústica). Esta función es muestreada en puntos discretos y puede conformarse el vector que representa al patrón tomando los valores de la función tomando:

$$x_1 = f(t_1), x_2 = f(t_2), \ldots, x_3 = f(t_3)$$

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \quad con \quad x_i = f(t_i)$$

Los vectores patrones son definidos como vectores columna pero pueden eventualmente ser expresados por su traspuesta: $\mathbf{x} = (x_1, x_2, \ldots, x_n)'$



Toda la información medida disponible acerca de los patrones está contenida en los vectores de patrones. Cuando los vectores de patrones están formados por números reales es útil interpretarlos como puntos en el espacio **Euclidiano n-dimensional.** En casos sencillos, el conjunto de puntos que corresponden a una misma clase resulta así agrupados en alguna región del espacio. A este agrupamiento se lo denomina **Cluster (ω).**

Por ejemplo, para estudiar la incidencia de alguna enfermedad se podría "censar" la altura, el peso, el sexo, la edad y el nivel educativo de varias personas. Otros atributos frecuentemente usados en distintas áreas son: Intensidades lumínicas (imágenes), presión sonora (audio), texto (mails, tweets, artículos científicos, prensa gráfica, etc.), microarrays de expresión génica.

2. **Extracción de Características:** Un segundo problema en el área del Reconocimiento de Patrones, está referido a la *reducción de la dimensión de los vectores de los patrones*. Se suele mencionar a esta etapa como preproceso o extracción de características. El método analítico más conocido se denomina **"Análisis de las Componentes Principales"**, o también **"Transformación de los Ejes Principales".** Lo que se logra con esto es reducir la dimensión de los patrones pero con la posibilidad de perder un bajo porcentaje de la información contenida en ellos. A su vez al poder emplear vectores de menor dimensión casi sin

pérdida de información se disminuye el costo computacional lo que a veces es imprescindible por limitaciones tecnológicas.

3. **Clasificación:** El tercer y quizás más crítico problema del Reconocimiento de Patrones, se refiere a la determinación de un procedimiento óptimo de clasificación.

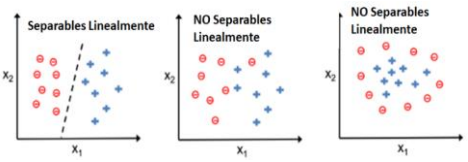
   Para lograr esto, se requiere diseñar un mecanismo que, dado un conjunto de patrones de los cuales no se conoce a priori la pertenencia a la clase de cada uno de ellos, permita determinar a qué clase pertenece cada patrón. Suponiendo el sistema diseñado para reconocer M clases diferentes, denominadas ω1, ω2 …ωn puede considerarse al espacio de patrones compuesto por M regiones cada una de las cuales corresponde a los patrones de una clase.

   La solución puede interpretarse como generar los límites de decisión entre las regiones. Estos pueden estar dados por las llamadas **Funciones de Decisión o Funciones Discriminantes** d1(x), d2(x), …, dn(x), las que son funciones escalares de cada vector x.

   Si una función tiene el mayor valor entre todas las demás, entonces el vector pertenece a la clase correspondiente a esa función, es decir si $d_i(x) > d_j(x)$ para i, j=1, 2, …, M y j≠i entonces el patrón x pertenece a la clase ωi

   Es decir que la pertenencia a una clase es determinada por el mayor valor de la función discriminante $d_i(x)$

   Tipos de problemas reales: Linealmente separables y No linealmente separables



El resultado de las etapas de sensado y extracción de características es un conjunto de vectores de características. Cada vector tiene la forma: $x=[x_1,x_2, \ldots, x_n]$ donde

- x_i es el valor de la característica i

Aclaración de nomenclatura:

- u es un escalar.
- **u** (en negrita) es un vector formado por escalares u_i
- U es una matriz formada por escalares u_{ij}

Después de obtener los vectores de características se realiza una de las siguientes tareas:

- Clasificación. Asignar cada vector de características a una clase. Por ejemplo, para detectar qué carácter aparece en una imagen.
- Regresión. Predecir un valor (continuo) para cada vector de características. Por ejemplo, para calcular el valor futuro de ciertas acciones bursátiles.

Los métodos de aprendizaje automático que vamos a ver en la materia son clasificadores, sin embargo es importante tener en cuenta que ambas tareas (clasificación y regresión) son comunes y que gran parte de los métodos que se usan para una de las tareas, pueden también ser útiles para la otra si se aplican pequeñas modificaciones. Por lo tanto, el conocimiento de los temas que veremos a partir de ahora es útil también para tareas de predicción.

Entrenamiento.

Los clasificadores tienen parámetros internos que determinan la salida. El proceso de ajuste de estos parámetros se llama entrenamiento. Existen dos tipos (principales):

- Entrenamiento supervisado: Los parámetros se ajustan de forma tal que la salida del clasificador se aproxime a una salida esperada conocida.
- Entrenamiento no supervisado: Los parámetros se ajustan para encontrar relaciones o agrupaciones entre los valores de las características.

Representación de datos

Los datos censados se representan mediante un vector $X = (x_1, x_2, \ldots, x_n)$ donde xi es el valor del atributo i. En el caso de una imagen, xi sería la intensidad del pixel i. En el caso del audio, la energía en el tiempo i. En el caso del texto, este se guardaría como un atributo, x1 por ejemplo, y también algunos metadatos (fecha, fuente, etc.) como otros atributos.

Codificación de variables

Variables cuantitativas: Se representan como números del tipo que sea necesario. En el primer ejemplo, peso, altura y edad son variables cuantitativas y se podrían representar con un float. En una imagen, las intensidades de los pixeles se pueden representar con enteros.

Variables categóricas: Pueden tomar solo un valor dentro de un conjunto limitado de valores. En el caso del nivel educativo, estos valores podrían ser "ninguno", "primario, "secundario", "terciario", "universitario o superior".

Las variables categóricas pueden ser ordinales o nominales. En las ordinales los valores que pueden tomar pertenecen a una escala (hay algunos superiores a otros), como por ejemplo, en el caso del nivel educativo.

Las variables categóricas nominales, como la nacionalidad, especie, color, etc., no tienen un orden, por lo tanto se deben representar de forma tal que se prevenga al clasificador asumir un orden.

Los clasificadores están formados por funciones matemáticas, por lo tanto, reciben valores numéricos. Las variables categóricas deberán tener representaciones numéricas. Por ejemplo, si la variable color puede tomar tres valores, "rojo", "verde" y "azul", podríamos tentarnos de codificarla de la siguiente forma: "rojo" = 1, "verde" = 2 y "azul" = 3. Esto es una

representación válida para un clasificador, sin embargo, estaríamos creando un orden que no existe en la realidad. El clasificador podría interpretar que "azul" es mayor que "rojo" o que "rojo" es más parecido a "verde" que a "azul". Para estos casos se utiliza la codificación one-hot, donde se expande la variable categórica original en m atributos binarios, donde "m" es la cantidad de valores posibles para la variable original.

Para utilizar variables en una red neuronal, necesitamos convertirlas en valores numéricos. Una opción común es la codificación one-hot para variables categóricas y la codificación ordinal para variables con un orden natural. Normalmente la codificación es en binario para evitar darle distintos pesos. Po ejemplo:
Para las variables de Estado civil ("casado", "soltero", "divorciado"):

- Casado: [1, 0, 0]
- Soltero: [0, 1, 0]
- Divorciado: [0, 0, 1]

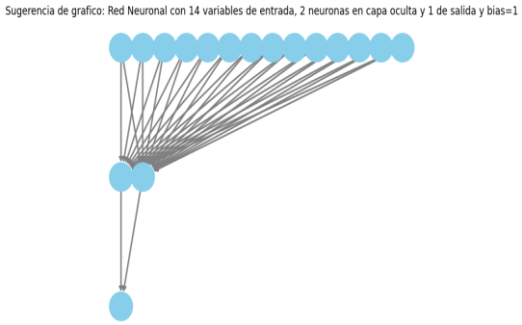
Vector de entrada codificado: Concatenamos las codificaciones de los atributos de entrada, es decir concatenamos estos vectores de características en un solo vector

$$Entrada = [codificación\ variable\ 1] + \ldots + [codificación\ variable\ n]$$

Modelos de representación gráfica para el análisis

En general, todo el estudio sobre los sistemas de reconocimiento se encara a partir de la **representación de patrones en dos dimensiones,** debido a la simplicidad del modelo y a la validez general (en n dimensiones) de los resultados que de él se deriven. Pero también es importante el hecho de que las personas podemos representar y visualizar claramente las situaciones, solo en el plano y en el espacio tridimensional.

A pesar de esto, a veces se realizan esfuerzos para visualizar resultados en más dimensiones, por ejemplo, si estuviéramos en cuatro dimensiones, la cuarta dimensión podría ser representada asignando colores –en nuestro estudio a los patrones- que variarían según el espectro térmico, de cálidos a fríos o viceversa-, eso nos permitiría ver quizás algo mejor las características de la información disponible.



La red tendría conexiones (pesos) desde cada neurona de entrada hacia las neuronas de la capa oculta, y luego desde estas hacia la neurona de salida. Cada peso es un valor ajustable que el modelo aprende durante el entrenamiento.

La Función de Decisión Lineal / Función discriminante lineal (clasificador lineal)

El clasificador lineal pertenece al grupo de métodos de aprendizaje supervisado, es decir, es necesario contar con un conjunto de vectores previamente clasificados para entrenarlo. Este conjunto de vectores se llama **vectores de entrenamiento.** Solo puede resolver problemas de clasificación donde las clases son linealmente separables.

La función principal de un sistema de reconocimiento de patrones es tomar decisiones acerca de la pertenencia a una clase dada de los patrones con que se lo confronta. Para lograr esto, es necesario establecer algunas reglas en que basar estas decisiones. Una aproximación fundamental a este problema es el empleo de *funciones de decisión*.

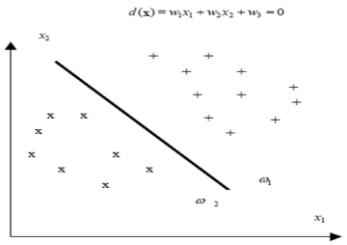
Considérese $d(x) = w_1x_1 + w_2x_2 + w_3 = 0$ como la ecuación de una línea de separación donde w es el vector de parámetros y x es el vector de las variables de la entrada.

Cualquier modelo que pertenece a una clase ω1 producirá una cantidad positiva cuando sea sustituido en la ecuación del hiperplano d(x).

Similarmente d(x) se hace negativo con la substitución de cualquier patrón de la otra clase ω2. Por consiguiente esta línea d(x) puede usarse como una función de decisión dado un patrón x de clasificación desconocida, podemos decir que este pertenece a una determinada clase ω1 si d(x) > 0 o a ω2, en caso contrario.

Ninguno de estos conceptos necesita ser restringido a dos clases, solo se emplean dos clases por simplicidad.

Desgraciadamente, a menos que alguna información esté disponible a priori, la única manera de establecer la efectividad de una función de decisión escogida es por ensayo directo. Una vez que una cierta función (o funciones si se trata de más de dos clases) se ha seleccionado, el problema se reduce a la determinación de los coeficientes. Es posible usar los patrones de una muestra para determinar los coeficientes que caracterizan estas funciones.



Extensión al Espacio euclideano n-dimensional:

Se puede extender a n-dimensiones definiendo una función de decisión lineal general de la forma: $d(x) = w_1x_1 + w_2x_2 + w_nx_n + w_{n+1}$

$d(x) = w_0'x + w_{n+1}$

Donde el vector es llamado **vector de pesos o vector de parámetros.** Es además una convención muy difundida adicionar un 1 como última componente de todos los vectores patrones con lo que la igualdad anterior puede expresarse como $d(x) = w_0'x$ donde $x = (x_1, x_2, \ldots x_n, 1)'$ es llamado *vector patrón aumentado.*

La función de decisión tiene la forma: $d(x) = w_1x_1 + w_2x_2 + w_nx_n + w_{n+1}x_{n+1}$

Como la misma cantidad es agregada a todos los patrones de todas las clases en juego las propiedades geométricas entre las mismas se mantienen. Dependiendo del contexto del

problema y del sistema de clasificación se emplea el vector patrón aumentado o no, pero en cualquier caso siempre en la práctica se hace referencia al mismo simplemente como vector patrón.

En resumen de la función de decisión lineal:

Si las entradas de un clasificador lineal tienen la forma $\boldsymbol{x} = [x_1, x_2, \ldots, x_n]$, la función de decisión se define como:

$$d(\boldsymbol{x}) = w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + w_{n+1}$$

O con vector de enradas aumentado, $\boldsymbol{x} = [x_1, x_2, \ldots, x_n, 1]$,

$$d(\boldsymbol{x}) = w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + x_{n+1} w_{n+1}$$

Cálculo con vectores

El vector de entradas x es un vector fila. Si representamos el conjunto de coeficientes con un vector columna $w=[w_1, w_2, \ldots, w_{n+1}]^T$, entonces ***d(x)=Xw***. Notar que la salida de d(x) es un escalar.

Es común calcular la función lineal al mismo tiempo para todos los vectores de entrada. En ese caso, ***d(X)=Xw*** donde X es la matriz cuyas filas son los vectores de entrada y el resultado de d(x) es un vector columna.

$$\begin{bmatrix} x_1^1 & x_2^1 & x_3^1 & 1 \\ x_1^2 & x_2^2 & x_3^2 & 1 \\ x_1^3 & x_2^3 & x_3^3 & 1 \\ x_1^4 & x_2^4 & x_3^4 & 1 \\ x_1^5 & x_2^5 & x_3^5 & 1 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ w_4 \end{bmatrix} = \begin{bmatrix} d^1 \\ d^2 \\ d^3 \\ d^4 \\ d^5 \end{bmatrix}$$

Cálculo de la Matriz de Coeficientes

Los Vectores Patrones conocidos permiten calcular la **Matriz de Coeficientes.** Asumiendo que se deban clasificar patrones que corresponden a dos clases ω1 y ω2, y que cada clase contiene dos patrones de dimensión dos $\{x_1^1, x_2^1\}$ y $\{x_1^2, x_2^2\}$ donde los supra índices indican las clases ω1 y ω2, respectivamente. Si las clases son linealmente separables el problema es encontrar un vector $w = w_1, w_2, w_3$ tal que:

$$\begin{aligned} w_1 {x_{11}}^1 + w_2 {x_{12}}^1 + w_3 &> 0 \\ w_1 {x_{21}}^1 + w_2 {x_{22}}^1 + w_3 &> 0 \\ w_1 {x_{11}}^2 + w_2 {x_{12}}^2 + w_3 &< 0 \\ w_1 {x_{21}}^2 + w_2 {x_{22}}^2 + w_3 &< 0 \end{aligned}$$

Se desea que $d(x) > 0\ para\ \omega_1\ y\ d(x) < 0\ para\ \omega_2$, entonces se crea una salida esperada y para cada vector, donde

$$y = \begin{cases} +1 & si\ x \in \omega 1 \\ -1 & si\ x \in \omega 2 \end{cases}$$

y es un escalar, pero si se contempla la salida esperada para todos los vectores de entrada, se obtiene el vector columna y.

$$\begin{bmatrix} x_1^1 & x_2^1 & x_3^1 & 1 \\ x_1^2 & x_2^2 & x_3^2 & 1 \\ x_1^3 & x_2^3 & x_3^3 & 1 \\ x_1^4 & x_2^4 & x_3^4 & 1 \\ x_1^5 & x_2^5 & x_3^5 & 1 \end{bmatrix} \begin{bmatrix} y^1 \\ y^2 \\ y^3 \\ y^4 \\ y^5 \end{bmatrix}$$

Es decir que W es la solución del sistema lineal determinado por todos los patrones de ambas clases. Esta expresión para obtener los valores de W proviene de hacer mínimo del error cuadrático medio.

Al igual que en la regresión lineal, se buscan los coeficientes que minimizan el valor esperado del error. Para hallar el vector w óptimo se parte de la aproximación del error cuadrático medio:

$$S^2(\boldsymbol{w}) = \frac{\sum_{i=1}^{m} \left(y_i - \sum_{j=1}^{n+1} w_j x_{ji}\right)^2}{m} = E(|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}|^2)$$

donde m es la cantidad de vectores de entrenamiento y n cantidad de características.

Haciendo $\frac{\partial S^2(w)}{\partial w} = 0$ se llega a la siguiente expresión para el cálculo de los coeficientes (para vectores de entrada definidos como filas):

$$w = (X^T X)^{-1} X^T y$$

donde: w es el vector de pesos, X la matriz de entrada y y el vector de salida. T significa transpuesta

La matriz x se forma de los vectores clase 1 y clase 2 en las filas y luego, el vector patrón aumentado

$$\mathbf{w} = \left(\mathbf{x}\mathbf{x}^{\mathrm{T}}\right)^{-1}\mathbf{x}\mathbf{y}^{\mathrm{T}}$$

Ejemplo de Cálculo de la Matriz de Coeficientes

Consideramos el siguiente problema ya clásico en la literatura, relativo a la discriminación lineal de 2 clases de patrones bidimensionales. (Se requerirán al menos 3 vectores ya que no podemos tener menos ecuaciones que incógnitas)
Vectores de la Clase 1: (0 0), (0 1)
Vectores de la Clase 2: (1 0), (1 1)
Expandiendo los vectores:

$$X = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix} \qquad X' = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \qquad X'X = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 2 \\ 1 & 2 & 2 \\ 2 & 2 & 4 \end{bmatrix}$$

Calculamos la inversa:

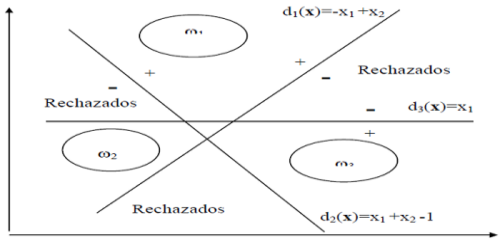
Un concepto importante a tener presente es que de acuerdo a lo visto, un sistema de clasificación lineal para clases separables puede ser calculado matemáticamente en forma precisa.

Esta posibilidad es **teóricamente** aplicable a casos de mayor complejidad, tales como clasificación no lineal, no separable y con dimensión de los patrones mayor que dos.

Sin embargo como se verá enseguida los cálculos necesarios aumentan notablemente. De modo que si bien es teóricamente posible calcular un clasificador cualquiera, generalmente es **impracticable**.

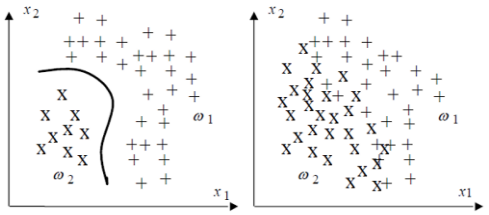
Este problema da lugar a la aplicación de métodos numéricos de solución para el Clasificador Polinomial, a métodos recursivos, como las Redes Neuronales, o a métodos de optimización más elaborados como los Clasificadores de Margen Óptimo o Máquina de soporte vectorial.

Extensión a Multiclases. Clasificación "Uno Contra Todos"



Pasar a este último esquema, partiendo del anterior, permite mantener la validez del concepto mostrado anteriormente por el cual, al presentar un patrón al clasificador ya optimizado, una función de decisión resultará ganadora y determinará la pertenencia a la clase.

Características de Problemas Reales



La situación que se encuentra usualmente en problemas reales es no ser posible separar las clases de una forma sencilla. Un caso muy frecuente y de mayor complejidad se tiene cuando los patrones de cada clase están a su vez distribuidos en múltiples clústeres. En la imagen, se representan dos clusters de patrones bidimensionales correspondientes a dos clases. Caso en que son separables en forma no-lineal (izquierda) y caso no separable (derecha).

Funciones de Decisión Generalizadas

Los límites de decisión pueden establecerse entre clases de modelos que no comparten patrones idénticos. La complejidad de estos límites va desde expresiones de carácter lineal a otras fuertemente no lineales, las que requieren un número grande de términos para su descripción. A menudo en aplicaciones prácticas las clases del modelo no son verdaderamente separables dentro de las limitaciones económicas o técnicas, y es deseable entonces buscar aproximaciones a las funciones de decisión. Una manera conveniente de generalizar el concepto de función de decisión lineal es considerar las funciones de decisión de la forma:

$$d(\mathbf{x}) = w_1 f_1(\mathbf{x}) + w_2 f_2(\mathbf{x}) + \ldots + w_k f_k(\mathbf{x}) + w_{k+1} = \sum_{i=1}^{k+1} w_i f_i(\mathbf{x}),$$

donde $\{f_i(x)\}, i = 1,2,\ldots,k$ son funciones reales simples del patrón x donde $f_{k+1}(x) = 1$ donde k+1 es el número de términos usados en la expansión. La última ecuación representa una infinita variedad de funciones de decisión, dependiendo de la elección de funciones $\{f_i(x)\}$ y del número de términos usados en la expansión. $f_i(x)$ es una función real simple del vector de entradas x. la cantidad de términos de la función generalizada es k+1, no n+1 como en la función de decisión lineal.

Considerando el hecho de que la ecuación d(x) puede representar funciones de decisión muy complejas, es posible tratar dichas funciones como si ellas fueran lineales en virtud de una transformación. Para mostrar esto, se define el vector x* cuyos componentes son las funciones $\{f_i(x)\}$ quedando

$$\mathbf{x}^* = \begin{pmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \\ \vdots \\ f_k(\mathbf{x}) \\ 1 \end{pmatrix}$$

Es decir: $d(x) = w'x^*$

Donde: $w = (w_1, w_2, \ldots, w_k, w_{k+1})'$

Una vez evaluadas, las funciones $\{f_i(x)\}$ no son nada más que un conjunto de valores numéricos, y x* es simplemente un vector K-dimensional el cual ha sido aumentado en 1.

Por consiguiente, $d(x) = w'x *$ representa una función lineal con respecto a los nuevos patrones x*. Claramente, si nosotros transformamos todos los patrones originales x en los patrones x* evaluando las funciones para todos los x, se obtiene una representación lineal. Esto da la posibilidad de restringir la discusión a la decisión lineal sin pérdida de generalidad.

Nada ha cambiado realmente en cuanto a que se conservan las propiedades no lineales de x , sin embargo si se ha aumentado la dimensión de los patrones, pasando a un espacio de patrones de dimensión K > n.

Clasificador polinomial:

Es un caso particular de la función de decisión generalizada. El caso más simple se da para x^2 donde la función de decisión toma la siguiente forma para un vector de dimensión 2

$$d(\mathbf{x}) = w_{11} x_1^2 + w_{12} x_1 x_2 + w_{22} x_2^2 + w_1 x_1 + w_2 x_2 + w_3$$

La que podría transformarse a forma lineal como:

$$d(x^*) = w' x^*$$

donde:

$$x^* = [x_1^2, x_1 x_2, x_2^2, x_1, x_2, 1]$$

$$w = [w_{11}, w_{12}, w_{22}, w_1, w_2, w_3]$$

Métricas

Matriz de confusión: Es una herramienta que permite la visualización del desempeño de un algoritmo de clasificación. Muestra la distribución de clasificaciones correctas e incorrectas en términos de TP, FP, TN y FN. *En las columnas se encuentran los valores predichos y en las filas los reales*. En el caso de una clasificación binaria:

	Predicción positiva	**Predicción negativa**
Caso positivo	***TP***	***FN***
Caso negativo	***FP***	***TN***

True positive (TP) es el número de datos correctamente clasificados en la clase positiva
False positive (FP) es el número de datos clasificados para estar en la clase positiva pero en realidad pertenecen a la clase negativa
True negative (TN) es el número de datos correctamente clasificados en la clase negativa
False negative (FN) es el número de datos clasificados para estar en la clase negativa pero en realidad pertenecen a la clase positiva.

Son predicciones dos son verdaderos y dos son falsos.

Las columnas o filas pueden invertirse, siempre y cuando se respeten los valores de las celdas. Regla mnemotécnica: La segunda letra (P o N) coincide en toda la fila si la predicción es por fila y en toda la columna si la predicción es por columna. La primera letra nunca cambia (T o F) y arma siempre una equis.

Factor de clasificación: $FC = \frac{a}{b}$, a hace referencia a la suma de elementos diagonales de M y b hace referencia a la suma de todos los elementos de M.

Accuracy: proporción de predicciones correctas realizadas por un modelo de inteligencia artificial con respecto al número total de predicciones. $Accuracy = \frac{TP + TN}{TP + TN + FN + FP}$

Recall (exhaustividad - sensibilidad): responde a la pregunta qué proporción de positivos se identificó como positivos, mejor valor 1, peor valor 0. El Recall de una clase expresa cuán bien puede el modelo detectar a esa clase. $Recall = \frac{TP}{TP + FN}$

Precisión: Responde a la pregunta qué proporción de identificaciones positivas fue correcta, mejor valor 1, peor valor 0. La Precisión de una clase define cuán confiable es un modelo en responder si un punto pertenece a esa clase. $Precisión = \frac{TP}{TP + FP}$

Especificidad: Dice que tan bueno es el modelo para detectar los negativos. También conocida como la Tasa de Verdaderos Negativos, ("true negative rate") o TN. Se trata de los casos negativos que el algoritmo ha clasificado correctamente. Expresa cuan bien puede el modelo detectar esa clase. Por arriba de 0.5 está acertando bien las clases (las sabe diferenciar), menos de 0.5 está clasificando al azar

$$Especificidad = \frac{TN}{N} == \frac{TN}{FP + TN}$$

F1 score: La puntuación F1 es la media armónica de la precisión y recall, donde la puntuación de la F1 alcanza su mejor valor en 1 (precisión y recall perfectas) y el peor en 0. $F1 = 2 * \frac{precisión * recall}{precisión + recall}$

La sensibilidad y la especificidad son dos valores que nos indican la capacidad de nuestro estimador para discriminar los casos positivos, de los negativos. La sensibilidad se representa como la fracción de verdaderos positivos, mientras que la especificidad, es la fracción de verdaderos negativos.

¿Por qué la media armónica? Dado que la media armónica de una lista de números se inclina fuertemente hacia últimos elementos de la lista, tiende (en comparación con la media aritmética) a mitigar el impacto de los grandes valores atípicos y a agravar el impacto de los pequeños. La medida F1 castiga más los valores extremos.

Idealmente, la medida F1 podría ser una métrica de evaluación efectiva en los siguientes escenarios de clasificación:

- Cuando los Falsos Positivos y la Falsos Negativos son igualmente costosos - lo que significa que se pasan verdaderos positivos o se encuentran falsos positivos - ambos

impactan el modelo casi de la misma manera, como por ejemplo en la detección de cáncer

- Añadir más datos no cambia el resultado de manera efectiva
- La tasa de TN, o sea los verdaderos negativos, son altos (como en las predicciones de inundaciones, predicciones de cáncer, etc.)

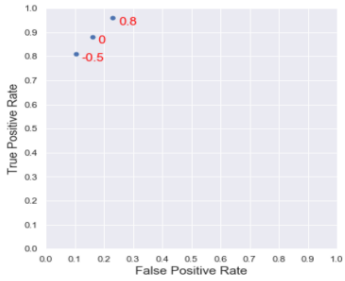
Métrica	Formula	Formula	Mejor valor	Significado
Accuracy **(Exactitud)**	$\frac{VP + VN}{VP + VN + FP + FN}$	$\frac{VP + VN}{VP + VN + FP + FN}$	1	Muestras bien clasificadas sobre el total de muestras
Recall **(Sensibilidad)** VPR Verdaderos positivos	$\frac{VP}{P}$	$\frac{VP}{VP + FN}$	1	Capacidad clasificador para detectar los casos **positivos**
FPR Falsos positivos	$\frac{FP}{N}$	$\frac{FP}{FP + VN}$	0	Proporción de los **negativos** etiquetados como **positivos**
Especifidad VNR = 1 - FPR	$\frac{VN}{N}$	$\frac{VN}{FP + VN}$	1	Proporción de los **negativos** etiquetados como **negativos**
Precision PPV Positive Predictive Value	$\frac{VP}{VP + FP}$	$\frac{VP}{VP + FP}$	1	Proporción de los **positivos** etiquetados como **positivos**

Tenemos cuatro casos posibles para cada clase:

- **Alta precisión y alto recall:** el modelo maneja perfectamente esa clase
- **Alta precisión y bajo recall:** el modelo no detecta la clase muy bien, pero cuando lo hace es altamente confiable.
- **Baja precisión y alto recall:** el modelo detecta bien la clase pero también incluye muestras de otras clases.
- **Baja precisión y bajo recall:** El modelo no logra clasificar la clase correctamente.

Cuando tenemos un dataset con desequilibrio, suele ocurrir que obtenemos un alto valor de precisión en la clase Mayoritaria y un bajo recall en la clase Minoritaria

Curva ROC (Receiver Operating Characteristic)



Para entender el concepto de curva ROC (Receiver Operating Characteristic) tenemos que revisar previamente cómo funciona un algoritmo de clasificación lineal con dos clases posibles, a la hora de realizar una predicción, el modelo da como resultado una puntuación (un "score") a cada muestra del conjunto de pruebas. Este score se compara con un cierto valor límite ("threshold"). Si, para una muestra, su score es mayor que el threshold, se asigna a la clase principal, si es menor, se asigna a la clase secundaria, por ejemplo, supongamos que el threshold es 0, si el score es mayor que el threshold se asigna a la clase 1 (principal), y si es menor, a la clase 0 (secundaria)

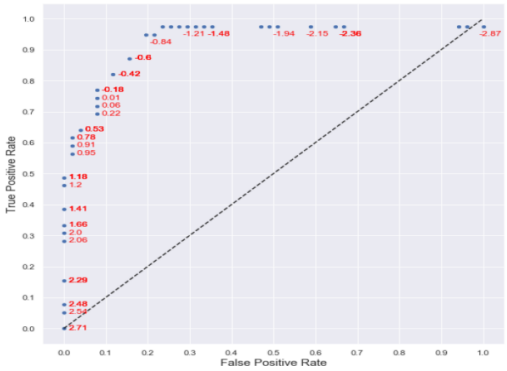
El clasificador con el threshold 0, supone un cierto VPR y un cierto FPR.

Si aumentamos el threshold hasta 0.8, estaríamos mejorando el VPR de nuestro modelo, es decir la capacidad del clasificador para detectar los casos positivos o sea disminuyendo el número de falsos negativos, pero a costa de empeorar el FPR, aumentando

el número de falsos positivos. Si disminuimos el threshold hasta - 0.5, vemos, estaríamos mejorando el FPR (disminuyendo el número de falsos positivos) a costa de empeorar el VPR de nuestro modelo, es decir la capacidad clasificador para detectar los casos positivos, aumentando el número de falsos negativos.

Recordemos que el mejor valor del FPR es 0 y el mejor valor del VPR es 1.

Hemos probado tres valores del threshold. Vamos a probar todos los thresholds desde el valor mínimo del score hasta el valor máximo.



Lo que estamos viendo es la curva ROC del clasificador que nos da bastante información: en primer lugar nos dice que aumentando o disminuyendo lo suficiente el threshold podemos conseguir que nuestro algoritmo de clasificación alcance un VPR o un FPR casi perfecto. El único "pero" es que lo conseguiremos a costa de empeorar la otra métrica. Esto significa que, una vez creado el modelo, podemos configurarlo en función de nuestros intereses fijando un threshold u otro (para conseguir un mayor VPR o un FPR). Por otro lado, cuanto más se aproxime la gráfica al punto (0, 1), más se comportará como un clasificador ideal. En la gráfica se muestra la línea diagonal que marca el comportamiento de los clasificadores que tienen tantos verdaderos positivos como falsos negativos, y tantos verdaderos negativos como falsos positivos. Muestra que tan bien el modelo está detectando los positivos. Plantea que tan bueno es el modelo prediciendo.

Si en la diagonal principal los valores son altos, quiere decir que el modelo es bueno. La curva ROC si acierta, crece. Es acumulativo. Es decir, crece verticalmente, para arriba. La curva no sigue porque no hay datos para probar.

Curva AUC: El área bajo la curva ROC (AUC, por sus siglas en inglés) es una métrica que cuantifica la capacidad del modelo para discriminar entre las clases. Cuantifica el desempeño global del modelo. Mide la capacidad de discriminación del clasificador (un valor más alto es mejor). Mientras más grande es la superficie, mejor es el modelo en general. Indica el área bajo la curva ROC. Por arriba de 0.6 o 0.7 es bueno el modelo. Un AUC de 1 indica un modelo perfecto, mientras que un AUC de 0.5 indica un modelo que no tiene mejor desempeño que un clasificador aleatorio.

La curva ROC (Receiver Operating Characteristic) y el valor de AUC (Area Under the Curve) son útiles para evaluar el rendimiento del clasificador.

Clasificador malo: Si la curva está debajo de la diagonal, indica que el clasificador tiene un rendimiento peor que adivinar al azar. Esto podría indicar que el modelo necesita mejoras o que las etiquetas están mal asignadas.

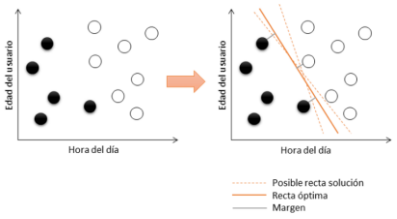
SVM Vector Support Machines (Máquinas de soporte vectorial)

Este método fue desarrollado en la década de los 90, dentro del campo de la ciencia computacional. Si bien originariamente se desarrolló como un método de clasificación binaria, su aplicación se extendió a problemas de clasificación múltiple y regresión. SVMs ha resultado ser uno de los mejores clasificadores para un amplio abanico de situaciones, por lo que se considera uno de los referentes dentro del ámbito del machine learning.

Clasificación

En este tipo de problemas, la solución es categórica, es decir, pasa por determinar una etiqueta para clasificar un elemento de una u otra forma.

Si se quiere encontrar qué tipo de usuario tiene más probabilidad de hacer clic en un determinado banner. La decisión implica varias variables: las características del propio usuario, su región geográfica, la tecnología empleada, día/hora en que se encuentra con el banner, etc. Si solo dos de estas variables fueran determinantes, podríamos encontrarnos en una situación similar a la de la imagen, donde el círculo negro identifica que el usuario hace clic, y el blanco que no. Con SVM podemos *obtener la "superficie óptima"* que delimitará el comportamiento clic – no clic de un usuario:

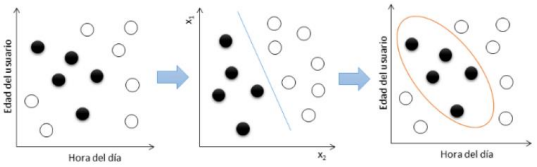


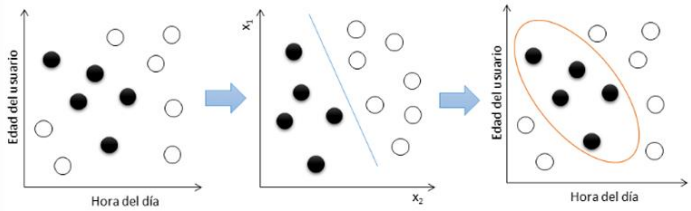
Es una "superficie" porque en un problema *bidimensional*, esta superficie será una recta, si hubiera tres dimensiones sería un plano y, en el caso N-dimensional… un hiperplano de la dimensión correspondiente. Y es "óptimo", ya que esta superficie minimiza el posible error de clasificación, maximizando la distancia desde el hiperplano al elemento más cercano de cada clase. Se utilizan los valores más cercanos a la recta porque son los que marcan la frontera entre una clase y otra (si se mueve la recta de más puede pasar que un elemento que es de una clase quede en otra clase). Se busca minimizar ese error.

Frontera de decisión: Todo lo que está dentro cumple con la clase y todo lo que está afuera no cumple con la clase o es de otra clase. Se puede tener varias fronteras de decisión.

¿Y si el problema no es lineal? SVM permite utilizar las llamadas **funciones *Kernel (no lineales)***. Estas funciones resuelven el problema de clasificación trasladando los datos a

un espacio donde el hiperplano solución es lineal y, por tanto, más sencillo de obtener. Una vez conseguido, la solución se transforma, de nuevo, al espacio original.





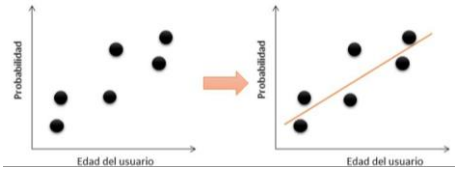
Con una serie de datos de prueba se obtendrá caracterizada la clasificación, es decir, la SVM ha sido entrenada. Y tras este entrenamiento, se consigue un modelo en base al que se podrá clasificar cualquier otro caso existente en el futuro.

Regresión

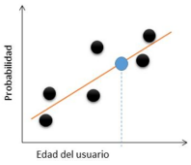
Si se quiere una solución numérica, por ejemplo, determinar qué probabilidad hay de que un usuario vuelva a un sitio web, si se quiere predecir el número de clics en el futuro, o qué cantidad de impresiones de un anuncio se tiene. *En este caso se está frente a un problema de regresión.*

La regresión se basa en buscar la curva que modele la tendencia de los datos y, según ella, predecir cualquier otro dato en el futuro.

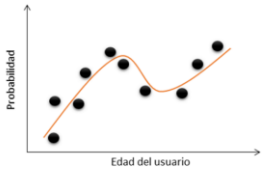
Por ejemplo, si la probabilidad de hacer clic en un determinado anuncio dependiera únicamente de la edad del usuario, se puede definir (siempre minimizando el error, como las SVM garantizan) una línea de tendencia:



De forma que podamos encontrar la respuesta (en este ejemplo, la probabilidad) para un nuevo caso:



En problemas no lineales siempre será posible utilizar una función tipo kernel que, tras resolver el problema en un espacio donde el mismo sea lineal, obtenga la curva que modele los datos.



En un espacio p-dimensional, un hiperplano se define como un s*ubespacio plano y afín de dimensiones p–1*. El término afín significa que el subespacio no tiene por qué pasar por el origen.

- En un espacio de dos dimensiones, el hiperplano es un subespacio de 1 dimensión, una recta.

- En un espacio tridimensional, un hiperplano es un subespacio de dos dimensiones, un plano convencional.
- Para dimensiones p > 3 no es intuitivo visualizar un hiperplano, pero el concepto de subespacio con p−1 dimensiones se mantiene.

En el caso de dos dimensiones, el hiperplano se describe acorde a la ecuación de una recta:

$$\beta_0 + \beta_1.x_1 + \beta_2.x_2 = 0$$

Dados los parámetros β0, β1 y β2, todos los pares de valores x = (x1, x2) para los que se cumple la igualdad son puntos del hiperplano. Esta ecuación puede generalizarse para p-dimensiones:

$$\beta_0 + \beta_1.x_1 + \beta_2.x_2 + \ldots + \beta_p.x_p = 0$$

De igual manera, todos los puntos definidos por el vector ($x=x_1,x_2, \ldots, x_p$) que cumplen la ecuación pertenecen al hiperplano.

Cuando x no satisface la ecuación, la ecuación puede ser:

$\beta_0 + \beta_1.x_1 + \beta_2.x_2 + \ldots + \beta_p.x_p < 0$ ó $\beta_0 + \beta_1.x_1 + \beta_2.x_2 + \ldots + \beta_p.x_p > 0$

El punto x cae a un lado o al otro del hiperplano. Así se puede entender que un hiperplano divide un espacio p-dimensional en dos mitades. Para saber en qué lado del hiperplano se encuentra un determinado punto x, hay que calcular el signo de la ecuación.

Clasificación binaria empleando un hiperplano

Cuando se dispone de n observaciones, cada una con p predictores y cuya variable respuesta tiene dos niveles (de aquí en adelante identificados como +1 y −1), se pueden emplear hiperplanos para construir un clasificador que permita predecir a qué grupo pertenece una observación en función de sus predictores. Este mismo problema puede abordarse también con otros métodos (regresión logística, árboles de clasificación…) cada uno con ventajas y desventajas.

Se tomará un espacio de dos dimensiones, donde un hiperplano es una recta. Sin embargo, los mismos conceptos son aplicables a dimensiones superiores.

<u>Casos perfectamente separables linealmente</u>

Si la distribución de las observaciones se puede separar linealmente de forma perfecta en las dos clases (+1 y −1), entonces, un hiperplano de separación cumple que:

$$\beta_0 + \beta_1.x_1 + \beta_2.x_2 + \ldots + \beta_p.x_p > 0 \quad \text{si} \quad y_i = 1$$

$$\beta_0 + \beta_1.x_1 + \beta_2.x_2 + \ldots + \beta_p.x_p < 0 \quad \text{si} \quad y_i = -1$$

Al identificar cada clase como +1 o −1, y dado que si se multiplican dos valores negativos se obtiene uno positivo, las condiciones pueden simplificarse en una única:

$$y_i . (\beta_0 + \beta_1.x_1 + \beta_2.x_2 + \ldots + \beta_p.x_p) > 0 \quad \text{si} \quad i = 1 \ldots n$$

El clasificador más sencillo consiste en asignar cada observación a una clase dependiendo del lado del hiperplano en el que se encuentre. Es decir, la observación x* se clasifica acorde al signo de la función:

$$f(x*) = \beta_0 + \beta_1.x_1 + \beta_2.x_2 + \ldots + \beta_p.x_p$$

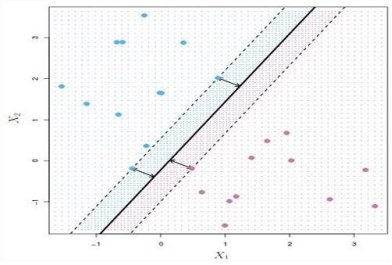
Si $f(x*)$ es positiva, la observación se asigna a la clase +1, si es negativa, a la clase −1. Además, la magnitud de $f(x*)$ permite saber cómo de lejos está la observación del hiperplano y con ello la confianza de la clasificación. Si un punto está lejos del hiperplano, la clasificación será de mayor confianza, es decir, si está lejos es más probable que la clasificación sea correcta.

La definición de hiperplano para casos perfectamente separables linealmente resulta en un número infinito de posibles hiperplanos, por lo que se necesita un método que permita seleccionar uno de ellos como clasificador óptimo.

Se selecciona como clasificador óptimo al que se conoce como *maximal margin hyperplane o hiperplano óptimo de separación*, que se corresponde con el hiperplano que se encuentra más alejado de todas las observaciones de entrenamiento.

Para obtenerlo, se tiene que calcular la distancia perpendicular de cada observación a un determinado hiperplano. La menor de estas distancias (margen) determina como de alejado está el hiperplano de las observaciones de entrenamiento. El hiperplano óptimo de separación se define como el hiperplano que consigue un mayor margen, es decir, que la distancia mínima entre el hiperplano y las observaciones es lo más grande posible.

Esta idea no es posible aplicarla, ya que habría infinitos hiperplanos contra los que medir las distancias. En su lugar, se recurre a métodos de optimización.



Los puntos más cercanos a la línea son los que van a servir para trazar la línea óptima.

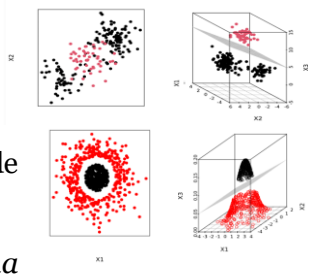
Las tres observaciones equidistantes respecto al maximal margin hyperplane se encuentran a lo largo de las líneas discontinuas que indican la anchura del margen. A estas observaciones se les conoce como *vectores soporte*, ya que son vectores en un espacio p dimensional y soportan (definen) el maximal margin hyperplane. Cualquier modificación en estas observaciones conlleva cambios en el maximal margin hyperplane. Sin embargo, **modificaciones en observaciones que NO son vector soporte NO tienen impacto alguno en el hiperplano.**

Máquinas de Vector Soporte sin separabilidad lineal

El Support Vector Classifier descrito consigue buenos resultados cuando el límite de separación entre clases es aproximadamente lineal. Si no lo es, su capacidad decae drásticamente. Una estrategia para enfrentarse a escenarios en los que la separación de los grupos es de tipo no lineal consiste en expandir las dimensiones del espacio original.

El hecho de que los grupos no sean linealmente separables en el espacio original no significa que no lo sean en un espacio de mayores dimensiones.

Ejemplo donde dos grupos, cuya separación en dos dimensiones no es lineal, sí lo es al añadir una tercera dimensión. Tener una dimensión más es agregar otro atributo para clasificarlo. La clasificación es descriptiva, no es para un objetivo. Pero si en base a la clasificación después se pueden plantear objetivos de optimización.



El método de *Máquinas Vector Soporte (SVM) se puede considerar como una extensión del Support Vector Classifier* obtenida al aumentar la dimensión de los datos. Los límites de separación lineales generados en el espacio aumentado se convierten en límites de separación no lineales al proyectarlos en el espacio original.

Aumento de la dimensión, kernels

Una vez definido que las Máquinas de Vector Soporte siguen la misma estratégica que el Support Vector Classifier, pero aumentando la dimensión de los datos antes de aplicar el algoritmo, se debe determinar cómo se aumenta la dimensión y qué dimensión es la correcta.

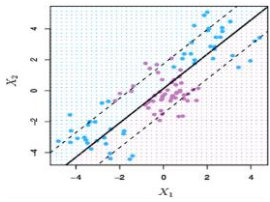
La dimensión de un conjunto de datos puede transformarse combinando o modificando cualquiera de sus dimensiones, para transformar un espacio de dos dimensiones en uno de tres se aplica:

$$f(x_1, x_2) = (x^2, \sqrt{2}.x_1.x_2, x_2^2)$$

¿Cómo saber qué transformación es la adecuada? *Con los kernel.* Un kernel (K) es una función que devuelve el resultado del *dot product* entre dos vectores realizado en un nuevo espacio dimensional distinto al espacio original en el que se encuentran los vectores.

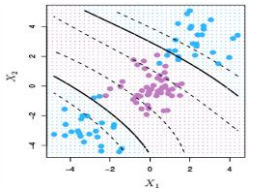
Si se sustituye este dot product por un kernel, se obtienen directamente los vectores soporte (y el hiperplano) en la dimensión correspondiente al kernel. A esto se lo conoce como kernel trick, porque, con solo una ligera modificación del problema original, con los kernels, se puede obtener el resultado para cualquier dimensión.

Tipos de Kernel



- Kernel lineal: Si se emplea un Kernel lineal, el clasificador Support Vector Machine obtenido es equivalente al Support Vector Classifier.

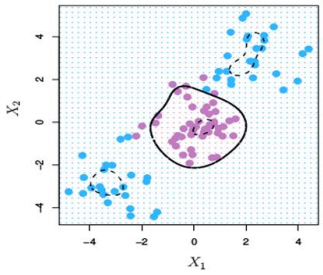
$$K(x, x') = x \cdot x'$$



- Kernel polinómico: Cuando se emplea d=1 y c=0, el resultado es el mismo que el de un kernel lineal. Si d>1, se generan límites de decisión no lineales, aumentando la no linealidad a medida que aumenta d. No suele ser recomendable emplear valores de d mayores 5 por problemas de overfitting (sobre entrenamiento).

$$K(x, x') = (x \cdot x' + c)^d$$

- Gaussian Kernel (Radial Basis Function): Permite hacer los círculos. El valor de γ controla el comportamiento del kernel, cuando es muy pequeño, el modelo final es equivalente al obtenido con un kernel lineal, a medida que aumenta su valor, también lo hace la flexibilidad del modelo. Tiene dos ventajas:



    - Solo tiene dos hiper parámetros que optimizar (γ y la penalización C común a todos los SVM).
    - Su flexibilidad puede ir desde un clasificador lineal a uno muy complejo.

      $K(x, x') = exp(-\gamma \mid\mid x - x' \mid\mid^2)$

Cada uno de los kernel tiene una serie de hiper parámetros cuyo valor óptimo puede encontrarse mediante validación cruzada. No puede decirse que haya un kernel que supere al resto, depende de la naturaleza del problema que se esté tratando.

Costo de error: El costo de error de clasificar depende del problema. Se debe determinar el valor de c que dice cuál es el costo que sea de una clase u otra. En base a eso se realiza la curva para minimizar los errores y que sea menos costoso. Si se erra en la clasificación de una clase puede que el costo no sea tan alto como clasificarlo de forma errónea en otra clase.

Sobreentrenamiento: Un modelo sobreentrenado se puede utilizar para un problema específico, si se cambia el set de datos no realiza correctamente la clasificación (no tiene margen para aceptar nuevos datos). A veces es mejor tener un margen de error aceptable y que esté más generalizado para utilizarlo en varios conjuntos de datos.

Algunas formas de reducir el sobreentrenamiento

- Early-stop: Detener el entrenamiento cuando el rendimiento en el conjunto de validación deja de mejorar, evitando sobreajuste.
- Aumentar la cantidad de datos de entrenamiento: Añadir más datos que ayudan a generalizar mejor el modelo y reducir el riesgo de sobreajuste.
- Regularización: Penaliza modelos demasiado complejos, restringiendo los valores de los pesos para evitar sobreajuste. Formas de **regularización** utilizadas para reducir el sobreajuste:
    - **L1 (Regularización Lasso)**: Agrega un término que penaliza la suma de los valores absolutos de los pesos w_i. Promueve la generación de pesos pequeños o incluso cero, lo que conduce a un modelo más simple.

$$L1: L(x, y) = E^2(x, y) + \lambda \sum_{i+1}^{n} |w_i|$$

- **L2 (Regularización Ridge)**: Penaliza la suma de los cuadrados de los pesos w_i, empujándolos hacia valores más pequeños pero sin llevarlos a cero, manteniendo todos los parámetros involucrados.

$$L2: L(x, y) = E^2(x, y) + \lambda \sum_{i+1}^{n} {w_i}^2$$

- Dropout: Apaga aleatoriamente neuronas durante el entrenamiento para evitar que el modelo dependa demasiado de ciertas características. Previene la co-adaptación desmedida de las neuronas.

Redes Neuronales

Las redes neuronales son más que otra forma de emular ciertas características propias de los humanos, como la capacidad de memorizar y de asociar hechos. Si se examinan con atención aquellos problemas que no pueden expresarse a través de un algoritmo, se observará que todos ellos tienen una característica en común: la experiencia. El hombre es capaz de resolver estas situaciones acudiendo a la experiencia acumulada. Así, una forma de aproximarse al problema consiste en la construcción de sistemas que sean capaces de reproducir esta característica humana.

Las redes neuronales son un modelo artificial y simplificado del cerebro humano. Una red neuronal es "un nuevo sistema para el tratamiento de la información, cuya unidad básica de procesamiento está inspirada en la célula fundamental del sistema nervioso humano, la neurona".

Todos los procesos del cuerpo humano se relacionan en alguna u otra forma con la (in)actividad de estas neuronas. Las mismas son un componente relativamente simple del ser humano, pero cuando se conectan en forma conjunta se hacen muy poderosas. Los humanos son capaces de aprender, es decir que aquellos problemas que inicialmente no pueden resolverse pueden ser resueltos después de obtener más información acerca del problema.

Las Redes Neuronales consisten de unidades de procesamiento que intercambian datos o información. Se utilizan para reconocer patrones, incluyendo imágenes, manuscritos y secuencias de tiempo, tendencias financieras. Pueden definirse como:

- Un sistema de computación compuesto por un gran número de elementos simples, elementos de procesos muy interconectados, los cuales procesan información por medio de su estado dinámico como respuesta a entradas externas.
- Redes neuronales artificiales: redes interconectadas masivamente en paralelo de elementos simples (usualmente adaptativos) y con organización jerárquica, las cuales intentan interactuar con los objetos del mundo real del mismo modo que lo hace el sistema nervioso biológico.

Ventajas que ofrecen las redes neuronales

Debido a su constitución y fundamentos, las redes neuronales artificiales presentan un gran número de características semejantes a las del cerebro.

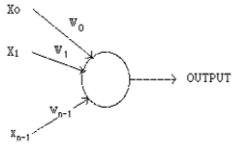
Son capaces de aprender de la experiencia, de generalizar de casos anteriores a nuevos casos, de abstraer características esenciales a partir de entradas que representan información irrelevante, etc. Esto hace que tengan numerosas ventajas y que este tipo de tecnología se esté aplicando en múltiples áreas.

- *Aprendizaje Adaptativo:* capacidad de aprender a realizar tareas basadas en un entrenamiento o en una experiencia inicial.
- *Auto-organización:* una red neuronal puede crear su propia organización o representación de la información que recibe mediante una etapa de aprendizaje.
- *Tolerancia a fallos:* la destrucción parcial de una red conduce a una degradación de su estructura, pero, algunas capacidades se pueden retener, incluso con un gran daño.
- *Operación en tiempo real:* los cómputos neuronales pueden ser realizados en paralelo; para esto se diseñan y fabrican máquinas con hardware especial para obtener esta capacidad.
- *Fácil inserción dentro de la tecnología existente:* se pueden obtener chips especializados para redes neuronales que mejoran su capacidad en ciertas tareas. Ello facilitará la integración modular en los sistemas existentes.

Algunas aplicaciones de redes neuronales artificiales son: Reconocimiento de caracteres impresos, Reconocimiento de caracteres manuscritos, Memorias asociativas, Reconocimiento de voz, Control de robots, Toma de decisiones, Reconocimiento de enfermedades cardiacas, Reconocimiento de señales de radio, Generación de reglas para sistemas expertos.

El Neurón Artificial

Es un elemento procesador al que se aplican un conjunto de entradas, cada una representando la salida de otro neurón. Estas entradas llamadas X, se multiplican por un peso (número real) asociado a ellas, llamadas W.



$$OUTPUT = F\left(\sum_{i=0}^{n-1} Xi\ Wi\right)$$

F es una función de activación

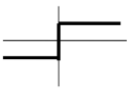
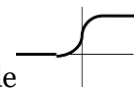
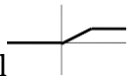
Donde:

$$NET = \sum_{i=0}^{n-1} X_i \, W_i$$

La suma ponderada se "filtra" por medio de una función, llamada función de activación: $OUT = F(NET)$

Se dice que un neurón "enciende" si su valor de salida es mayor o igual a un valor umbral, el cual es determinado por los posibles valores de la función de activación.

Las funciones de activación pueden ser las siguientes:

- Escalón 
- sigmoide 
- Lineal con umbral 
- tangente hiperbólica 

Características de las redes neuronales

Existen cuatro aspectos que caracterizan una red neuronal: su topología, el mecanismo de aprendizaje, tipo de asociación entre la información de entrada y de salida, y la forma de representación de estas informaciones.

- *Topología: Monocapa y Multicapa*. Consiste en la organización y disposición de las neuronas en la red formando capas o agrupaciones de neuronas. Los parámetros fundamentales de la red son: número de capas, número de neuronas por capa, grado de conectividad y tipo de conexión entre neuronas. Se clasifican en:
    - Redes monocapa: se establecen conexiones laterales entre las neuronas que pertenecen a la única capa que constituye la red. Ejemplos red de HOPPFIELD. Las redes monocapa se utilizan en tareas relacionadas con la autoasociación; por ejemplo, para regenerar informaciones de entrada que se presentan como incompletas o distorsionadas.
    - Redes multicapa: disponen las neuronas agrupadas en varios niveles. Dado que este tipo de redes disponen de varias capas, las conexiones entre neuronas pueden ser del tipo feedforward (conexión hacia adelante) o del tipo feedback (conexión hacia atrás).
- *Mecanismo de aprendizaje:* El aprendizaje es el proceso por el cual una red neuronal modifica sus pesos en respuesta a una información de entrada. Los cambios que se producen durante la etapa de aprendizaje se reducen a la destrucción (el peso de la conexión toma el valor 0), modificación y creación (el peso de la conexión toma un valor distinto de 0) de conexiones entre las neuronas. Se puede considerar que el proceso de aprendizaje ha terminado cuando los valores de los pesos permanecen estables, mediante un número fijo de ciclos o cuando el error es menor que el error inicialmente establecido. Un aspecto importante es determinar los

criterios de la regla de aprendizaje; cómo se van a modificar los pesos. Se tienen dos mecanismos de aprendizaje, supervisado y no supervisado:

La diferencia entre ambos está en la existencia o no de una agente externo que controle todo el proceso. Otro criterio para diferenciar las reglas de aprendizaje se basa en considerar si la red puede aprender durante su funcionamiento (aprendizaje ON LINE) o requiere de una fase previa de entrenamiento (aprendizaje OFF LINE). En este último debe existir un conjunto de datos de entrenamiento y un conjunto de datos de test o prueba; los pesos de las conexiones no se modifican después de terminar la etapa de entrenamiento. En la red ON LINE los pesos varían dinámicamente cada vez que se presenta una nueva información al sistema.

- Redes con aprendizaje supervisado: El proceso de aprendizaje se realiza mediante un entrenamiento controlado por un agente externo (supervisor, maestro) que determina la respuesta que debería generar la red a partir de una entrada determinada. El supervisor comprueba la salida generada por el sistema y en el caso de que no coincida con la esperada, se modificarán los pesos de las conexiones. Los parámetros se ajustan de forma tal que la salida del clasificador se aproxime a una salida esperada conocida. En este tipo de aprendizaje se distinguen tres formas de llevarlo a cabo:
    - *Por corrección de error:* Ajusta los pesos de las conexiones de la red en función de la diferencia entre los valores deseados y los obtenidos en la salida. Algoritmos que lo utilizan son: Regla de aprendizaje del perceptrón (utilizada en la red PERCEPTRON), Regla delta generalizada (utilizada en redes multicapa).
    - *Por refuerzo:* Este tipo de aprendizaje es más lento que el anterior y se basa en la idea de no disponer de un ejemplo completo del comportamiento deseado; es decir, de no indicar durante el entrenamiento la salida exacta que se desea que proporcione la red ante una determinada entrada. La función del supervisor se reduce a indicar mediante una señal de refuerzo si la salida obtenida en la red se ajusta a la deseada (éxito = +1 o fracaso = -1) y en función de ello se ajustan los pesos basándose en un mecanismo de *probabilidades*.
    - *Estocástico:* Consiste en realizar cambios aleatorios en los valores de los pesos y evaluar su efecto a partir del objetivo deseado y de distribuciones de probabilidad. Una red que utiliza este tipo de aprendizaje es la red Boltzman Machine y la red Cauchy Machine desarrollada por Szu en 1986.
- Redes con aprendizaje NO supervisado: Estas redes, que no dependen de influencia externa para ajustar los pesos entre sus neuronas, se autoorganizan sin recibir información del entorno sobre la corrección de sus salidas. Identifican características, regularidades, correlaciones o categorías en los datos de entrada, generando una salida

basada en esos patrones. Los parámetros se ajustan para encontrar relaciones o agrupaciones entre los valores de las características. La red de salida genera:

- La salida representa el grado de familiaridad o similitud entre la información de entrada y las informaciones mostradas con anterioridad.
- Clusterización o establecimiento de categorías, indicando la red a la salida a qué categoría pertenece la información de entrada, siendo la propia red la que debe establecer las correlaciones oportunas.
- Codificación de los datos de entrada, generando a la salida una versión codificada con menos bits, pero manteniendo la información relevante de los datos.
- Mapeo de características, obteniéndose una disposición geométrica que representa un mapa topográfico de las características de los datos de entrada.

Tipo de asociación entre las informaciones de entrada y salida

Las redes neuronales son sistemas que almacenan cierta información aprendida; esta se registra de forma distribuida en los pesos asociados a las conexiones entre neuronas. Hay que establecer cierta relación o asociación entre la información presentada a la red y la salida ofrecida por esta. *Es lo que se conoce como memoria asociativa*. Existen dos formas realizar esta asociación entrada/salida y que generan dos tipos de redes:

- *Redes heteroasociativas:* La red aprende parejas de datos [(A1,B1), (A2,B2), …, (An,Bn)], de tal forma que cuando se le presente determinada información de entrada Ai responda con la salida correspondiente Bi. Al asociar informaciones de entrada con diferentes informaciones de salida, precisan al menos de 2 capas, una para captar y retener la información de entrada y otra para mantener la salida con la información asociada. Si esto no fuese así se perdería la información inicial al obtenerse la salida asociada; es necesario mantener la información de entrada para acceder varias veces a ella, por lo que debe permanecer en la capa de entrada. *El aprendizaje de este tipo de redes suele ser supervisado.*
- *Redes autoasociativas:* La red aprende ciertas informaciones A1, A2, … ,An de forma que cuando se le presenta una información de entrada realizará una autocorrelación, respondiendo con uno de los datos almacenados, el más parecido al de entrada. Este tipo de redes pueden implementarse con una sola capa de neuronas. E*l tipo de aprendizaje utilizado es el no supervisado* y se utiliza en tareas de filtrado de información para la reconstrucción de datos, eliminando distorsiones o ruido, explorar relaciones entre informaciones similares para facilitar la búsqueda por contenido en bases de datos y para resolver problemas de optimización.

Representación de la información de entrada y salida

- *Redes continuas:* En un gran número de redes, tanto los datos de entrada como de salida son analógicos (valores reales continuos y normalmente normalizados, por lo que su valor absoluto será menor que la unidad). En este caso las funciones de activación de las neuronas serán también continuas, del tipo lineal o sigmoidal.
- *Redes discretas:* Estas redes sólo admiten valores discretos [0,1] a la entrada, generando también en la salida respuestas de tipo binario. *La función de activación en este caso es del tipo escalón.*
- *Redes híbridas:* La información de entrada es continúa pero a la salida ofrecen información binaria.

Perceptrón monocapa

Un perceptrón está formado por varias neuronas lineales para recibir las entradas a la red y una neurona de salida; es capaz de decidir cuando una entrada presentada a la red pertenece a una de las clases que es capaz de reconocer. Ante un conjunto de entradas, tengo una salida.

La única neurona de salida del Perceptrón realiza la suma ponderada de las entradas y pasa el resultado a una función de transferencia. *La regla de decisión es responder +1 si el patrón presentado pertenece a la clase A, o -1 si el patrón pertenece a la clase B.*

Regla de Aprendizaje: El algoritmo de aprendizaje es del *tipo supervisado*, lo cual requiere que sus resultados sean evaluados y se realicen las modificaciones del sistema si fuera necesario.

Modelos de redes neuronales

- Modelo biológico
- Modelo matemático - Perceptrón. La salida del perceptrón se define como:

$$y = f(\sum_{i=0}^{N} w_i x_i - \theta)$$

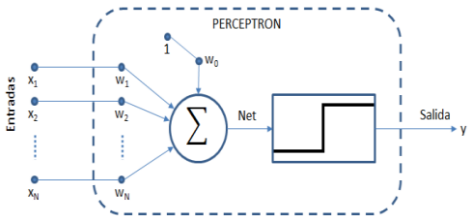
donde:

x_i = *es el elemento i del vector de entrada* $x = [x_1, x_2, \ldots, x_n]$

w_i = *es el elemento i del vector de pesos* $w = [w_1, w_2, \ldots, w_n]$

Perceptrón con vector de entradas aumentado

La salida del perceptrón se define como: $y = f(\sum_{i=0}^{N} w_i x_i)$ donde $w_0 = -\theta$



Entrenamiento: Para ajustar los parámetros internos (pesos) del perceptrón se utiliza un método de aprendizaje supervisado, por lo tanto es necesario contar con datos en la forma:

x_0	x_1	x_2	x_3	...	x_N	y'
1	0.14	0.24	0.34	...	0.55	1
1	0.11	0.89	0.33	...	0.9	1
1	0.97	0.27	0.34	...	0.53	0
1	0.93	0.36	0.72	...	0.75	1
1	0.92	0.15	0.29	...	0.97	0
1	0.58	0.31	0.52	...	0.14	0
1	0.29	0.52	0.54	...	0.88	1
1	0.58	0.59	0.51	...	0.06	1

donde d es la salida esperada (o deseada) y los pesos de adaptan según la ley de aprendizaje: $\Delta_{wi} = \alpha\,(d - y)\,xi$

Por lo tanto: $w_{nuevo} = w_{viejo} + \alpha\,(d - y)\,xi$

α *es la tasa de aprendizaje*

w_{nuevo}: *es el nuevo valor de los pesos*

w_{viejo}: *es el nuevo valor de los actual*

y: *salida calculada por el perceptron*

Los pasos del método son:

1. Definir los valores de α
2. Inicializar los pesos con valores aleatorios
3. Mientras error > 0 para algún vector de entradas, hacer:
    a. Tomar un vector entrada/salida del set de datos
    b. Calcular la salida $y = f(\sum_{i=0}^{N} w_i x_i)$
    c. Calcular el $error = y' - y$
    d. Calcular $\Delta_{wi} = \alpha(y' - y)x_i$
    e. Modificar los pesos haciendo $w_{i\,t+1} = w_{i_t} + \Delta_{wi}$

Tal como está definido el método y a menos que se defina una cantidad máxima de ciclos, el entrenamiento termina cuando todos los vectores están bien clasificados.

Condiciones de corte

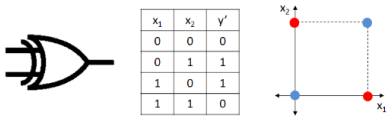
- Todos los errores sean 0
- Luego de cierta cantidad de iteraciones definida
- Hasta que los errores sean menores a un valor establecido

Luego de llegar a la condición de corte, se puede trazar la recta con los pesos obtenidos y dicha recta es la que separa una clase de otra.

Desventaja del perceptrón

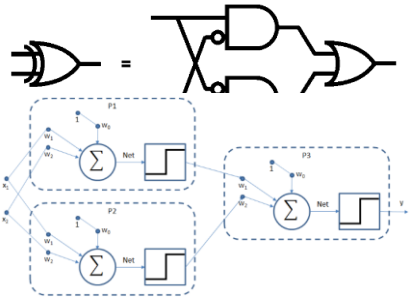
Al igual que el clasificador lineal, el perceptrón no puede clasificar correctamente si las clases no son linealmente separables.

- *Problema de la XOR:* Este problema NO se puede solucionar, el perceptrón NO puede resolver el problema, sin embargo, hay una alternativa teórica que involucra otro modelo y abre un camino para tratar este tipo de problemas. Conectando perceptrones entre sí, en lo que se llama perceptrón multicapa, se podría resolver el problema si supiéramos cómo adaptar los pesos. El valor de este modelo es únicamente educativo.

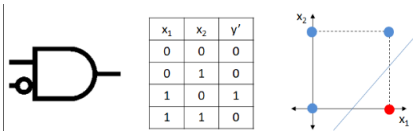


x_1	x_2	y'
0	0	0
0	1	1
1	0	1
1	1	0

- *Problema de la XOR con perceptrón multicapa:* Una compuerta XOR se puede construir con otras compuertas. El enfoque para tratar el problema con perceptrones es entrenar perceptrones individuales para que funcionen como cada una de las compuertas del circuito y después conectarlos.

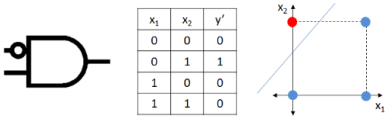


  - Primero hacemos que el perceptrón P1 funcione como la compuerta C1 (AND con la segunda entrada negada)



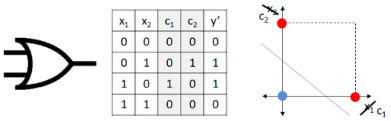
x_1	x_2	y'
0	0	0
0	1	0
1	0	1
1	1	0

  - Lo mismo con el perceptrón P2, lo entrenamos para que funcione como C2 (AND con la primera entrada negada)



x_1	x_2	y'
0	0	0
0	1	1
1	0	0
1	1	0

  - Por último, entrenamos el perceptrón P3 para que funcione como la compuerta C3 (OR)



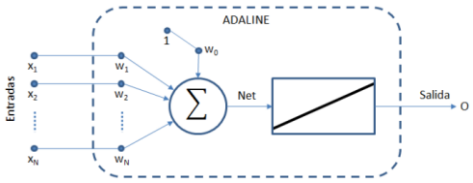
x_1	x_2	c_1	c_2	y'
0	0	0	0	0
0	1	0	1	1
1	0	1	0	1
1	1	0	0	0

El último perceptrón resuelve correctamente el problema porque trabaja en un espacio distinto. Este nuevo espacio $c1 - c2$ fue creado por los dos perceptrones de la primera capa.

¿Por qué no se puede aplicar este modelo? No sabemos cómo entrenarlo. No sabemos cómo modificar los pesos de los perceptrones de la primera capa porque no sabemos cuánto contribuyó cada uno al error de la última salida final.

Adaline

Adaline es un tipo de neurona con dos diferencias importantes respecto al perceptrón:



Cambia el mecanismo de aprendizaje, se utiliza la regla delta, basada en el mínimo error cuadrático medio (Least Mean Squared o LMS error). Adaline tiene una función de activación lineal.

Considerando:

- *O:* salida de la red (por output). En Adaline O: $\sum_{i=0}^{N} w_i x_i = Net$
- y: salida esperada
- k: identificador de un vector de entrenamiento.

El error cuadrático se define como: $E = \frac{1}{2L} \sum_{k=1}^{L} (y_k - O_k)^2$

donde L es la cantidad de vectores de entrenamiento.

La adaptación de los pesos se hace a través de una búsqueda sobre la superficie del error. Para encontrar el mínimo de la función del error los pesos se modifican en cantidades proporcionales al gradiente decreciente de la función E.

Ley de aprendizaje de Adaline o regla delta:

$$\Delta_{w_i} = -\alpha \frac{\partial E_k}{\partial w_i}$$

Aplicando la regla de la cadena:

$$\frac{\partial E_k}{\partial w_i} = \frac{\partial E_k}{\partial O_k} \frac{\partial O_k}{\partial Net_k} \frac{\partial Net_k}{\partial w_i}$$

$$\frac{\partial E_k}{\partial O_k} = \frac{\partial \left(\frac{1}{2}(y_k - O_k)^2\right)}{\partial O_k} = \frac{1}{2} 2(y_k - O_k)(-1) = -(y_k - O_k)$$

$$\frac{\partial O_k}{\partial Net_k} = 1$$

$$\frac{\partial Net_k}{\partial w_i} = \frac{\partial \left(\sum_{i=1}^{N} w_i x_{ki}\right)}{\partial w_i} = \frac{\partial (w_0 x_0 + \cdots + w_i x_i + \cdots + w_N x_N)}{\partial w_i} = \frac{\partial (w_i x_i)}{\partial w_i} = x_i$$

Es común llamar $\delta = \frac{\partial E_k}{\partial O_k} \frac{\partial O_k}{\partial Net_k}$

La regla delta queda deninida como:

$$\Delta_{w_{ik}} = -\alpha \delta x_i$$

$$= -\alpha (-(y_k - O_k)) x_i$$

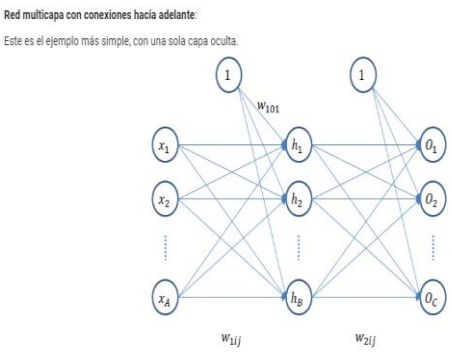
$$= \alpha (y_k - O_k) x_i$$

Finalmente, la ley de aprendizaje de Adaline, $\Delta w_{ik} = \alpha(y_k - O_k)x_i$ es igual a la del perceptrón, pero no hay que confundirse, el comportamiento no es el mismo porque el error puede tomar cualquier valor en R.

Método de entrenamiento: El método de entrenamiento también es similar al del perceptrón, con la diferencia de que el error nunca es cero, así que se necesita otra condición de corte. Típicamente se corta el entrenamiento cuando se llega a un umbral de error previamente definido o cuando el error se estabiliza.

Backpropagation: es un método para entrenar redes neuronales multicapas. La regla delta ha sido extendida (regla delta generalizada o backpropagation) a redes con capas intermedias con

conexiones hacia adelante (feedforward networks) y cuyas neuronas tienen funciones de activación continuas, no decrecientes y derivables.



¿Para qué sirve entrenar redes multicapa? Para clasificar datos que no sean linealmente separables.

donde i es la i-ésima entrada de la neurona j .

Notar que:

- No hay conexiones entre neuronas de la misma capa.
- No hay conexiones hacia neuronas de capas anteriores.
- Cada capa puede tener distintas cantidades de neuronas.
- La imagen corresponde a una red de tres capas (una capa oculta, la capa de salida y la de entrada).

Funciones de activación

Recordar que un Adaline, o cualquier neurona artificial antes de aplicar la función de activación, realiza una transformación lineal. Entonces, una red de n capas formadas por neuronas lineales se puede reemplazar por una red de una sola capa y, por lo tanto, no puede resolver problemas no linealmente separables. Por esta razón es necesario agregar no-linealidades entre las capas.

La función de activación no lineal más utilizada es la función sigmoidal:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Resolución Redes Perceptrón

- Primera iteración

RESOLUCIÓN DE EJERCICIO TIPO PARA PERCEPTRON SIMPLE (MONOCAPA)

Un perceptron está formado por varias neuronas lineales para recibir las entradas a la red y una neurona de salida; es capaz de decidir cuando una entrada presentada a la red pertenece a una de las clases que es capaz de reconocer.

La única neurona de salida del Perceptron realiza la suma ponderada de las entradas y pasa el resultado a una función de transferencia. La regla de decisión es responder +1 si el patrón presentado pertenece a la clase A, o -1 si el patrón pertenece a la clase B.

Regla de Aprendizaje:
El algoritmo de aprendizaje es del tipo supervisado, lo cual requiere que sus resultados sean evaluados y se realicen las modificaciones del sistema si fuera necesario.

EJERCICIO 4.a

Dado un perceptron en una red neural de una capa, con dos entradas y una salida, ajuste los pesos asociados a cada entrada para que la salida de este sistema responda según la función AND de las entradas.

La función de activación o transferencia está dada por:

$$Y = \begin{cases} 1 \text{ si } u > 0 \\ 0 \text{ si } u \leq 0 \end{cases}$$

Siendo $u = \sum w_i \cdot x_i = w_1 * x_1 + w_2 * x_2 + w_3$

Factor de ganancia $\alpha = 1$

ENTRADAS		SALIDA
X1	X2	DESEADA
0	0	0
0	1	0
1	0	0
1	1	1

PRIMER PASO: INICIALIZAR LOS PESOS

$w_3 = 1,5$ $w_1 = 0$ $w_2 = -2,5$

SEGUNDO PASO: PRESENTAR ENTRADAS

$x_3 = 1$ (umbral) $x_1 = 0$ $x_2 = 0$ salida deseada: 0

TERCER PASO: CALCULAR LA SALIDA ACTUAL Y ERROR

$Y_{(t)} = f[\sum w_{i(t)} \cdot x_{i(t)}]$

Y= f [(1,5* 1) + (0 * 0) + (- 2,5 * 0)]

Y = f [1,5] > 0

Y = 1 como la salida deseada es 0 y la salida calculada es 1 debemos calcular el error y modificar los pesos.
(aprendizaje por corrección de error)

Error = Salida deseada – salida calculada

Error = $[d_{(t)} - y_{(t)}]$

- 1 = 0 – 1

CUARTO PASO: ADAPTACIÓN DE LOS PESOS

$w_{i\,(t+1)} = w_{i\,(t)} + \alpha\,[\,d_{(t)} - y_{(t)}\,]\,.\,x_{i\,(t)}$

$w_3 = 1{,}5 + 1\,.\,[\,-1\,]\,.\,1 = 0{,}5$

$w_1 = 0 + 1\,.\,[\,-1\,]\,.\,0 = 0$

$w_2 = -2{,}5 + 1\,.\,[\,-1\,]\,.\,0 = -2{,}5$

- Segunda iteración

PRIMER PASO: INICIALIZAR LOS PESOS

$w_3 = 0{,}5 \qquad w_1 = 0 \qquad w_2 = -2{,}5$

SEGUNDO PASO: PRESENTAR ENTRADAS

$x_3 = 1$ (umbral) $\qquad x_1 = 0 \qquad x_2 = 1 \qquad$ salida deseada: 0

TERCER PASO: CALCULAR LA SALIDA ACTUAL Y ERROR

$Y = f\,[\,(0{,}5 * 1) + (0 * 0) + (-2{,}5 * 1)\,]$

$Y = f\,[\,-2\,] \leq 0$

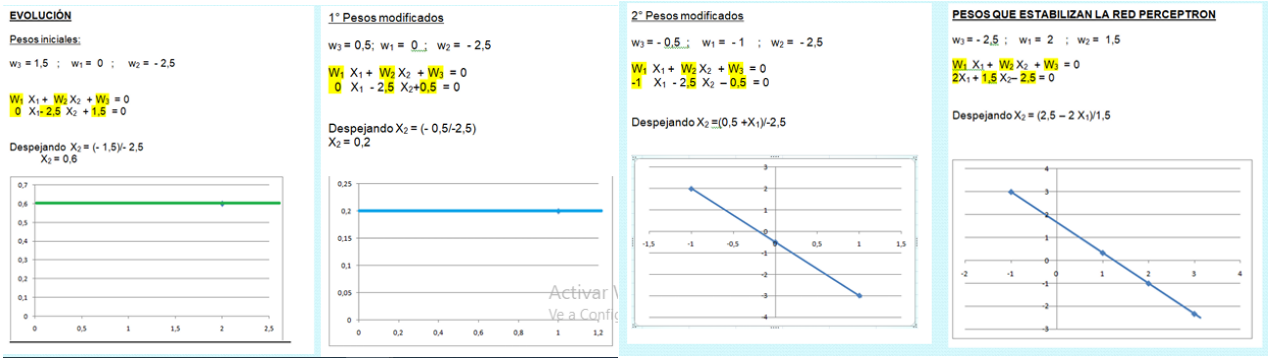
$Y = 0$

Error = Salida deseada – salida calculada

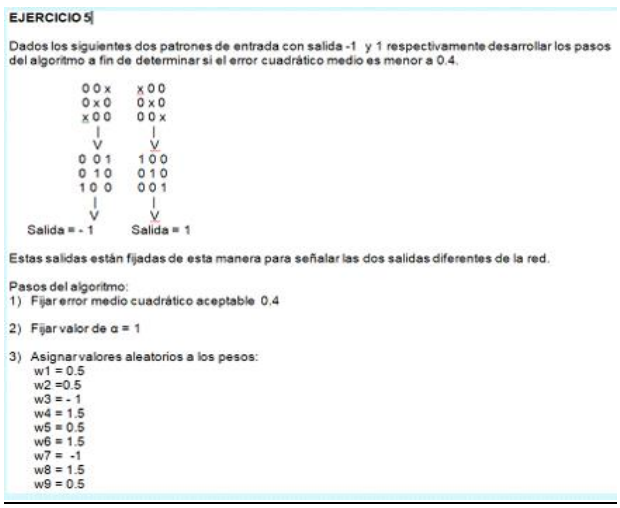
0 – 0 = 0 como no hay error los pesos no se actualizan y continúo con la entrada siguiente.

Aclaraciones:

- Cada iteración cambia las entradas
- Si hay error, se adaptan los pesos. Al adaptarlos, para la siguiente entrada se utilizan esos pesos “actualizados”. Si hay error, se sigue iterando hasta que todos los errores sean cero y allí se dice que se estabilizó la red.



Resolución Redes Adaline



4) Presentar vector de entrada:
0 0 1 0 1 0 1 0 0

5) Obtener la salida lineal (función rampa) de la red:

$$Sk = \sum_{j=0}^{N} wj\ xkj$$

S1= 0 (0.5) + 0 (0.5) + 1 (-1) + 0 (1.5) + 1 (0.5) + 0 (1.5) + 1 (-1) + 0 (1.5) + 0 (0.5) = -1.5

Calcular el error ε1 = Sd – Sr = (-1 – (-1.5)) = 0.5

6) Actualizar pesos:

$w(t+1) = w_{1(t)} + \alpha \cdot \varepsilon_1 \cdot x_1$
w1(t+1) = 0.5 + 1[0.5] 0 = 0.5
w2(t+1) = 0.5 + 1[0.5] 0 = 0.5
w3(t+1) = -1 + 1[0.5] 1 = -0.5
w4(t+1) = 1.5 1[0.5] 0 = 1.5
w5(t+1) = 0.5 + 1[0.5] 1 = 1
w6(t+1) = 1.5 + 1[0.5] 0 = 1.5
w7(t+1) = -1 + 1[0.5] 1 = -0.5
w8(t+1) = 1.5 + 1[0.5] 0 = 1.5
w9(t+1) = 0.5 + 1[0.5] 0 = 0.5

Con los pesos modificados probamos con la siguiente entrada

Con los pesos modificados probamos con la siguiente entrada

1) Entrada 1 0 0 0 1 0 0 0 1

2) Obtener la salida lineal (función rampa)

$$Sk = \sum_{j=0}^{N} wj\ xkj$$

S2= 1 (0.5) + 0 (0.5) + 0 (-0.5) + 0 (1.5) + 1 (1) + 0 (1.5) + 0 (-0.5) + 0 (1.5) + 1 (0.5) = 2

Calcular el error ε1 = Sd – Sr = (1 – 2) = -1 Hay error, se deben actualizar los pesos, pero antes verificar si no está dentro del error cuadrático medio aceptado.

Calcular el error cuadrático medio $< \varepsilon^2 > = 1/2L \sum_{K=1}^{L} \varepsilon^2_k$

$< \varepsilon^2 > = 1 / 2 (2) ((0.5)^2 + (-1)^2) = ¼ (0.25 +1) = 1.25/4 = 0.3125$

Está dentro del error cuadrático medio aceptado, la red está estable.

Aprendizaje profundo

REDES NEURONALES: PERCEPTRÓN → REDES NEURONALES → DEEP LEARNING

Deep learning

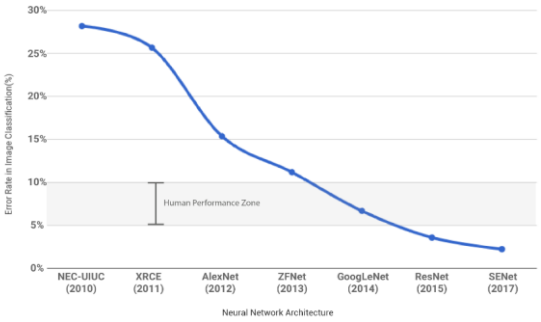
El aprendizaje profundo imita la forma en que los humanos aprenden a través de la experiencia. Es un conjunto de herramientas de aprendizaje automático que logra entrenar redes neuronales con varias capas de profundidad.

Se basa en redes neuronales que consisten en capas de neuronas artificiales, donde cada capa extrae características más abstractas a partir de los datos de la capa anterior. Estas capas **están ocultas**, con pesos desconocidos, y pueden aprender representaciones complejas directamente a partir de los datos. *No se conoce qué parámetros utiliza cada capa para aprender a partir de los datos, pero se entrenan igual.*

También se puede considerar como un tipo muy poderoso de aprendizaje automático, la reencarnación moderna de las redes neuronales artificiales, un conjunto de funciones matemáticas simples y entrenables, el estado del arte para la mayoría de los desafíos que enfrenta el reconocimiento de patrones (como visión artificial, reconocimiento de audio, procesamiento del lenguaje natural (NLP), sistemas de diagnóstico médico, predicción de terremotos, uso de energía, valores de mercado, etc.), o una "amenaza" otras áreas de la IA.

Resultados por año del ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

Se utiliza para reconocimiento facial.

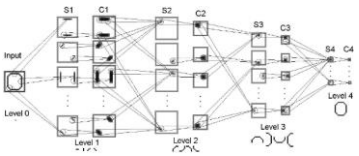


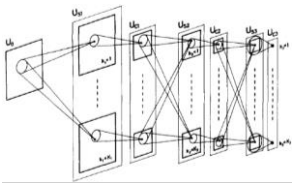
Tienen más cantidad de datos de entrenamiento para el reconocimiento, porque comienzan desde 0 para el reconocimiento. Los algoritmos mejoran pero se deben entrenar con muchos más datos. El ser humano no necesita datos de entrenamiento desde cero porque tiene reconocimiento innato de patrones.

Inicios

Inspiración biológica

- En 1981 Hubel y Wiesel ganan el premio nobel por sus estudios (durante los '60) sobre la estructura y función de la corteza visual.
- Detección de formas simples y modelo jerárquico del sistema nervioso visual
- Modelo propuesto por Fukushima en 1980, Neocognitron
- Modelo propuesto por Fukushima en 1980, Neocognitron





Algunos hitos históricos

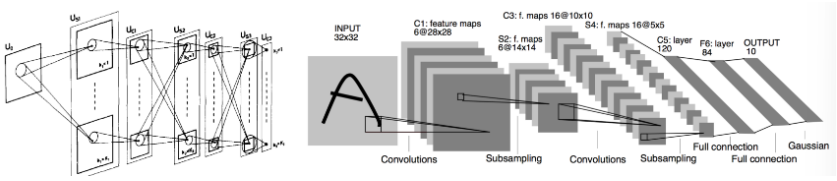
- En 1989 LeCun et al aplican backpropagation a las capas de convolución del modelo de Fukushima.
- En 1991 Hochreiter descubre que la dificultad para entrenar redes neuronales de muchas capas con backpropagation se debe al desvanecimiento del gradiente.
- En 2009 un modelo de aprendizaje profundo ganó por primera vez una competencia internacional de reconocimiento de patrones.

Independencia

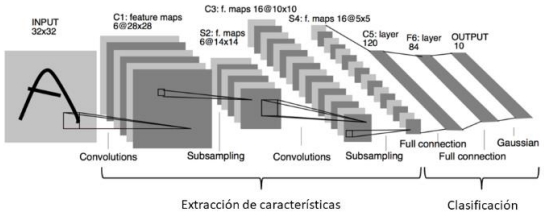
- El aprendizaje profundo se transforma en un diseño de ingeniería
- Alejamiento de la biología.
- Alejamiento de la matemática los matemáticos.

End-to-End

- Neocognitron y LeNet5 (1998)



- Los modelos de aprendizaje profundo incluyen la etapa de extracción de características.



- Proveer una solución de extremo-a-extremo es una de las mayores diferencias entre el aprendizaje profundo y el resto del aprendizaje automático. Se supone también que es la gran ventaja.

Características

- *Capas Múltiples:* Las redes neuronales profundas tienen varias capas (a menudo cientos o miles) que permiten modelar datos de alta dimensionalidad.
- *Aprendizaje Jerárquico:* Cada capa de la red aprende una representación más compleja y abstracta de los datos de entrada.
- *Necesidad de grandes Cantidades de Datos:* El aprendizaje profundo generalmente requiere grandes volúmenes de datos para entrenar las redes neuronales.
- *Alta Capacidad Computacional:* Debido a su complejidad, el aprendizaje profundo necesita hardware especializado, como GPUs (unidades de procesamiento gráfico), para entrenar los modelos de manera eficiente.
- *Autonomía en la Extracción de Características:* Las redes neuronales profundas pueden aprender y extraer automáticamente las características más relevantes de los datos sin intervención humana.

Funcionamiento

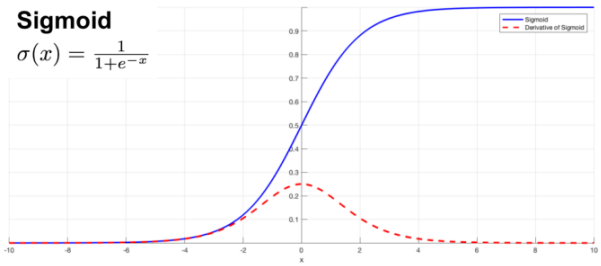
- *Entrenamiento:* El modelo de red neuronal se entrena utilizando un conjunto de datos etiquetados, donde se ajustan los pesos de las conexiones entre las neuronas para minimizar la diferencia entre las predicciones del modelo y los valores reales. Se realiza a través de un *algoritmo de optimización como el descenso de gradiente*.
- *Propagación hacia adelante:* Durante el proceso de predicción, los datos se pasan a través de las capas de la red (desde la entrada hasta la salida). Cada neurona calcula una salida a partir de una combinación lineal de sus entradas, seguida de una función de activación no lineal.

- *Propagación hacia atrás:* Durante el entrenamiento, después de calcular la salida, se ajustan los pesos mediante retropropagación, un proceso que calcula el error y lo distribuye hacia atrás a través de la red para actualizar los pesos.
- *Validación y evaluación:* Una vez entrenado, el modelo se valida y evalúa utilizando un conjunto de datos de test para medir su rendimiento y ajustar hiperparámetros si es necesario.

Uno de los problemas más importantes de estas redes que surge durante la fase de entrenamiento es el desvanecimiento del gradiente, especialmente en aquellas con muchas capas, como las redes recurrentes (RNNs) y las convolucionales (CNNs) muy profundas.

Desvanecimiento de gradiente

El desvanecimiento del gradiente ocurre cuando, durante el proceso de retropropagación (backpropagation), los gradientes que se utilizan para actualizar los pesos de la red se vuelven extremadamente pequeños a medida que se propagan hacia las capas iniciales de la red. Como resultado/consecuencia, los pesos de las primeras capas de la red no se actualizan significativamente ni de manera efectiva, llevando a un entrenamiento muy lento o a que la red no aprenda.



Se elige la neurona que minimiza más el error (la que tiene mayor desvanecimiento de gradiente) para optimizar el proceso de aprendizaje de la red. Llega un momento en donde el error se aplana, si pasa eso la red deja de aprender.

El desvanecimiento del gradiente es un problema, se busca evitar que el gradiente se desvanezca (se haga muy pequeño). Un gradiente mayor significa que los pesos de la neurona se pueden actualizar de manera más significativa, lo que permite un aprendizaje más efectivo. Al elegir neuronas con gradientes mayores, se está seleccionando aquellas que tienen un mayor impacto en la reducción del error de la red.

Dificultad en el Entrenamiento de Redes Profundas

Cuanto más profunda sea la red, mayor es la probabilidad de que el problema de desvanecimiento del gradiente ocurra, lo que hace que sea difícil entrenar redes profundas de manera efectiva.

Soluciones:

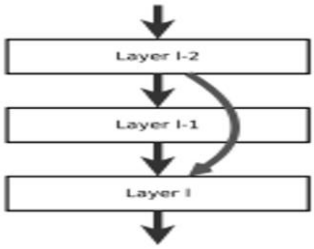
- *Funciones de activación diferentes:* Utilizar funciones de activación como ReLU (Rectified Linear Unit) o sus variantes (Leaky ReLU, ELU), que no saturan, es decir, no tienen regiones donde sus derivadas se vuelvan muy pequeñas tan fácilmente y, por lo tanto, no tienden a provocar desvanecimiento del gradiente.



- *Inicialización de pesos adecuada:* Técnicas de inicialización de pesos que permiten que los gradientes se mantengan en un rango más adecuado durante la propagación hacia atrás. La inicialización de los pesos puede ser aleatoria, debe ser de forma que los gradientes no sean 0.
- *Normalización de Batch (Batch Normalization):* Ayuda a estabilizar el aprendizaje al normalizar las salidas de cada capa antes de que pasen a la siguiente, lo que puede reducir la posibilidad de desvanecimiento del gradiente. De manera estadística, normalizando entre 0 y 1.
- *Redes residuales*

Redes residuales

Las redes residuales o ResNets (Residual Networks) son un tipo de arquitectura de red neuronal profunda introducida para superar el problema del desvanecimiento del gradiente y facilitar el entrenamiento de redes muy profundas.



El concepto central es la introducción de conexiones de salto o skip connections. Estas conexiones permiten que la información y los gradientes se salten una o más capas, facilitando el entrenamiento y ayudando a mitigar problemas como el desvanecimiento del gradiente.

Características

- *Facilidad de Entrenamiento:* Las conexiones de salto permiten que las redes residuales se entrenen con muchas más capas que las redes tradicionales, sin sufrir de los problemas de desvanecimiento del gradiente. Al tener miles de capas, va saltando entre ellas para entrenarlas. Se determina a qué capa salta mediante un algoritmo.
- *Prevención de degradación:* En redes muy profundas, añadir capas puede empeorar el rendimiento debido a la dificultad en el entrenamiento. Las ResNets previenen este problema permitiendo que la red aprenda funciones de identidad más fácilmente, si lo necesita, haciendo que la adición de capas no degrade el rendimiento.
- *Redes Extremadamente Profundas:* Las ResNets han permitido el entrenamiento exitoso de redes con más de 100 capas, algo que era prácticamente imposible antes de su introducción.

Funcionamiento

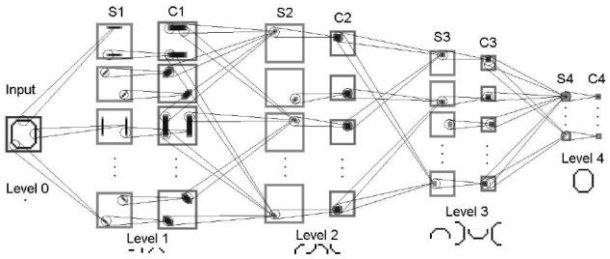
- *Bloque Residual Básico:* En su forma más simple, un bloque residual consiste en dos capas (generalmente convolucionales) seguidas de una función de activación, para la capa de salida.
- *Bloques Residuales con Más Complejidad:* En implementaciones más avanzadas, un bloque residual puede incluir más capas y procesos de normalización.

Neocognitron

El Neocognitron es considerado uno de los primeros modelos de redes neuronales profundas y precursor de las redes neuronales convolucionales (CNNs) modernas que se utilizan en visión por computadora.

Fue diseñado para emular el proceso de reconocimiento de patrones en el sistema visual de los animales. Inspirado en la estructura del córtex visual, el modelo es capaz de aprender a reconocer patrones visuales, como caracteres escritos a mano, de manera jerárquica. Fue una de las primeras redes en introducir conceptos como la organización en capas, la convolución, y la invariancia a traslaciones, fundamentales en las CNNs modernas.

El Neocognitron se organiza en múltiples capas, cada una de las cuales extrae características a partir de la salida de la capa anterior.



Se entrena con imágenes, pero la forma es de clasificación. El problema se subdivide y cada capa detecta una porción de la imagen e identifica ciertas características. En el ejemplo de la imagen, las primeras capas detectan líneas, las segundas semicurvas y el tercer nivel curvas completas.

A su vez también se pueden detectar colores, para ello se debe agregar un nivel más que reconozca colores. Se complejiza más la red. Para detectar cosas más complejas, se combinan más y más capas, cada una con una funcionalidad específica, aunque al estar ocultas no se conoce qué parámetro analiza cada una. Se pueden armar redes neuronales con bloques previamente armados. Se arma la red a medida y no neurona por neurona.

Hay dos tipos principales de capas en este modelo

- *Capas S (Simple Cells - Capas simples):* Estas capas están diseñadas para detectar características locales de la imagen de entrada, como bordes y texturas. Son análogas a las células simples en el córtex visual de los mamíferos. *Cada célula en una capa S responde a un patrón específico en una pequeña región de la entrada.*

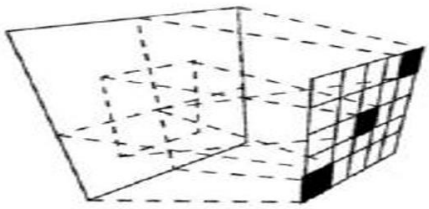
- *Capas C (Complex Cells - Capas complejas):* Las capas C generalizan las características detectadas por las capas S, es decir, combinan la información de varias células S logrando invariancia (independencia) a la posición y tamaño, es decir, una célula C activada puede reconocer un patrón sin importar su ubicación exacta dentro de la región. Hace que a la red no le afecte los cambios de los lugares donde estén los bordes.

  Otros tipos de capas

  - Densamente conectadas
  - Convolucionales
  - Pooling (MaxPooling, AveragePooling)
  - Recurrentes - Recursivas
  - LSTM

Funcionamiento

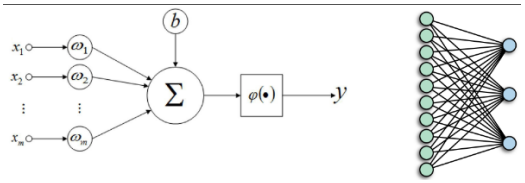
- *Convolución:* Las capas S aplican operaciones de convolución sobre la imagen de entrada para extraer características locales, como bordes o texturas.
  - Toma una ventana y se tiene una máscara con determinados valores (ya definida y sirve para definir o encontrar bordes). La máscara se va corriendo para obtener distintos valores para obtener los bordes.



  - Se opera con los valores de la máscara y la subárea de la imagen original sobre la que se posiciona en cada iteración. El resultado genera un único valor que ocupa una de las posiciones de la imagen resultante.
  - Se puede extender la imagen (con distintas técnicas: Todos ceros, repitiendo la imagen) para disminuir la pérdida de información. Ej.: Imagen de 6x6 y kernel 3x3 va a generar un resultado de 4x4. Si amplío la imagen con alguna técnica a 8x8 mi resultado sigue siendo 6x6 y no perdí tanta información de los bordes. Depende del contexto.
  - Se va moviendo según el paso que se defina. También depende del contexto y de cuanto peso se le quiere dar a cada parte de la imagen. Pasos más chicos generan que se utilice más de una vez una celda, por lo que incide más sobre el resultado final. Termina cuando se analizó toda la imagen original.
  - Cada capa de convolución está compuesta por varios kernels que se especializan en detectar distintos patrones.

- *Invariancia:* Las capas C integran las respuestas de las células S en una región más amplia, permitiendo que la red sea insensible a la ubicación exacta del patrón (invariancia a traslaciones).
- *Aprendizaje Jerárquico:* A medida que la información se propaga a través de las capas S y C, el Neocognitron construye representaciones más abstractas y generales de los patrones, permitiendo reconocer formas más complejas en niveles superiores.
- *Clasificación:* Finalmente, en las capas superiores, las combinaciones de características detectadas en las capas anteriores se utilizan para tomar decisiones de clasificación, como reconocer un número o una letra escrita a mano.

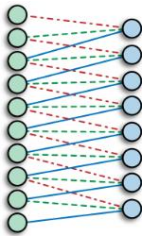
Redes densas (dense, feed forward, fully connected)



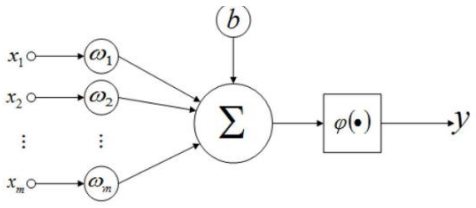
Descubren características generales. La salida se calcula como la aplicación de la función de activación sobre la suma ponderada de las entradas. Las neuronas se conectan con todas las de la capa anterior. No hay conexiones entre neuronas de la misma capa. Las clasificaciones normalmente están densamente conectadas

Convolución

Una convolución es una transformación sobre una matriz que permite modificar los datos iniciales en otro conjunto de datos que pueda procesarse. La salida se calcula como la aplicación de la función de activación sobre la suma ponderada de las entradas (Tiene una máscara). Las neuronas tienen un campo perceptivo acotado. No hay conexiones entre neuronas de la misma capa. Los pesos se comparten entre neuronas de la misma capa y kernel (tipos de convoluciones/distintas fórmulas). El kernel/máscara se va aplicando sucesivamente en los distintos elementos de los datos iniciales.



NEURONA



Entradas (x) - Pesos (ω) - Término independiente (b/bias) - Sumatoria de pesos (Σ) - Funcion de activacion (φ) - Salida (y)

Convolución 1D

Una función de convolución unidimensional es una transformación que se aplica al multiplicar un vector de origen por una máscara y obtener de esta manera un resultado, sumando cada uno de los resultados parciales.

Los resultados de estas operaciones se colocan en un valor de salida que es el resultado del vector de entrada, alterado por la máscara.

Para aplicar un filtro de convolución f al vector g, el proceso consiste en desplazar el filtro f a lo largo del vector g y calcular el producto punto (suma de productos) en cada posición.

$$g * f = vector\ resultante$$

La convolución 1D es un vector lineal y utiliza datos como por ejemplo señales de audio.

Pasos

1. Se tiene el vector y la máscara. Al vector de entrada, se le aplica la máscara para obtener la matriz transformada.
2. Al aplicar la máscara sobre el vector, se obtiene un solo resultado.
3. Al aplicar sucesivas veces la máscara sobre el resultado se va obteniendo nuevos resultados pero con la información anterior.

Convolución 2D

La convolución discreta (o simplemente convolución), al igual que la convolución de funciones, es un operador matemático. Por lo tanto no tiene sólo una aplicación, sino que, de la misma forma que el producto o la derivación, se puede utilizar con distintos propósitos prácticos.

Por lo que: Sea W una ventana de tamaño $n \times m$ (según la vecindad definida) sobre una imagen y M una máscara (matriz) del mismo tamaño. La convolución digital entre W y M se define como:

$$conv = \sum_{i=1}^{n}\sum_{i=1}^{m} W_{ij}M_{ij}$$

donde $z = 1a + 2b + 3c + 4d + 5e + 6f + 7g + 8h + 9p$

siendo M:

$M =$

a	b	c
h	p	d
g	f	e

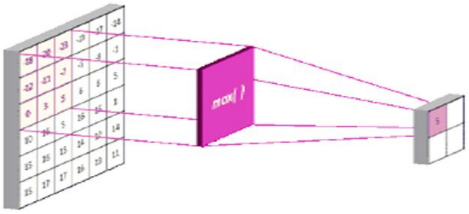
Se superponen las máscaras para tener contexto. Se van tomando las informaciones de aplicar la máscara y va obteniendo información del paso anterior.

Alternativas para completar con valores:

- Se puede agregar 0 para que no agregue relevancia en el cálculo (se hace padding)
- Se puede repetir el mismo valor que el inicial, para que tenga el mismo peso.
- Utilizar circularidad (el primer valor se coloca en el último lugar y el último valor se coloca en el primer lugar). Se asume que hay una repetición de valores.

No hay necesidad de mantener siempre el tamaño del vector. Es mejor agregar valores que ir achicando el vector/matriz en cada convolución, ya que si se van iterando se van perdiendo cada vez más datos hasta que no se pueda seguir trabajando.

Pooling



En las redes convolucionales (CNN) el pooling es una operación que se aplica a las *salidas de una capa convolucional* para reducir las dimensiones espaciales de las características extraídas, manteniendo la información más importante. El pooling es crucial para hacer las redes *más eficientes* y reducir la cantidad de parámetros y el costo computacional, además de introducir invariancia a pequeñas traslaciones en los datos de entrada. Busca que sea más chica la imagen de los elementos a analizar. Las imágenes son capas de procesamiento. Se superponen datos para no perder ningún dato de la imagen original. Cada capa procesa la información y va achicando el set de datos.

Tipos de Pooling

- *Max Pooling:* Es el tipo más común. Detecta características más importantes y deja las menos importantes. Se espera una frontera definida (por ej. bordes). En cada región de la matriz de características, se selecciona el valor máximo. Ejemplo: Si la región de pooling es 2x2, se toma el valor máximo de esos 4 elementos.
    - *Propósito:* Captura la característica más prominente dentro de la región, lo que puede ser útil para detectar la presencia de ciertas características clave en diferentes posiciones.
- *Average Pooling*: Calcula el promedio de los valores dentro de la región de pooling. Difumina y capta áreas de importancia (zonas) y distinto a las otras zonas. Ejemplo: Si la región de pooling es 2x2, se calcula el promedio de esos 4 elementos.
    - *Propósito:* Mantiene la suavidad de las características, siendo útil en situaciones donde la preservación de la información de textura es importante.
- *Global Pooling:* Es una variante especial que aplica un pooling (max o average) sobre toda la matriz de características, reduciendo la salida a un solo valor por canal.
    - *Propósito:* Muy útil en las etapas finales de las redes, justo antes de la clasificación, donde se quiere condensar toda la información en un conjunto muy pequeño de características.

Son complementarias, se espera que una capa esté contigua a la otra. Normalmente es mejor primero detectar zonas y luego las fronteras se esas zonas, pero depende de la imagen que se quiera detectar.

También se puede utilizar para reconocer patrones, como patrones de audio por ejemplo, para detectar distintas voces.

Funcionamiento del pooling

- *Reducción dimensional:* El pooling se realiza sobre una ventana o región específica de la matriz de características, que se mueve (o se desliza) sobre la totalidad de la matriz, similar a una convolución. El tamaño de la ventana y el stride (desplazamiento de la ventana) determinan el grado de reducción dimensional.
- *Invariancia a traslaciones:* El pooling contribuye a que la red sea menos sensible a cambios pequeños en la posición de las características detectadas en la entrada. Por ejemplo, si una característica aparece en una posición ligeramente diferente en una imagen, el pooling ayuda a asegurar que la red aún la reconozca.
- *Control de sobreajuste:* Al reducir la dimensionalidad, el pooling ayuda a simplificar el modelo, reduciendo el riesgo de sobreajuste al eliminar detalles innecesarios o redundantes.

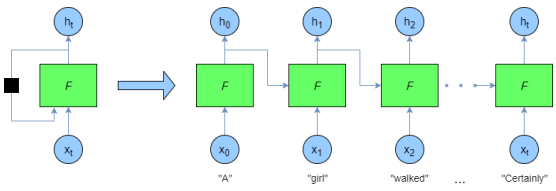
Aplicaciones

El pooling es utilizado principalmente para:

- *Clasificación de imágenes:* Se busca extraer características robustas que no dependan de la posición exacta de los objetos en la imagen.
- *Detección de objetos:* Para reducir las dimensiones de las características extraídas y permitir que la red se concentre en los patrones globales. No va al detalle, sino que generaliza.

Redes neuronales recurrentes

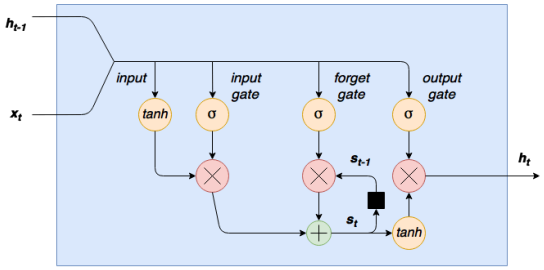
Las redes neuronales recurrentes (RNN) son un tipo de red neuronal diseñada para trabajar con datos secuenciales.



El concepto clave es la recurrencia, que permite que la red procese una secuencia de datos un paso a la vez, mientras mantiene un estado interno que captura la información relevante de pasos anteriores en la secuencia. Esto las hace adecuadas para tareas donde el orden de los datos es importante, como el procesamiento del lenguaje natural, predicción de series temporales, y reconocimiento de voz.

Normalmente se usa para el reconocimiento de texto. Cada neurona se alimenta de la anterior (Lo usa ChatGpt) de forma recursiva, por lo que cada neurona analiza un segmento del texto e intenta detectar lo siguiente. Muchas veces se llaman así mismas para tener información de contexto.

Redes LSTM - Long Short Term Memory



Las redes LSTM son un tipo avanzado de red neuronal recurrente (RNN) diseñada específicamente para abordar las limitaciones de las RNNs básicas, especialmente el problema del desvanecimiento del gradiente. Tienen la capacidad de añadir y eliminar información del estado de la célula.

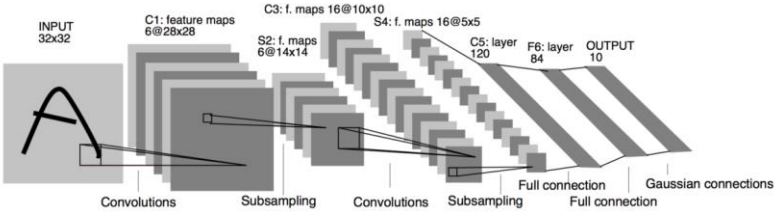
Las LSTM están diseñadas para "recordar" información durante largos periodos de tiempo y "olvidar" información irrelevante, lo que las hace mucho más efectivas que las RNNs estándar para aprender dependencias a largo plazo en los datos secuenciales. Son redes neuronales con memoria, lo que la red aprende, no lo olvida. Se le puede agregar información y la red responderá tomando todo lo aprendido hasta el momento como contexto.

Ventajas

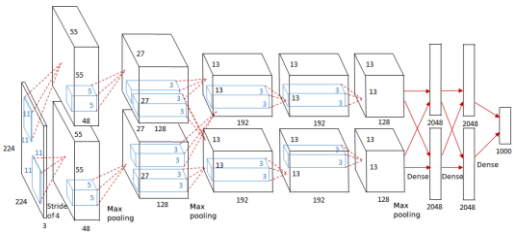
- *Capacidad para capturar dependencias a largo plazo:* Gracias a su estructura, las LSTM pueden recordar información relevante durante muchos pasos de tiempo, lo que es esencial en tareas como el procesamiento de texto, donde el contexto de palabras anteriores puede ser importante para entender las palabras actuales. Lo utiliza ChatGPT.
- *Control Dinámico del Flujo de Información:* Las puertas permiten a la red decidir qué información conservar y qué olvidar, lo que la hace más robusta y eficiente en el manejo de datos secuenciales complejos.

Redes más populares en Visión Artificial

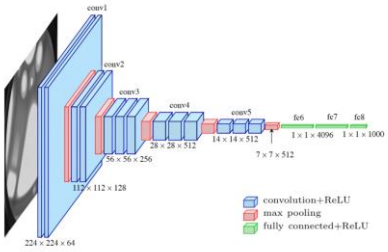
- *LeNet5 (1998):* Capas: 7, Parámetros: 220.248. Aportes: Backpropagation sobre redes convolucionales (LeCun).



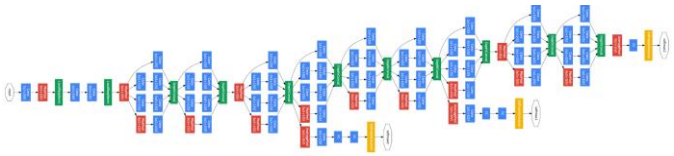
- *AlexNet (2012):* Capas: 8, Parámetros: 60.000.000. Error ImageNet: 16.4% * Aportes: ReLu, Dropout, Image Augmentation, GPUs.



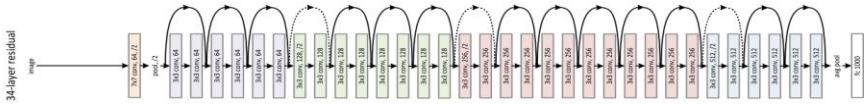
- *VGG (2014)*: Capas con pesos: 11-19, Parámetros: 140.000.000. Error: 7.3% * Aportes: Kernels pequeños. La profundidad es importante.



- *GoogleNet (2014):* Capas con pesos: 22, Parámetros: 4.000.000. Error: 6.67% * Aportes: Conv 1x1.



- *ResNet (2015):* Capas: 34-152, Parámetros: 25.600.000 (152). Error: 3.57% * Aportes: Residual networks.



- *EfficientNet (2019)*
- *YOLO (v1, 2015)*

<u>Redes end-to-end</u>

Las redes end-to-end son aquellas en las que el sistema aprende a realizar una tarea completa directamente a partir de los datos de entrada, sin la necesidad de pasos intermedios manualmente diseñados o de características predefinidas. En un tipo de red.

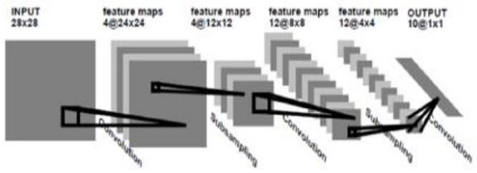
Se refiere a un sistema que toma los datos de entrada brutos (como imágenes, texto, señales, etc.) y produce directamente la salida deseada (como una clasificación, predicción, o generación) en un solo paso continuo. No requiere preprocesamiento manual o extracción de características separadas.

<u>Proceso de Aprendizaje:</u> Durante el entrenamiento, la red aprende a ajustar sus parámetros internamente para optimizar la tarea de principio a fin. Esto significa que todas las transformaciones de los datos de entrada a la salida son aprendidas automáticamente por la red, no se sabe cómo trabaja por dentro.

<u>Ventajas</u>

- *Automatización del Proceso:* Al eliminar la necesidad de diseñar manualmente características intermedias, el proceso es más automatizado y puede adaptarse mejor a la tarea específica.

- *Optimización Global:* Dado que la red se optimiza de manera conjunta, es más probable que encuentre soluciones que sean globalmente óptimas para la tarea completa.
- *Capacidad de Generalización:* Estas redes pueden, en algunos casos, aprender a generalizar mejor al no estar limitadas por preconceptos o características manuales.



Va realizando convoluciones y poolings internamente para ir achicando el tamaño

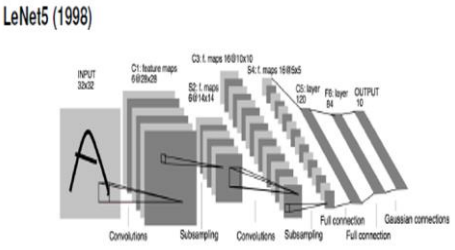
Ejemplos:

- Reconocimiento de Imágenes: En lugar de extraer características como bordes, texturas, etc., se toma la imagen en bruto y directamente produce la clasificación (por ejemplo, si es un gato o un perro).
- Traducción Automática: En procesamiento del lenguaje natural, se podría tomar una oración en un idioma y producir la traducción en otro idioma sin pasos intermedios como análisis gramatical. Las redes que surgieron con análisis gramatical.
- Conducción Autónoma: En sistemas de conducción autónoma, estas redes pueden recibir la imagen de una cámara y producir comandos de dirección sin pasar por pasos intermedios de detección de objetos.

<u>Desafíos</u>

- *Datos Necesarios:* Requieren grandes cantidades de datos para entrenar correctamente, porque todas las transformaciones deben ser aprendidas por la red.
- *Interpretabilidad:* Puede ser más difícil interpretar lo que está haciendo la red en comparación con sistemas donde los pasos intermedios son más claros.
- *Computación:* A menudo requieren más poder computacional para entrenar debido a su complejidad.

Un ejemplo práctico de una red end-to-end es el modelo LeNet-5 aplicado a la tarea de reconocimiento de dígitos escritos a mano, como los del conjunto de datos MNIST que contiene imágenes de dígitos escritos a mano (0-9), y la tarea es clasificar cada imagen en el dígito correspondiente.



Entrada: La red recibe una imagen en escala de grises de 28x28 píxeles (784 píxeles en total).

Capas Convolucionales:

- *Primera Capa Convolucional:* Aplica 6 filtros 5x5 a la imagen de entrada para detectar características básicas como bordes.

- *Segunda Capa Convolucional:* Aplica 16 filtros 5x5 a la salida de la primera capa, capturando características más complejas.
- *Capas de submuestreo (Pooling):* Después de cada capa convolucional, se aplica esta capa que reduce la resolución espacial de las imágenes, preservando las características más importantes y reduciendo la cantidad de parámetros.
- *Capas Completamente Conectadas:* La salida de las capas convolucionales es aplanada y alimentada a una o más capas completamente conectadas. Aquí es donde se combinan las características aprendidas para hacer la clasificación final.
- *Capa de salida:* Finalmente, la red tiene una capa de salida con 10 neuronas, cada una correspondiente a un dígito (0-9). *La neurona con el valor más alto indica la predicción del modelo.*

Función de Activación: Se utilizan funciones de activación no lineales como ReLU en las capas convolucionales y Softmax en la capa de salida para convertir las puntuaciones en probabilidades.

Proceso End-to-End

- *Entrada a Salida:* En el proceso completo, se ingresa la imagen de 28 x 28 píxeles y la red automáticamente la procesa, extrayendo características relevantes a través de las capas convolucionales y realizando la clasificación final en la capa de salida.
- *Entrenamiento:* El modelo ajusta automáticamente todos sus pesos para minimizar el error en la clasificación, optimizando la red de manera global sin intervención manual en la extracción de características.
- *Resultados:* La red LeNet-5 logra una alta precisión en la tarea de clasificación de dígitos en MNIST, demostrando cómo un modelo end-to-end puede aprender a realizar una tarea compleja como el reconocimiento de imágenes directamente a partir de los datos de entrada sin pasos intermedios manuales.

YOLO (You Only Look Once)

Es una red. Es un enfoque de aprendizaje profundo para la detección de objetos en imágenes y videos, conocido por su alta velocidad y precisión.

A diferencia de otros métodos de detección que pueden realizar múltiples pasos de procesamiento, *YOLO realiza la detección de objetos en una sola pasada a través de la red*, lo que lo hace extremadamente eficiente.

Es capaz de realizar detección de objetos en tiempo real debido a su arquitectura end-to end. Mientras que otros métodos dividen la imagen en regiones y luego clasifican cada región individualmente, YOLO trata toda la imagen de una vez, lo que le permite ser rápido y práctico para aplicaciones donde la velocidad es crucial, sin sacrificar demasiada precisión.

Cuadrícula de Predicción

La imagen de entrada se divide en una cuadrícula de $S \times S$. Cada celda de la cuadrícula es responsable de predecir un número fijo de cajas delimitadoras (bounding boxes) y las probabilidades de las clases para los objetos presentes dentro de esa celda. La imagen se divide en cuadrículas y a su vez cada cuadrícula en celdas se va fragmentando para ir acumulando información.

Cada celda predice:

- Las coordenadas de las cajas delimitadoras (posición y tamaño).
- La confianza de la predicción, que es una medida de si la caja contiene un objeto y qué tan precisa es la predicción.
- Las probabilidades de clase, que indican a que clase (coche, persona, etc.) pertenece el objeto.

Unión de la Predicción: Las cajas delimitadoras y las probabilidades de clase se combinan para formar las predicciones finales. YOLO produce múltiples predicciones por celda, y mediante técnicas como la supresión de no máximos (Non- Maximum Suppression, NMS), se filtran las cajas que se superponen y se conserva la más precisa.

Arquitectura: La versión original de YOLO utiliza una arquitectura basada en convoluciones similar a una red neuronal convolucional (CNN) profunda, que extrae características a través de capas convolucionales y luego realiza la detección y clasificación. YOLO ha evolucionado (YOLOv1, YOLOv2, YOLOv3, YOLOv4, y YOLOv5) mejorando la precisión y manteniendo la velocidad de detección.

Funcionamiento

- *Entrada:* Se ingresa una imagen de tamaño fijo, por ejemplo, 448×448 píxeles.
- *Salida:* La red produce una cuadrícula $S \times S$ donde cada celda tiene predicciones para B cajas delimitadoras. Además, para cada celda se predicen probabilidades de clase para cada una de las C clases posibles.
- *Entrenamiento:* El entrenamiento se realiza minimizando una función de pérdida que combina el error en la localización de las cajas, la confianza, y las probabilidades de clase. *Se utiliza una única función de pérdida para todo el modelo, lo que unifica la detección y clasificación.*

Ejemplo de Aplicación

- *Seguridad y Vigilancia:* En sistemas de vigilancia, se puede utilizar para detectar y rastrear personas, vehículos, y otros objetos en tiempo real, alertando sobre actividades sospechosas.
- *Conducción Autónoma:* YOLO puede integrarse en sistemas de conducción autónoma para detectar peatones, señales de tráfico, vehículos, y otros obstáculos en tiempo real, lo que permite tomar decisiones rápidas.

UNIDAD 3: RAZONAMIENTO EN AMBIENTES DETERMINISTAS

Búsqueda

El primer paso para solucionar un problema es la formulación del objetivo, basado en la situación actual y la medida de rendimiento del agente. Consideraremos un objetivo como un conjunto de estados del mundo, aquellos estados que satisfacen el objetivo.

La tarea del agente es encontrar qué secuencia de acciones permite obtener un estado objetivo, para lo cual necesitamos decidir qué acciones y estados considerar lo que constituye la formulación del problema.

Consideremos un agente en la ciudad de Arad, Rumania que ha adoptado el objetivo de conducir a Bucarest y considera a dónde ir desde Arad. Existen tres carreteras desde Arad, una hacia Sibu, una a Timisoara y una a Zerind.

Dado que el agente no tiene conocimiento adicional, lo mejor que puede hacer es escoger al azar una de las acciones. Un agente con distintas opciones inmediatas de valores conocidos puede decidir qué hacer, examinando las diferentes secuencias posibles de acciones que le conduzcan a estados de valores conocidos, y entonces escoger la mejor secuencia.

El agente **individual** (hay un solo agente que participa y toma decisiones en el ambiente de búsqueda) supone que el entorno es **estático,** porque la formulación y búsqueda del problema se hace sin prestar atención a cualquier cambio que puede ocurrir en el entorno. El agente diseñado también supone que se conoce el **estado inicial**; si el entorno es **observable** (se puede saber todo el contexto), **episódico** (Varía en momentos discretos (por ej: t1, t2) donde no hay cambios continuos), **discreto** (característica del paso del tiempo, el cambio se produce en momentos determinados) y supone además que el entorno es **determinista** (ante una determinada entrada, se tiene una determinada salida). Se debe conocer el ambiente (Conocido, si se conoce el ambiente donde se está trabajando o es totalmente desconocido y exploratorio). Consideraremos un **estado objetivo** como un conjunto de estados del mundo (exactamente aquellos estados que satisfacen el objetivo). Lo que se quiere obtener. Depende del ambiente y del objetivo de la búsqueda. La tarea del agente es encontrar una secuencia de acciones que permita obtener un estado objetivo.

Un problema puede definirse por cuatro componentes:

- El *estado inicial* en el que comienza el agente
- Una *descripción de las posibles acciones disponibles del agente*, es decir utilizar una función sucesor, dado un estado x la función SUCESOR-FN(x) devuelve un conjunto de pares ordenados (acción, sucesor) donde cada acción es una de las acciones legales en el estado x y cada sucesor es un estado que puede alcanzarse desde x, aplicando la acción. Implícitamente el estado inicial y la función sucesor definen el espacio de estados del problema, es decir el conjunto de todos los estados alcanzables desde el estado inicial. Un camino en un espacio de estados es una secuencia de estados conectados por una secuencia de acciones.

- El *test objetivo*, el cual determina si un estado es un estado objetivo. Verifica si se ha alcanzado el objetivo del problema. La idea es buscar los costos que sean más baratos.
- Una función *costo del camino* que asigna un costo numérico a cada camino, esta función costo debe reflejar una medida de rendimiento. Indica cuánto cuesta tomar una decisión. Es la suma de todos los costos individuales de ir por ciertos caminos que se decidió.

Definición del problema

Para resolver un problema de búsqueda es necesario:

- La definición formal del problema
    - Estado objetivo: Hasta donde quiero llegar. Toda la búsqueda depende de a dónde se quiere llegar.
    - Estado inicial: Plantea todo el conjunto de acciones que tiene que realizar para llegar al objetivo.

Saber dónde quiero ir y donde estoy es fundamental para determinar las acciones que se van a tomar.

    - Conjunto de acciones posibles: (son todas las acciones que se pueden llegar a tomar) son las acciones que voy tomando y las que se van descartando.
    - Función de costo del camino: puede ser de tiempo (minimizar tiempo). Define el conjunto de acciones que se toman.
- Una estrategia de búsqueda: Es dependiendo lo que se necesite (minimizar tiempos se utiliza una estrategia o minimizar costos se utiliza otra)

Para implementar la búsqueda es necesario encontrar o crear:

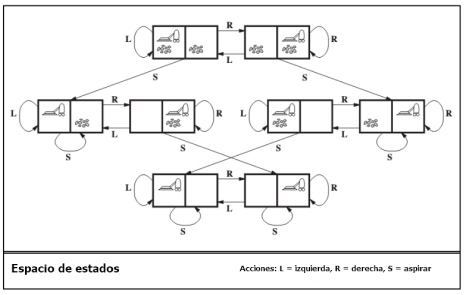
- Una forma de representar en los estados inicial, final e intermedios. Los estados intermedios son estados donde tengo que tomar una decisión, donde tengo subproblemas. Los estados intermedios son las rutas hasta llegar al estado final.
- Un conjunto de reglas compuestas por una condición de aplicación y una operación.
- Una estrategia de control para decidir el orden de aplicación de las reglas.

El espacio de estados o espacio de búsqueda es el conjunto de todos los posibles estados en los que puede encontrarse el sistema, es decir, **conjunto de todos los estados que pueden ser alcanzados aplicando acciones a partir del estado inicial**.

Una solución de un problema es un camino desde el estado inicial a un estado objetivo. La calidad de la solución se mide por la función costo del camino, y una solución óptima tiene el costo más pequeño del camino entre todas las soluciones.

Máquina de estados finitos. Cada nodo es un estado y las transiciones son dirigidas de acuerdo al estado que se va a pasar de acuerdo a las decisiones que se toman.

Los estados de arriba son los estados iniciales, los del medio los estados intermedios (las distintas combinaciones de estados por los que se pasa) y los de abajo son los estados objetivos.



Camino solución vs camino de búsqueda

Camino: conjunto de pasos por el cual se llega desde el inicio al fin.

- **El camino de la solución** es la sucesión de estados, que comienza con el estado inicial y termina con el estado objetivo, donde cada estado puede ser generado aplicando una operación sobre el estado anterior. Es el camino "encontrado" por el método como solución al problema. Alguna solución se debe tener. Por lo menos se debe tener uno.
- **El camino de búsqueda** es la sucesión de estados explorados por el método hasta encontrar la solución. No están conectados por acciones (Por ejemplo, para ir desde la facultad a la falda se tienen varios caminos, si llegó a Cosquín es un camino de búsqueda porque fue una búsqueda, pero no óptima porque no se llegó al objetivo). Para una solución se puede tener múltiples caminos de búsqueda que pueden ser óptimos o no.

Rendimiento del método

- Completitud: el algoritmo asegura encontrar una solución, si la hay
- Optimalidad: el algoritmo encuentra una solución óptima
- Complejidad temporal: ¿Cuánto tiempo toma encontrar la solución?
- Complejidad espacial: ¿Cuánta memoria necesito para encontrar la solución?

Un problema de juguete se utiliza para ilustrar o ejercitar los métodos de resolución de problemas, mientras que un problema del mundo real es aquel en el que la gente se preocupa por sus soluciones.

<u>Problema del juguete:</u>

Mundo de la aspiradora:

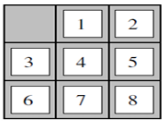
- Estados: el agente está en una de dos localizaciones, cada una con o sin suciedad. Hay 2 * 2 * 2 = 8 posibles estados del mundo.
- Estado inicial: cualquier estado
- Función sucesor: los estados legales que resultan al intentar las tres acciones, izquierda, derecha, aspirar.
- Test objetivo: si el cuadrado está limpio.
- Costo del camino: cada costo individual es 1, así que el costo del camino es el número de pasos que lo compone.

Problemas de juguete:

8-puzzle consiste en un tablero de 3 x 3 con ocho fichas numeradas y un espacio en blanco.



Estado Inicial

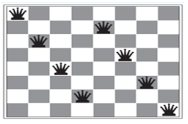


Estado Objetivo

- Estados: La descripción de un estado especifica la localización de cada una de las ochos fichas y el blanco en cada uno de los nueve cuadrados.
- Estado inicial: cualquier estado
- Función sucesor: los estados legales que resultan al intentar las cuatro acciones, mover el blanco a la izquierda, derecha, arriba y abajo.
- Test objetivo: comprueba si el estado coincide con la configuración objetivo.
- Costo del camino: cada costo individual es 1, así que el costo del camino es el número de pasos que lo compone.

Problemas de juguete:

El problema de las 8-reinas en un tablero de ajedrez de manera que cada reina no ataque a ninguna otra, una reina ataca alguna pieza si está en la misma fila, columna o diagonal.



- Estados: cualquier combinación de cero a ocho reinas en el tablero es un estado.
- Estado inicial: ninguna reina sobre el tablero.
- Función sucesor: añadir una reina a cualquier cuadrado vacío.
- Test objetivo: ocho reinas sobre el tablero, ninguna es atacada.

Una mejor formulación deberá prohibir colocar una reina en cualquier cuadrado que este atacado.

- Estados: son estados la combinación n reinas $0 \leq n \leq 8$ una por columna desde la columna de más a la izquierda sin que una reina ataque a otra.
- Estado inicial: ninguna reina sobre el tablero.
- Función sucesor: añadir una reina a cualquier cuadrado en la columna más a la izquierda vacía tal que no sea atacada por cualquier otra reina.
- Test objetivo: ocho reinas sobre el tablero, ninguna es atacada.

<u>Problemas del mundo real</u>

Consideremos un ejemplo simplificado de un problema de viajes de líneas aéreas que especificamos como:

- Estados: cada uno está representado por una localización, por ejemplo un aeropuerto y la hora actual.
- Estado inicial: especificado por el problema.

- Función sucesor: devuelve los estados que resultan de tomar cualquier vuelo programado desde el aeropuerto actual a otro, que salgan a la hora actual más el tiempo de tránsito del aeropuerto.
- Test objetivo: Tenemos nuestro destino para una cierta hora especificada?
- Costo del camino: esto depende del costo en dinero, tiempo de espera, tiempo de vuelo, procedimientos de inmigración, calidad del asiento, hora, tipo de avión, etc.

Para resolver el problema de búsqueda se utiliza un **árbol de búsqueda** explícito generado por el estado inicial y la **función sucesor**, definiendo así el espacio de estados.

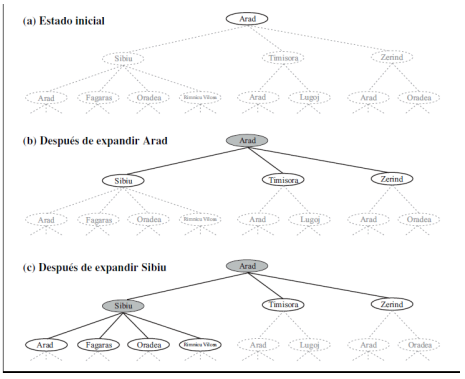
La raíz del árbol corresponde al estado inicial.

El primer paso es comprobar si éste es un estado objetivo, como no estamos en un estado objetivo, tenemos que considerar otros estados. Esto se hace expandiendo el estado actual y generar así un nuevo conjunto de estados.

El estado a expandir está determinado por la estrategia de búsqueda.

Un nodo es una estructura de datos con cinco componentes:

- *Estado:* el estado, del espacio de estados, que corresponde con el nodo
- *Nodo padre:* el nodo en el árbol de búsqueda que ha generado ese nodo
- *Acción:* la acción que se aplicará al padre para generar el nodo.
- *Costo del camino:* el costo, g(n) de un camino desde el estado inicial al nodo, indicado por los punteros a los padres
- *Profundidad:* el número de pasos a lo largo del camino desde el estado inicial.



Un **nodo** es una estructura de datos usada para representar el árbol de búsqueda. Un **estado** corresponde a una configuración del mundo. Así los nodos están en caminos particulares, según lo definido por los punteros del nodo padre, mientras que los estados no lo están.

A la colección de nodos que se han generado pero todavía no se han expandido le llamaremos **frontera,** cada uno de sus elementos es una hoja.

La **estrategia de búsqueda** será una función que seleccione de este conjunto el siguiente nodo a expandir, asumiremos que la colección de nodos se implementa en una cola. Las operaciones en una cola son como siguen:

- Hacer-cola(elemento, …) crea una cola con el elemento dado
- Vacía?(cola) devuelve verdadero si no hay ningún elemento en la cola
- Primero(cola) devuelve el primer elemento de la cola
- Borrar-Primero(cola) devuelve Primero(cola) y lo borra de la cola
- Inserta(elemento, cola) inserta un elemento en la cola y devuelve la cola resultado.
- Insertar-Todo(elementos, cola) inserta un conjunto de elementos en la cola y devuelve el resultado.

Medir el rendimiento de la resolución del problema. Hay cuatro formas:

- *Completitud:* está garantizado que el algoritmo encuentre una solución cuando esta exista?
- *Optimización:* encuentra en la estrategia la solución óptima, es decir la que maximiza el valor esperado de la medida de rendimiento.
- *Complejidad en el tiempo:* cuanto tarda en encontrar una solución? El tiempo se mide en términos de números de nodos generados durante la búsqueda
- *Complejidad en el espacio: ¿*cuánta memoria se necesita para el funcionamiento de la búsqueda? El espacio se mide en términos del máximo número de nodos que se almacena en memoria

En IA el árbol está representado de forma implícita por el estado inicial y la función sucesor, y se expresa en términos de tres cantidades: **b** el factor de ramificación o el máximo número de sucesores de cualquier nodo, **d** la profundidad del nodo objetivo más superficial, y **m** la longitud máxima de cualquier camino en el espacio de estados. **d** y **m** comienzan a contar 1 cuando nos movemos para abajo.

Los agentes más simples, los agentes reactivos, los cuales basan sus acciones en una aplicación directa desde los estados a las acciones. Tales agentes no pueden funcionar bien en entornos en los que esta aplicación sea demasiado grande para almacenarla y que tarde mucho en aprenderla. Por otra parte, los agentes basados en objetivos pueden tener éxito considerando las acciones futuras y lo deseable de sus resultados.

Los agentes resolventes-problemas deciden qué hacer para encontrar secuencias de acciones que conduzcan a los estados deseables. Comenzamos definiendo con precisión los elementos que constituyen el «problema» y su «solución».

Los algoritmos son no informados, en el sentido que no dan información sobre el problema salvo su definición. Los algoritmos de búsqueda informada, en cambio, tienen cierta idea de dónde buscar las soluciones.

Búsqueda no informada: No se sabe que tan bueno es el camino por donde se va yendo. En la búsqueda no informada no se tiene información adicional acerca de los estados más allá de la que proporciona la definición del problema

Las estrategias que saben si un estado no objetivo es más prometedor que otro se llama búsqueda informada o búsqueda heurística.

Estrategias de búsqueda englobadas bajo el nombre de búsqueda no informada (llamada también búsqueda a ciegas):

1. Búsqueda primero en anchura/ amplitud (BFS - Breadth-First Search):

Es una estrategia sencilla en la que se expande primero el nodo raíz, a continuación se expanden todos los sucesores del nodo raíz, después sus sucesores, etc. En general se expanden todos los nodos a una profundidad en el árbol de búsqueda antes de expandir cualquier nodo el próximo nivel.

La búsqueda primero en anchura se puede implementar comenzando con una frontera vacía que sea una cola primero en entrar - primero en salir (FIFO). La cola FIFO pone todos los nuevos sucesores generados al final de la cola, lo que significa que los nodos más superficiales se expanden antes que los nodos más profundos.

Esta búsqueda primero en anchura es **completa**, si el nodo objetivo más superficial está en una cierta profundidad finita **d**, la búsqueda primero en anchura lo encontrará después de expandir todos los nodos más superficiales, con tal que el factor de ramificación **b** sea finito.

La búsqueda primero en anchura es óptima si el costo del camino es una función no decreciente de la profundidad del nodo.

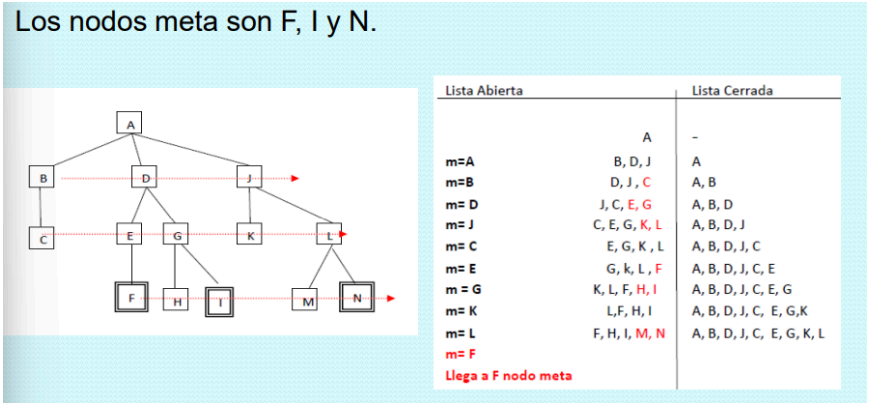
Consideremos un espacio de estados hipotético donde cada estado tiene **b** sucesores, la raíz del árbol de búsqueda genera b nodos en el primer nivel, cada uno de ellos genera b nodos más, teniendo un total de b^2 en el segundo nivel. Supongamos que la solución tiene profundidad **d**, en el peor caso expandiremos todo excepto el último nodo en el nivel d ya que el objetivo no se expande, generando $b^{d+1} - b$ nodos en el nivel d+1. El número total de nodos generados es:

$$b + b^2 + b^3 + \ldots + b^d + (b^{d+1} - b) = O(b^{d+1})$$

La complejidad espacial es la misma que la complejidad en tiempo más un nodo para la raíz. En el siguiente cuadro se enumera el tiempo y la memoria requerida para una búsqueda primero en anchura con el factor de ramificación b = 10, 10,000 nodos/segundo, 1000 bytes/nodo.

Profundidad	Nodos	Tiempo	Memoria
2	1.100	11 segundos	1 megabyte
4	111.100	11 segundos	106 megabytes
6	10^7	19 minutos	10 gigabytes
8	10^9	31 horas	1 terabytes
10	10^{11}	129 días	101 terabytes
12	10^{13}	35 años	10 petabytes
14	10^{15}	3.523 años	1 exabyte

De la figura anterior se desprende que los requisitos de memoria para la búsqueda primero en anchura son un problema más grande que el tiempo de ejecución. Hace falta incluir en los algoritmos una estructura de datos llamada lista cerrada, que almacene cada nodo expandido, la frontera de nodos no expandidos se llama lista abierta. *No es muy bueno desde el punto de vista espacial (consume mucha memoria) pero si desde el punto de vista temporal (porque encuentra más rápido)*



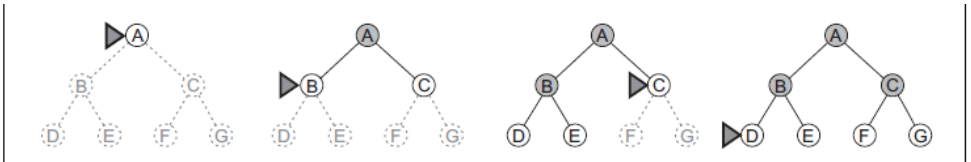


Figura 3.10 Búsqueda primero en anchura sobre un árbol binario sencillo. En cada etapa, el próximo nodo a expandir se indica con una marca.

Pasos:

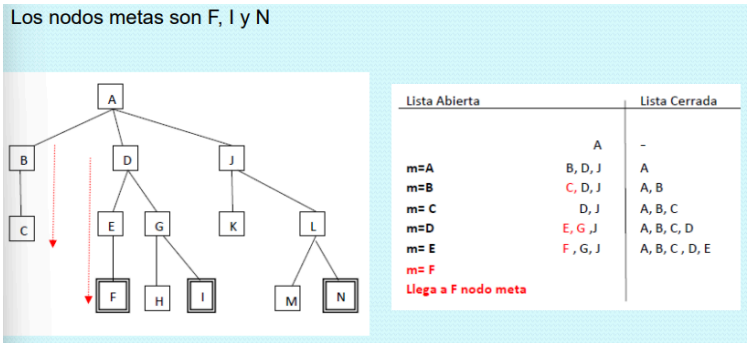
1. Inicia en el nodo raíz.
2. Agrega los hijos del nodo actual a una cola (FIFO).
3. Saca el siguiente nodo de la cola y repite el proceso hasta encontrar la solución o que no haya más nodos por explorar.

4. <u>Búsqueda primero en profundidad (DFS - Depth-First Search):</u>

En este tipo de búsqueda se expande el nodo más profundo en la frontera actual del árbol de búsqueda. Cuando esos nodos se expanden son quitados de la frontera, así entonces la búsqueda retrocede al siguiente nodo más superficial que todavía tenga sucesores inexplorados.

El algoritmo se implementa con una **cola último en entrar primero en salir (LIFO)**. La búsqueda primero en profundidad tiene unos requisitos muy modestos de memoria. Necesita almacenar sólo un camino desde la raíz a un nodo hoja, junto con los nodos restantes no expandidos para cada nodo del camino. Una vez que un nodo se ha expandido se puede quitar de la memoria tan pronto como todos sus descendientes han sido explorados. Para un espacio de estados con factor de ramificación **b** y máxima profundidad **m**, la búsqueda primero en profundidad requiere almacenar solo b*m + 1 nodos.

Una variante de la búsqueda primero en profundidad, llamada búsqueda hacia atrás, utiliza menos memoria. En la búsqueda hacia atrás, sólo se genera un sucesor a la vez, cada nodo parcialmente expandido recuerda que el sucesor se expande a continuación. De esta manera se necesita **O(m)** memoria en lugar que el **O(b*m)** anterior. El inconveniente de la búsqueda primero en profundidad es que puede hacer una elección equivocada y obtener un camino muy largo aún cuando una elección diferente llevaría a una solución cerca de la raíz del árbol de búsqueda. *Resuelve en el tiempo más rápido.*



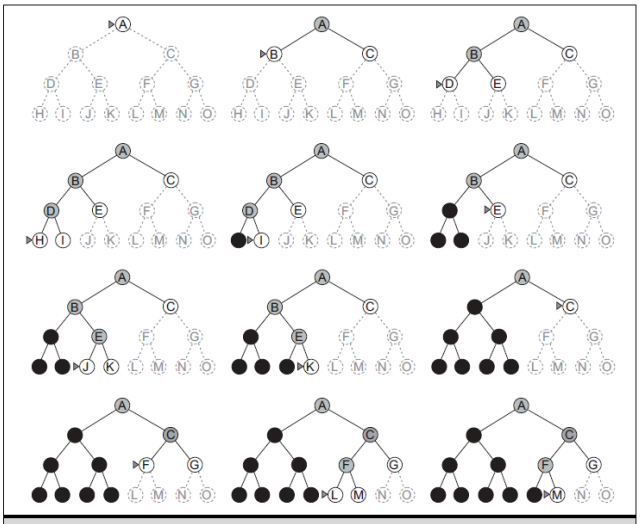


Figura 3.12 Búsqueda primero en profundidad sobre un árbol binario. Los nodos que se han expandido y no tienen descendientes en la frontera se pueden quitar de la memoria; estos se muestran en negro. Los nodos a profundidad 3 se suponen que no tienen sucesores y *M* es el nodo objetivo.

Pasos:

1. Inicia en el nodo raíz.
2. Selecciona uno de los hijos del nodo actual y lo expande.
3. Continúa profundizando hasta que no haya más hijos.
4. Retrocede y expande el siguiente nodo no visitado en la rama anterior.

   Ventajas: Requiere menos memoria que BFS. Puede ser eficiente si la solución está en una rama profunda ya que requiere menos memoria. Cuando el espacio de búsqueda es más profundo es mejor utilizar esta algoritmo.

   Desventajas: No garantiza encontrar la solución más corta. Puede quedarse atrapada en ramas infinitas si no se establece un límite de profundidad (se puede usar DFS limitada en profundidad).

   Dónde aplicarlo: Cuando el espacio de búsqueda es muy profundo y se necesita encontrar una solución rápida (aunque no sea la más corta). Para resolver puzles como el sudoku o problemas de backtracking como el problema de las n-reinas.

3. Búsqueda de profundidad limitada:

Se puede aliviar el problema de árboles ilimitados aplicando la búsqueda primero en profundidad con un **límite de profundidad** ℓ predeterminado, es decir, los nodos a profundidad ℓ se tratan como si no tuvieran ningún sucesor. A este algoritmo se lo denomina búsqueda de profundidad limitada.

4. Búsqueda primero en profundidad con profundidad iterativa:

La búsqueda con profundidad iterativa es en general usada en combinación con la búsqueda primero en profundidad, la cual encuentra el mejor límite de profundidad. Esto se hace aumentando gradualmente el límite, primero 0, después 1, después 2, etc. Hasta que encontremos el objetivo. Esto ocurrirá cuando el límite de profundidad alcanza **d** profundidad del nodo objetivo.

La profundidad iterativa combina las ventajas de la búsqueda primero en profundidad y primero en anchura. En la búsqueda primero en profundidad su exigencia de memoria es O(b*d).

En una búsqueda de profundidad iterativa, los nodos sobre el nivel inferior (profundidad d) son generados una vez, los anteriores al nivel inferior dos veces, etc. hasta los hijos del nodo raíz que son generados **d** veces. El número total de nodos generados es:

$$N(BPI) = (d)\, b + (d-1)\, b^2 + \ldots\ldots + (1)\, b^d.$$

En general la profundidad iterativa es el método de búsqueda no informada preferido cuando hay un espacio grande de búsqueda y no se conoce la profundidad de la solución.

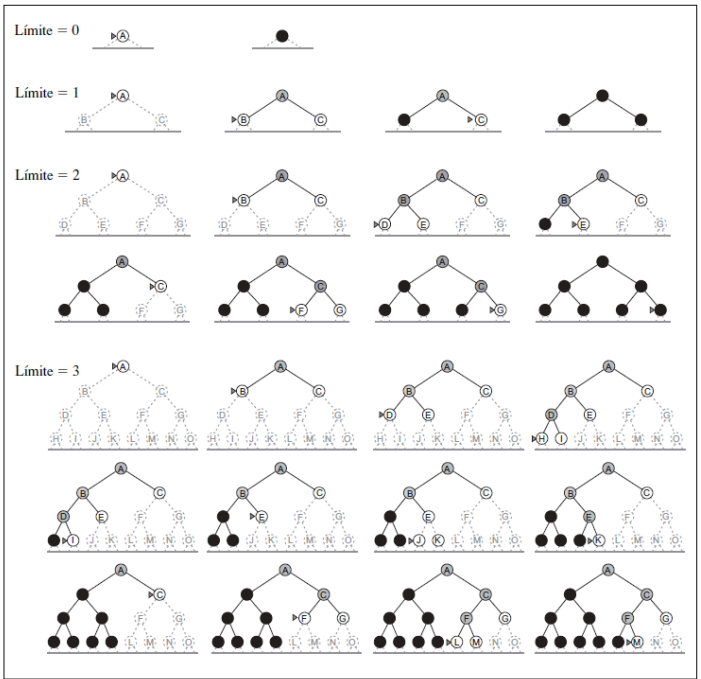


Figura 3.15 Cuatro iteraciones de la búsqueda de profundidad iterativa sobre un árbol binario.

Búsqueda informada: La búsqueda informada es la que utiliza el conocimiento específico del problema más allá de la definición del problema en sí mismo. Esta búsqueda informada puede encontrar soluciones de una manera más eficiente que una estrategia no informada. Se selecciona un nodo para la expansión basada en una **función de evaluación f(n).**

Un componente clave de estos algoritmos es una **función heurística** denotada h(n)

h(n)= costo estimado del camino más barato desde el nodo n a un nodo objetivo

Si n es un nodo objetivo h(n) = 0

1. Búsqueda primero el mejor (Best-First Search): Trata de expandir el nodo más cercano al objetivo, alegando que probablemente conduzca rápidamente a una solución. Así evalúa los nodos utilizando solamente la función heurística **f(n) =h(n)**

La búsqueda primero el mejor se parece a la búsqueda primero en profundidad en el modo que prefiere seguir un camino hacia el objetivo, pero volverá atrás cuando llegue a un callejón sin salida. Sufre los mismos defectos que la búsqueda primero en profundidad, no es óptima y es incompleta, porque puede ir hacia abajo en un camino infinito y nunca volver para intentar otras posibilidades. La complejidad en tiempo y espacio, del caso peor, es $O(b^m)$ donde **m es la profundidad máxima del espacio de búsqueda.** Con una buena función heurística se puede reducir la complejidad considerablemente. La cantidad de la reducción depende del problema particular y de la calidad de la heurística. *De múltiples vecinos, siempre se elige la que tiene distancia más corta para llegar al nodo objetivo. Se tiene en cuenta siempre el objetivo.*

Este método utiliza una función heurística para seleccionar el siguiente nodo a explorar, priorizando aquellos que parecen estar más cerca del objetivo. A diferencia de DFS o BFS, la selección de nodos no se basa en el orden de inserción sino en la estimación de qué nodo parece más prometedor.

- Utiliza una lista (ABIERTA) para guardar los estados a recorrer.
- La lista ABIERTA se ordena según el valor de la heurística (primero el mejor)
- Se utiliza otra lista (CERRADA) para guardar los estados recorridos.
- La heurística h′ es la distancia estimada al estado objetivo.

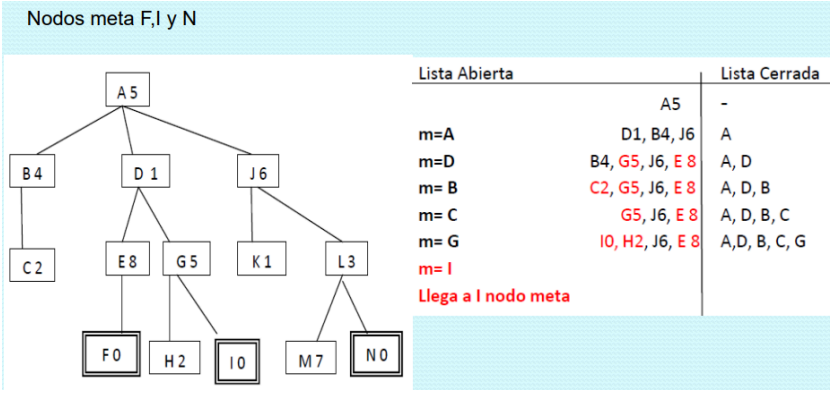
Pasos:

1. Inicia en el nodo raíz.
2. Se calcula el valor heurístico (una estimación) de los hijos del nodo actual.
3. Elige el nodo con el mejor valor heurístico (el que parece más cercano al objetivo).
4. Repite hasta que se encuentre el nodo objetivo.

Ventajas: Puede ser más rápido que BFS y DFS porque se guía por la heurística.

Desventajas: No garantiza encontrar la solución óptima. El rendimiento depende de la calidad de la heurística.

Dónde aplicarlo: Navegación de robots en entornos donde los obstáculos hacen que algunos caminos sean más difíciles de recorrer. Problemas de juegos como encontrar el movimiento más prometedor en un tablero de ajedrez.



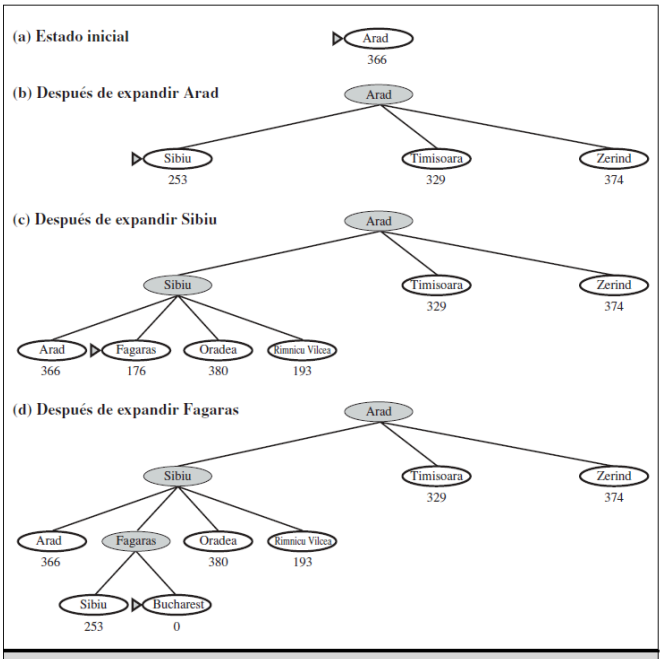


Figura 4.2 Etapas en una búsqueda primero el mejor avara para Bucarest utilizando la heurística distancia en línea recta h_{DLR}. Etiquetamos lo nodos con sus *h*-valores.

2. <u>Búsqueda A*:</u> minimizar el costo estimado total de la solución. Evalúa los nodos combinando g(n) el costo para alcanzar el nodo y h(n) el costo de ir al nodo objetivo.

   $f(n) = g(n) + h(n)$

   f(n) = costo más barato estimado de la solución a través de n.

Si tratamos de encontrar la solución más barata es razonable intentar primero el nodo con el valor más bajo de g(n) + h(n), siempre y cuando la función heurística satisfaga ciertas condiciones la búsqueda A* es completa y óptima.

A* es óptima si h(n) es una heurística admisible, es decir, con tal que la h(n) nunca sobrestime el costo de alcanzar el objetivo, se controla la condición de estado objetivo cuando se saca el estado de la lista ABIERTA y se tiene en cuenta un detalle que veremos en el ejemplo, garantiza la solución óptima. La ventaja de A* es que encuentra caminos más cortos.

La heurística debe ser también consistente, es decir si para cada nodo n y cada sucesor n' de n generado por cualquier

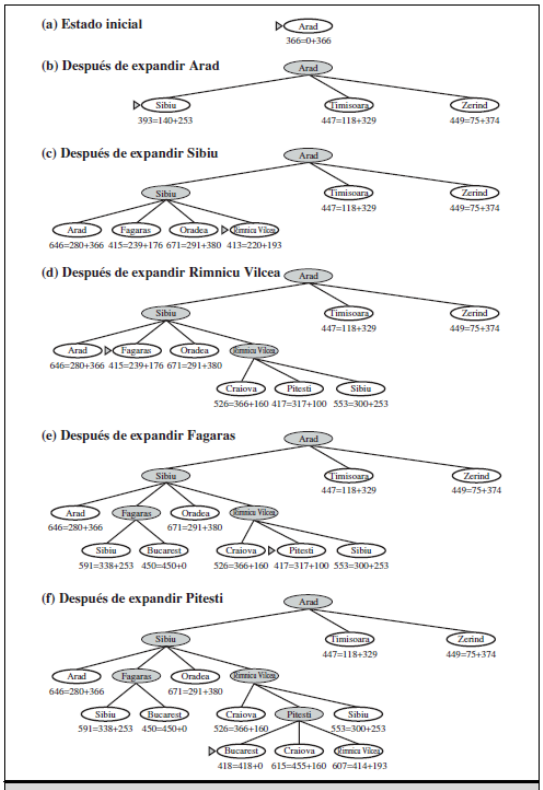
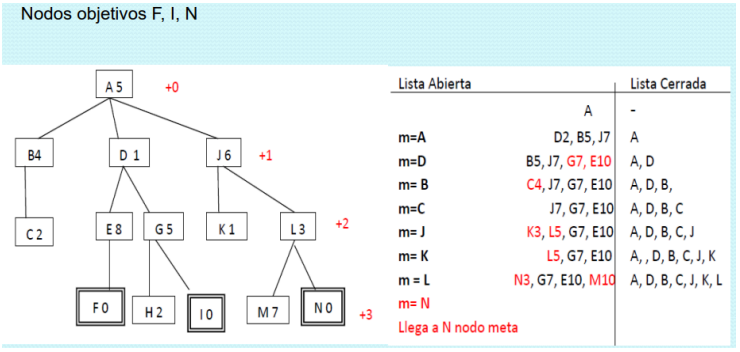


Figura 4.3 Etapas en una búsqueda A* para Bucarest. Etiquetamos lo nodos con $f = g + h$. Los valores *h* son las distancias en línea recta a Bucarest tomadas de la Figura 4.1.

acción a, el costo estimado de alcanzar el objetivo desde n no es mayor que el costo de alcanzar n' más el costo estimado de alcanzar el objetivo desde n'

$$h(n) \leq c(n,a,n') + h(n')$$



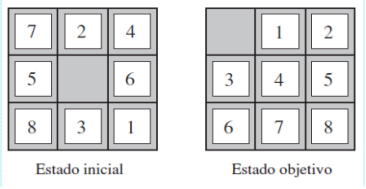
FUNCIONES HEURÍSTICAS:

Heurística: Para los métodos de búsqueda, es una técnica que aumenta la eficiencia del proceso, posiblemente sacrificando demandas de completitud. Es conocimiento (muchas veces intuitivo) que puede guiar el proceso de búsqueda. Las heurísticas no garantizan la solución óptima, pero generalmente mejoran la calidad del proceso.

Función heurística: El conocimiento heurístico generalmente se incorpora al proceso de búsqueda a través de una función heurística. La función heurística determina un grado de "bondad" para cada estado evaluado.

A medida que armamos el árbol, no hay que tener estados repetidos. Se va por el camino más corto, o mejor.

El 8-puzzle fue uno de los primeros problemas de búsqueda heurística, el objeto del puzle es deslizar las fichas horizontalmente o verticalmente al espacio vacío hasta que la configuración empareje con la configuración objetivo. Cuando la ficha vacía está en el medio hay cuatro movimientos posibles, cuando está en una esquina hay dos y cuando está a lo largo del borde hay tres. En el caso del 8-puzzle son aproximadamente 22 pasos con un factor de ramificación de 3, descartando los estados repetidos podríamos reducirlo a un factor de aproximadamente 170,000 estados posibles.



El efecto de la precisión heurística en el rendimiento

Una manera de caracterizar la calidad de una heurística es el b* factor de ramificación eficaz. Si el número total de nodos generados por A* para un problema particular es N, y la profundidad de la solución es d, entonces b* es el factor de ramificación que un árbol uniforme de profundidad d debería tener para contener N + 1 nodos. Así

$$N + 1 = 1 + b^* + (b^*)^2 + \ldots\ldots + (b^*)^d$$

El factor de ramificación eficaz puede variar según los ejemplos del problema, pero por lo general es constante para problemas suficientemente difíciles. Una heurística bien diseñada tendría un valor de b* cerca de 1, permitiría resolver problemas bastante grandes.

A un problema con menos restricciones se le llama problema relajado. El costo de una solución óptima en un problema relajado es una heurística admisible para el problema original.

Si el problema relajado es difícil de resolver, entonces los valores de la correspondencia heurística serán costosos de obtener.

Algoritmos de búsqueda local y problemas de optimización

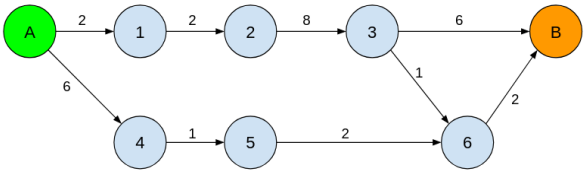
En muchos problemas, el camino al objetivo es irrelevante, por ejemplo el problema de las 8-reinas, lo que importa es la configuración final de las reinas, no el orden en las cuales se añaden. Estos problemas incluyen muchas aplicaciones como diseño de circuitos integrados, disposición del suelo de una fábrica, programación del trabajo en tiendas, programación automática, optimización de redes de telecomunicaciones, dirigir un vehículo, etc.

Si no importa el camino al objetivo, podemos considerar una clase diferente de algoritmos que no se preocupen de los caminos. Los algoritmos de búsqueda local funcionan con un solo estado actual y generalmente se mueven sólo a los vecinos del estado. Aunque los algoritmos de búsqueda local no son sistemáticos, tienen dos ventajas importantes, primero usan poca memoria y segundo pueden encontrar a menudo soluciones razonables en espacios de estados grandes o infinitos (continuos) para los cuales son inadecuados los algoritmos sistemáticos.

Búsqueda en grafos (en distintos escenarios)

Se puede considerar un grafo como un árbol generalizado, en donde se pueden encontrar ciclos (así como un árbol se puede considerar un grafo restringido).

Los mecanismos de búsqueda son similares pero los grafos plantean algunos escenarios interesantes de analizar. Uno de esos escenarios deriva de la posibilidad de contar o no con información que guíe las búsquedas. Consideremos el siguiente grafo que puede representar las rutas que hay entre dos ciudades, identificadas con diferentes colores.



Se va avanzando en el problema y se va decidiendo qué camino tomar, dependiendo qué camino vaya siendo el óptimo en ese momento hasta llegar al objetivo, normalmente se va decidiendo dependiendo del costo que se tenga ir de un nodo a otro, ya que se va a elegir el arco que tenga menos costo. Se va conociendo el contexto para saber por dónde ir. El problema conocido como de **"óptimos locales"** que es el fenómeno que nos lleva a tomar decisiones que son óptimas dentro de un entorno reducido pero que no lo son en un entorno ampliado. Puede pasar que si no se tiene la información completa se elija una ruta que no sea la óptima en todo el recorrido pero en base a la información que se tenía en ese momento sí, por lo que se está en presencia de “óptimos locales”.

Existen diversas técnicas para minimizar este problema, las cuales radican en realizar recorridos exploratorios, o en ampliar el ámbito de búsqueda de soluciones locales, etc.
A medida que el grafo crece en tamaño y conectividad, el acceso a información completa se hace cada vez más dificultoso y es necesario plantear algoritmos con mejores heurísticas para resolver esos escenarios.

Ejemplo laberinto

Primero en anchura

Se consideran dos estados posibles, 0 (lugar por donde se puede transitar) y 1 (obstáculo). Luego se plantea el estado inicial y hacia dónde se quiere llegar (ambos estados).

Se definen las acciones válidas, en este caso las acciones son moverse (porque es lo que se puede hacer).

Las acciones válidas son: Se puede mover en cualquier dirección que tenga un lugar válido y que no esté bloqueado por un obstáculo.

Se van planteando cuáles son las acciones que se pueden tomar de acuerdo a las condiciones del ambiente donde se esté. Para eso se debe saber cuáles son los vecinos válidos (es decir, los vecinos que tienen 0). Luego se chequea que no se vaya de ninguno de los bordes (para mantenerse siempre dentro del laberinto) y que la posición a la que estoy queriendo ir haya un 0. Si se cumplen todas las condiciones, se agrega a la lista de vecinos el nuevo punto.

Mientras haya nodos en la cola, la posición actual va a ser lo que hay en la cola, si la posición actual es el nodo objetivo, significa que ya se llegó al objetivo. Si no, de ese nodo actual busca los vecinos y para cada vecino válido moverse en esa dirección, marcarlo que ya se movió (se agregan a la lista de visitados, para no volver) a allí y en la cola se agregan todos los vecinos de ese nuevo nodo (se buscan los vecinos del vecino).

Manteniendo la lista de los lugares ya visitados para no volver puede recorrer vecinos nuevos, pero puede pasar que no se tengan más vecinos, por lo que puede llegar a un callejón sin salida. Si se decide no visitar nodos visitados, puede parecer una buena idea al principio pero puede que se bloqueen algunos caminos que si lleguen a la solución.

Primero en profundidad

Cuando se pasa por un visitado, para cada vecino, de forma recursiva se intenta resolver primero en profundidad para ese nodo. De esta forma, en lugar de tener un proceso iterativo, este es recursivo y se van resolviendo subárboles.

Primero el mejor

Implica tener información sobre el entorno. Si se tiene una lista de vecinos, el mejor es el vecino que más se acerque al objetivo. Se tiene un conocimiento del entorno para decir que cualquier vecino que esté en dirección a mi objetivo es mejor que cualquier otro. Los vecinos válidos pero que se alejen del objetivo no se van a evaluar, se descartan

Se va mejorando el criterio de búsqueda en base a lo que se sabe del entorno. Se descartan opciones que a priori serían válidas pero no son una buena opción porque implican volver hacia atrás en mi resolución.

Algoritmo de Manhattan: Hace una búsqueda pero la cola de mis vecinos que puedo recorrer está ordenada de acuerdo a esa distancia más corta. El primer estado de la cola es el que tiene la distancia más corta, entonces se va a priorizar recorrer en base a los que tengan distancia más cortas. La distancia se calcula entre el vecino y el objetivo.

Metaheurísticas

Familia de técnicas de optimización aproximada para problemas complejos, son consideradas Soluciones "aceptables" y con Tiempo "razonable". Son procedimientos con heurísticas en dos niveles. Se pueden ver como métodos de búsqueda de nivel superior o metodologías generales.

Algoritmos heurísticos: Los algoritmos heurísticos son los más fáciles de utilizar, ya que se basan en el conocimiento de una heurística que guía el proceso de búsqueda. El conocimiento del problema usualmente ayuda a encontrar una heurística razonable que encontrará rápidamente una solución aceptable. Un algoritmo de este tipo sólo buscará dentro de un subespacio del área total a una solución buena (que no necesariamente es la mejor) que satisfaga las restricciones impuestas. La principal limitación es su incapacidad para escapar de los óptimos locales (encontrar soluciones parcialmente óptimas).

Algoritmos metaheurísticos: Una metaheurística es un proceso iterativo maestro que guía y modifica las operaciones de una heurística subordinada para producir eficientemente soluciones de alta calidad. Las metaheurísticas pueden manipular una única solución completa (o incompleta) o una colección de soluciones en cada iteración. La heurística subordinada puede ser un procedimiento de alto o bajo nivel, una búsqueda local, o un método constructivo. Entre los algoritmos metaheurísticos más conocidos están, el recorrido simulado y los algoritmos genéticos. Los cuatro tipos fundamentales:

1. Las metaheurísticas de **relajación** se refieren a procedimientos de resolución de problemas que utilizan relajaciones del modelo original (es decir, modificaciones del modelo que hacen al problema más fácil de resolver), cuya solución facilita la solución del problema original. Se prueba con un modelo no tan restrictivo.
2. Las metaheurísticas **constructivas** se orientan a los procedimientos que tratan de obtener una solución a partir del análisis y selección paulatina de las componentes que la forman. Se divide el problema en subproblemas y se va solucionando cada uno de manera individual.

3. Las metaheurísticas de **búsqueda** guían los procedimientos que usan transformaciones o movimientos para recorrer el espacio de soluciones alternativas y explorar las estructuras de entornos asociadas.
4. Las metaheurísticas **evolutivas** están enfocadas a los procedimientos basados en conjuntos de soluciones que evolucionan sobre el espacio de soluciones.

Metaheurísticas evolutivas

Las metaheurísticas evolutivas establecen estrategias para conducir la evolución en el espacio de búsqueda de conjuntos de soluciones (usualmente llamados poblaciones) con la intención de acercarse a la solución óptima con sus elementos.

El aspecto fundamental de las heurísticas evolutivas consiste en la interacción entre los miembros de la población frente a las búsquedas que se guían por la información de soluciones individuales.

Las diferentes metaheurísticas evolutivas se distinguen por la forma en que combinan la información proporcionada por los elementos de la población para hacerla evolucionar mediante la obtención de nuevas soluciones.

Los algoritmos genéticos y meméticos y los de estimación de distribuciones emplean fundamentalmente procedimientos aleatorios, mientras que las metaheurísticas de búsqueda dispersa o de reencadenamiento de caminos (Path-Relinking) emplean procedimientos sistemáticos. Los algoritmos genéticos pertenecen al grupo de los algoritmos evolutivos y están inspirados en la selección natural.

Fundamentos

En ocasiones la computación se basa en procesos observados de la naturaleza para resolver ciertos problemas: redes neuronales que replican los procesos de sinapsis entre las neuronas. En este caso, los algoritmos genéticos replican el modelo de selección natural propuesto por Darwin, y que resume la famosa frase “la supervivencia del más fuerte o adaptado”.

Este modelo dice que, dentro de una población, los individuos que sobreviven son aquellos que están más adaptados al medio, por lo tanto, las generaciones futuras de estos estarán mejor adaptadas ya que serán combinaciones de los mejores genes de sus antepasados. Además, esta teoría de la evolución introduce el concepto de las mutaciones. *Una mutación es un pequeño cambio que se produce de manera aleatoria en ciertos individuos e introduce de esta manera la versatilidad en las poblaciones*. Habrá mutaciones que den lugar a cambios favorables y otros desfavorables.

Usos: Se utilizan para resolver problemas de Búsqueda y Optimización, ya que se basan en evolucionar poblaciones de soluciones hacia valores óptimos del problema.

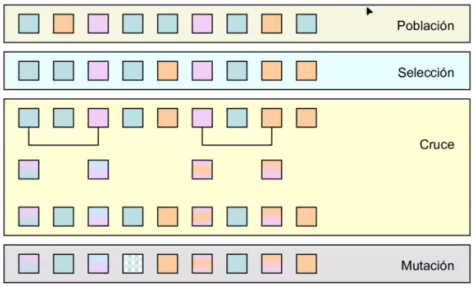
Algoritmos genéticos - Definiciones y conceptos

- Son algoritmos que tratan de simular la evolución biológica y se basan en conceptos similares de genética-molecular.
- Estos algoritmos hacen evolucionar una población de individuos por medio de operaciones de mutación, recombinación y selección. Esos tres elementos se aplican sobre cada uno de los datos que tenemos en el algoritmo y se trata de hacer que los valores de esos elementos (cromosomas) evolucionen en el tiempo para llevarnos a una solución del problema que estamos planteando.
- Por medio de sucesivas iteraciones se van obteniendo las mejores soluciones a un problema, mientras que las menos eficientes se van descartando. Se intenta mantener un conjunto de datos fijo, generando datos de más calidad y descartando datos que no son buenos para el algoritmo. Al evolucionar un conjunto de datos no es necesario ampliar la cantidad de datos de entrenamiento (el propio algoritmo deja los mejores datos y descarta los que no son útiles), lo que es muy útil en problemas grandes.

La población de individuos se debe mantener estable, con lo cual si en un paso se eliminan, en otro se tienen que agregar individuos.

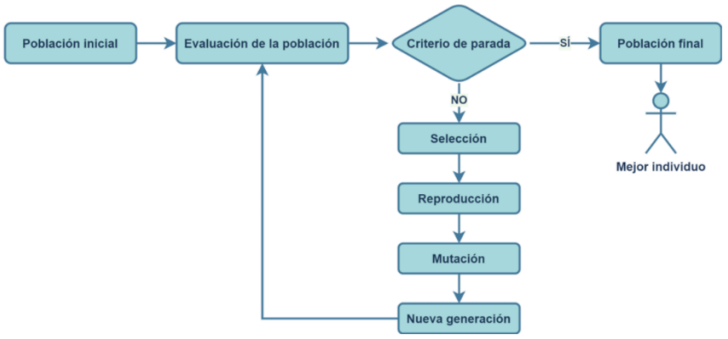
Estructura de un algoritmo genético

Conceptos previos



- Individuo: Posibles soluciones al problema que se intenta resolver.
- Población: Conjunto de datos que se va a utilizar para el algoritmo.
- Función fitness o de adaptación: Función que evalúa a los individuos y les asigna una puntuación en función de qué tan buenas sean las soluciones para el problema. Conjunto con sólo algunos individuos de la población, que se seleccionan según algún criterio. El resto de los individuos se descartan.
- Función de cruce: Función que dados dos individuos genera dos descendientes a partir de la combinación de genes de sus padres, esta función depende del problema en cuestión. Para mantener la población estable (en cuanto a cantidad) se deben generar individuos al mismo tiempo que se hace la selección. Luego se analiza si los nuevos individuos quedan o no en la población.
- *Mutación:* Cambio en las características de un individuo de una población. Es un cambio atípico y aleatorio, no es previsible, que permite obtener nuevos resultados. Se toma un

cromosoma y se le produce algún cambio aleatorio de acuerdo a alguna distribución de probabilidades. Las mutaciones tienen una probabilidad de aparición muy baja así que no se puede hacer que en cada iteración del algoritmo mute elementos porque no sería representativo de la realidad.



Estructura de un algoritmo genético

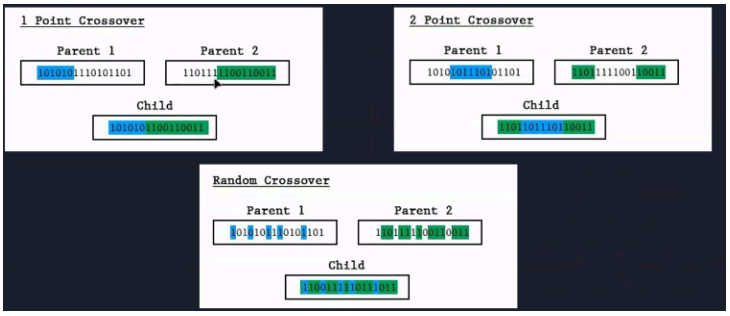
Fases

1. *Fase inicial*: se genera una población inicial de individuos (soluciones). En la naturaleza ya contamos con una población inicial, pero en nuestro problema artificial deberemos generar una. Idealmente se genera una cierta cantidad de "individuos", al azar, y cumpliendo con las características que buscamos (valores máximos, mínimos, medios, etc.). La población es la cantidad de datos que se pueden utilizar para resolver un problema determinado. Generalmente esa generación se hace construyendo cromosomas artificiales, los cuales son un dato de algún tipo que representa la información que yo tengo que analizar para resolver un problema, puede ser un valor, un vector de valores, un vector de valores 1 y 0, vector de características en una imagen, etc.

   Necesariamente se debe definir la población inicial y generarla. Normalmente se hace de forma aleatoria. Como se trata de encontrar patrones en la población, en los algoritmos genéticos es mejor que se generen de manera aleatoria.
2. *Fase de evaluación:* se evalúan los individuos de la población con la función fitness. Cada uno de los individuos se pone a prueba utilizando alguna función de optimización. A continuación se aplica algún criterio de corte que elimina los individuos que no producen valores cercanos a la función objetivo. La función objetivo es la evaluación de qué tan apto es cada uno de los individuos para solucionar el problema.
3. *Fase de selección:* se seleccionan los mejores individuos. Se evalúan cada uno de los cromosomas y se define qué tanto aportan a la solución del problema. Cuando se aplica en un algoritmo genético un algoritmo de selección se debe determinar el criterio se va a utilizar para seleccionar cuales son los elementos que van a pertenecer en la solución y cuáles son los que se descartan.
4. *Fase de reproducción - Crossover:* se cruzan los individuos seleccionados mediante la función de cruce, dando lugar a una nueva generación que va a sustituir a la anterior. Puede ser por cruce o mutación.

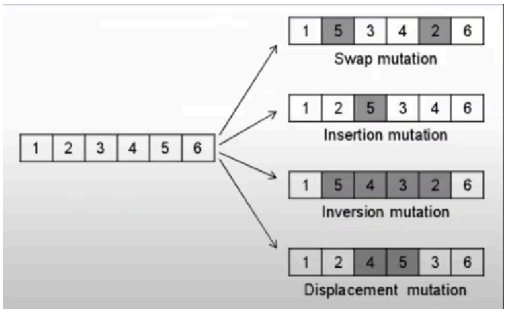
Tres ejemplos de técnicas de crossover (hay muchas más):

- One Point Crossover: Toma la primera mitad del cromosoma del padre 1, la segunda mitad del cromosoma del padre 2 y las fusiona generando uno nuevo que tiene los primeros componentes del padre 1 y los segundos del padre 2.



- Two Point Crossover: Toma un pedazo del cromosoma del padre 2, un pedazo del cromosoma del padre 1 y otro pedazo del cromosoma del padre 2, intercalando las piezas en el resultado.
- Random Crossover: toma de manera aleatoria bits o datos del padre 1 y 2 y se los transmite al hijo. Se puede asegurar que en dos generaciones consecutivas las características que se transmiten de uno al otro son distintas. Cruzando los mismos padres se pueden obtener distintos hijos, cosa que no pasa en los dos primeros casos.

5. *Fase de mutación:* se introducen mutaciones (pequeños cambios) en ciertos individuos de la nueva población de manera aleatoria (utilizando alguna función de probabilidad no demasiado alta) o un entrecruzamiento cromosómico (también llamado crossover o recombinación). Una vez generada la mutación, el individuo se incluye nuevamente en la población para su análisis.
    a. Swap mutation: Se define como se va a mutar. Intercambia elementos.
    b. Insertion mutation: Se saca un bit de cualquier lado y lo inserta en otro lugar, se modifican y desplazan el resto de los datos.
    c. Invertion mutation: Toma conjunto de bits del cromosoma original e invierte el orden.
    d. Displacement mutation: Se desplaza un bit o conjunto a la izquierda o derecha y se acomoda el conjunto.



    La mutación no produce un nuevo individuo, solo altera un individuo existente. Por ende, no modifica el tamaño de la población como sí lo hacen los otros dos pasos (selección y cruce).
6. *Se obtuvo una nueva generación*, en general, con soluciones mejores que la anterior. Se vuelve al punto 2.

Los algoritmos genéticos pueden finalizar cuando se alcanza un número de generaciones concreto o cuando cumplen una condición separada.

Ventajas

- Se desenvuelven bien en problemas con un paisaje adaptativo complejo: aquéllos en los que la función de aptitud es ruidosa, cambia con el tiempo, o tiene muchos óptimos locales, gracias a los cuatro componentes principales de los algoritmos genéticos, paralelismo, selección, mutación y cruzamiento, los que trabajan juntos para conseguir su buen desempeño.
- Pueden explorar el espacio de soluciones en múltiples direcciones a la vez, por lo que, los algoritmos genéticos funcionan bien resolviendo problemas cuyo espacio de soluciones potenciales es realmente grande, demasiado para hacer una búsqueda exhaustiva en un tiempo razonable.
- Los algoritmos genéticos realizan cambios aleatorios en sus soluciones candidatas y luego utilizan la función de aptitud para determinar si esos cambios producen una mejora. Como sus decisiones están basadas en la aleatoriedad, todos los caminos de búsqueda posibles están abiertos; en contraste a cualquier otra estrategia de resolución de problemas que dependa de un conocimiento previo.

Desventajas

- Si se elige mal una función de aptitud o se define de manera inexacta, puede que el algoritmo genético sea incapaz de encontrar una solución al problema, o puede acabar resolviendo el problema equivocado.
- Pueden tardar mucho en converger, o no converger, dependiendo en cierta medida de los parámetros que se utilicen como tamaño de la población, el ritmo de mutación y cruzamiento, el tipo y fuerza de la selección.
- El lenguaje utilizado para especificar soluciones candidatas debe ser robusto; es decir, debe ser capaz de tolerar cambios aleatorios que no produzcan errores fatales o resultados sin sentido. Una de las formas más usadas es definir a los individuos como listas de *números-binarios, enteros o reales- donde cada número representa algún aspecto de la solución candidata.*

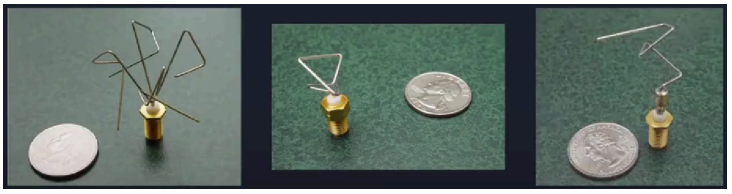
Algunas características que difieren entre los métodos:

- Inspiradas (o no) en la naturaleza
- Con o sin memoria
- Deterministas o estocásticos
- Poblacionales o individuales
- Iterativos o avaros

Productos de algoritmos genéticos. La antena evolucionada de la NASA.

El modelo de antena final maximiza la emisión y minimiza el espacio ocupado.

La antena debe tener un tamaño determinado por el lugar donde se debía colocar y se tenía que diseñar una antena que maximice la potencia sin exceder ese tamaño. Se probó con un algoritmo genético, donde se le indicó que se tenía que maximizar la potencia de la antena y minimizar el espacio.



La imagen más a la derecha es el diseño óptimo de la antena. Los otros dos diseños no óptimos generados por algoritmo genético.

Un algoritmo genético SOLO debe tener una sola función objetivo. Puede haber objetivos compuestos dentro de la función objetivo. La función objetivo es lo que permite la selección. Utilizando la función objetivo, se evalúa cada miembro de la población y se obtiene su "aptitud" para alcanzar esa función objetivo. Entre dos individuos, se selecciona el más apto (el que maximiza la función objetivo) y se elimina el menos apto.

Planificación

La planificación en términos cotidianos implica anticipar pasos para resolver un problema antes de ejecutarlos. En computación, cuando se trabaja de manera aislada, la distinción entre planificación y ejecución se difumina, haciendo que problemas de búsqueda de trayectorias se consideren casos de *planificación, ya que esta se puede ver como una búsqueda en un espacio de estados con el agregado de que lo que interesa es el camino recorrido entre los estados alcanzados hasta llegar al estado objetivo.* Sin embargo, el problema de una buena planificación puede requerir otras herramientas y conceptos además de estos.

En forma general, cuando añadimos este problema de tener que planificar (y ejecutar) una secuencia de pasos (operaciones, cambios de estado, etc.) en un mundo no completamente previsible, nos ubicamos frente al problema de la Planificación en IA. A pesar de definir un "mundo" para la planificación, es posible que no se puedan prever todos los estados futuros, lo que presenta obstáculos para el planificador. Este desafío se estudia mediante "mundos" simulados, siendo el "Mundo de los Bloques" uno de los más famosos para probar mecanismos de planificación.

Planificación en IA	Planificación en Robótica
* Numérica * Lógica Por pila de objetivos No lineal por fijación de restricciones Jerárquica. Se resuelven primero las tareas de mayor complejidad sin considerar detalles. * Reactiva	* Reactiva * Planificación de trayectorias en manipuladores Método de articulaciones interpoladas Método del espacio cartesiano

El problema de la planificación: Corresponde al área de resolución de problemas. Usualmente se ocupa de problemas "muy complejos" en el sentido conocido de que no pueden explorarse todas las opciones.

Se puede definir el proceso de resolución de un problema como una búsqueda en un espacio de estados donde:

- Cada estado se corresponde con una situación posible.
- La búsqueda comienza en una situación inicial.
- Se lleva a cabo una secuencia de operaciones permitidas hasta alcanzar una situación objetivo.

Entradas: Descripción del estado del mundo, Descripción del Objetivo, Conjunto de acciones.
Salida: Una secuencia de acciones que pueden ser aplicadas al estado, hasta alcanzar la descripción del estado final.

El Mundo de los Bloques

Las técnicas de planificación pueden aplicarse a una gran variedad de dominios, sin embargo, para hacer un estudio de los métodos y poder compararlos, resulta útil trabajar en un único dominio complejo que permita observar todos los mecanismos.

En el mundo de los bloques existe una superficie plana sobre la que se sitúan los bloques. Los bloques se pueden colocar unos sobre otros. Un manipulador puede tomar y reubicar los bloques.

Primero, se definen las acciones que puede llevar a cabo el manipulador, que son los operadores asociados a las reglas. Luego se consideran los predicados que se emplearán para definir los estados posibles.

Acciones del manipulador (operadores)

- (desapilar A B)
- (apilar A B)
- (levantar A)
- (bajar A)

Predicados

- (apilado A B)
- (desapilado A B)
- (sobrelamesa A)
- (despejado A)
- (agarrado A)

Se escribe de la forma *Pred_A(x,y,z)*

Definición de los estados

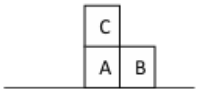
Estado Inicial:

- (sobrelamesa A)
- (apilado C A)
- (sobrelamesa B)
- (despejado B)
- (despejado C)

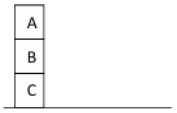
Estado Objetivo:

- (apilado A B)
- (apilado B C)

Estado inicial



Estado final



Codificación de operadores

Para poder pasar de un estado a otro, modificando predicados, se usan operadores, cada operador puede describirse mediante una lista de nuevos predicados que el operador provoca que sean ciertos y una lista de los viejos predicados que el operador provoca que sean falsos. Estas listas se denominan AÑADIR y BORRAR. Esto es lo que deberá codificarse en los módulos correspondientes a los operadores.

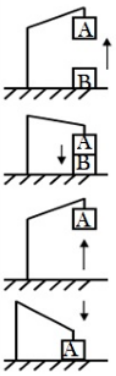
(desapilar A B)

P: (apilado A B) y (despejado A)
A: (agarrado A) y (despejado B)

(apilar A B)
P: (despejado B) y (agarrado A)
A: (apilado A B)

(levantar A)
P: (despejado A) y (sobrelamesa A)
A: (agarrado A)

(Bajar A)
P: (agarrado A)
A: (sobrelamesa A) y (despejado A)



Descripción de la situación inicial y su transformación mediante un operador

Dado el estado inicial:

- (apilado A B) y (sobrelamesa B) y (despejado A)
- aplicando: (desapilar A B)
- queda: (despejado B) y (agarrado A) y (sobrelamesa B)

Anomalía de Sussman

Los métodos de planificación pueden incluir pasos innecesarios en el plan. Esta anomalía se da cuando se hace una acción que no es la que conduce al objetivo. Cuando ocurre esto se debe volver para atrás, se deshace la acción que se hizo y se ejecuta otra acción válida.

El método explicado puede dar lugar a un comportamiento anómalo consistente en la inclusión en el resultado, (plan) de pasos que no llevan a la solución, y que por ese motivo deben anularse posteriormente a haberlos incluido. Esto puede dar lugar a ciclos.

Planificación no lineal mediante fijación de restricciones

El método de planificación con pila de objetivos aborda los problemas como objetivos conjuntos resolviendo por orden los objetivos uno cada vez. Este método genera un plan que contiene una secuencia de operadores que resuelven el primer objetivo, seguido por una secuencia completa para el segundo objetivo, etc.

Pero los problemas difíciles provocan interacciones entre los objetivos. Los operadores que se utilizan para resolver un problema pueden interferir en la solución de un subproblema anterior. *La mayoría de los problemas necesitan un plan entrelazado en el que se trabaje simultáneamente con múltiples subproblemas.* Este tipo de plan se denomina plan no lineal ya que no está compuesto por una secuencia lineal de subplanes completos.

Ambiente en la planificación

En este caso, el ambiente es: Determinista, Completamente observable, Finito, Estático y Discreto.

*Método A**

El método A* resuelve problemas del tipo mencionado

- Cada estado se corresponde con una situación posible.
- La búsqueda comienza en una situación inicial.
- Se lleva a cabo una secuencia de operaciones permitidas hasta alcanzar una situación objetivo.

Pero, tiene el problema que la manipulación de la descripción completa de un estado es posible solo para problemas sencillos y en problemas complejos es preferible trabajar por separado con partes pequeñas del problema y combinar las soluciones parciales al final.

Existen dos formas importantes de descomponer: Recálculo parcial de estados y división del problema en subproblemas.

Planificación mediante la aplicación de reglas

- Elegir el mejor operador aplicable (según función heurística)
- Aplicar operador
- Detectar si se ha llegado a una solución
- Repetir

Planificación por pila de objetivos

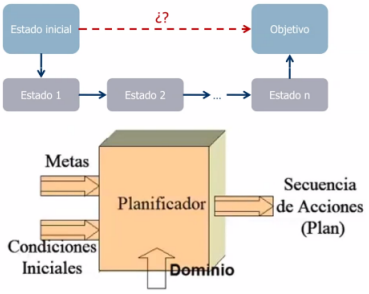
Se crea una pila y se van colocando todos los operadores y predicados que se necesitan para resolver el problema. Se van apilando los pasos y las decisiones que se toman y cada vez que se ejecuta una decisión se saca de la pila, cuando la pila queda vacía se llegó al estado objetivo.

Se utiliza una estructura de pila donde se insertan predicados y operadores. Cada vez que se inserta un predicado compuesto, se deben apilar además los predicados atómicos. Los pasos son:

- Inicializar la pila con el estado objetivo
- Repetir hasta que la pile esté vacía:
    - Sacar el primer elemento.
    - Si es un operador, aplicarlo.
    - Si es un predicado verdadero, eliminarlo.
    - Si es un predicado atómico falso, apilar un operador que haga que se cumpla (ver lista A) y apilar su lista P.
    - Si es un operador compuesto falso, volver a insertarlo.

Agentes planificadores

Los agentes planificadores crean y ejecutan un plan anticipado, considerando el estado actual (condiciones iniciales), los posibles estados futuros, las acciones disponibles y las metas a largo plazo. Interesa saber cuáles son los estados intermedios para saber el camino hacia el objetivo. En los algoritmos de búsqueda anteriores solo interesaba saber el estado objetivo o final, cuando se plantean los problemas de búsqueda no interesaba saber cómo se llegaba a dicho estado, en el planteo de agentes planificadores esto es diferente.



Los agentes planificadores crean y ejecutan un plan anticipado, considerando el estado actual (condiciones iniciales), los posibles estados futuros, las acciones disponibles y las metas a largo plazo.

Se necesita de los estados intermedios para saber el camino a seguir para cumplir con el objetivo. Necesita saber **cómo** se resuelve el problema y llegar a la solución final, no que solo se resuelva el problema.

Las condiciones de entorno determinan el contexto en el que opera el agente. Este contexto se refiere al ambiente donde se desarrolla el problema, es importante saber si es abierto o cerrado (cerrado es donde todos los cambios que se produzcan en el tiempo son producto de las

acciones del agente, es decir, no hay otros agentes y no hay elementos externos que alteran el estado del agente). Necesariamente todos los cambios que se produzcan en el agente en un ambiente cerrado son producidos únicamente por él, lo que implica que el ambiente es determinista; dado que no hay agentes externos que alteren el estado, se puede predecir el estado final y los estados intermedios.

Son sistemas o programas que toman decisiones sobre qué acciones realizar para alcanzar un objetivo o resolver un problema de manera eficiente. Estos agentes funcionan en el contexto de **planificación automática**, un área de la inteligencia artificial (IA) que se centra en la creación de secuencias de acciones (planes) que permiten al agente pasar de un estado inicial a un estado objetivo deseado. Las acciones son tan importantes como los resultados

Características

- *Tienen un objetivo:* Buscan alcanzar uno o más estados objetivos.
- *Trabajan con un modelo del mundo:* Usan un modelo que describe cómo las acciones afectan el entorno o los estados en los que pueden estar. Esto incluye una descripción del estado inicial, posibles acciones y las consecuencias de dichas acciones.
- *Generan planes:* El agente busca una secuencia de acciones que le permita pasar del estado actual al estado objetivo. Los planes pueden ser óptimos (los más eficientes en términos de tiempo, costo o recursos) o simplemente válidos (que logran el objetivo).
- *Evaluación y ajuste:* Pueden evaluar la calidad de los planes generados y ajustarlos si es necesario, por ejemplo, si el entorno cambia o si las acciones no producen los resultados esperados. Se puede tener un agente que tome decisiones extremadamente simples o complejas teniendo en cuenta muchas características.
- *Autonomía:* Estos agentes actúan de manera autónoma en entornos complejos, haciendo que las decisiones estén alineadas con la planificación establecida. Intentan generar planes cada vez mejor de acuerdo a las características del problema.

Ejemplos de agentes planificadores:

- Sistemas de navegación GPS: Estos agentes planifican la mejor ruta desde un punto de inicio hasta un destino, considerando el tráfico, distancias y otros factores. El GPS da los pasos intermedios resolviendo problemas más pequeños.
- Robots autónomos: Un robot que debe moverse con un entorno complejo puede planificar sus movimientos para evitar obstáculos y cumplir con su tarea.
- Juegos de ajedrez en IA: El agente planifica varias jugadas anticipadas para ganar el juego. Puede planificar distintos escenarios por ejemplo con 4 jugadas de anticipación, no se genera una partida completa pero se sabe cuáles van a ser las posibilidades dentro de esos pasos de juego y en base a ello evaluar cuales son los mejores. Mientras más jugadas pueda planificar mejor va a jugar.

Ejemplos de uso:

- Robótica. Un robot que planifica su ruta para llegar a un destino mientras evita obstáculos,
- Videojuegos: Personajes controlados por IA que planifican acciones para ganar un juego o cumplir una misión.
- Logística: Sistemas que planifican la distribución de mercancías de manera óptima.

Operadores STRIPS

Permite modelar cualquier conjunto de acciones de un planificador.

- Los operadores STRIPS (Stanford Research Institute Problem Solver) son una formalización utilizada para describir cómo las acciones afectan el estado del mundo en un problema de planificación.
- Definen las acciones que un agente planificador puede realizar, junto con las condiciones necesarias y las consecuencias de dichas acciones, para ayudar a resolver problemas de planificación. Plantea una técnica de cómo se pueden definir acciones y de cómo se pueden utilizar para resolver un problema de planificaciones.

Componentes

Está compuesto por tres partes fundamentales.

- *Precondiciones:* Son las condiciones que deben ser verdaderas en el estado actual para que la acción pueda ser ejecutada (**todas** las precondiciones deben ser verdaderas). Si estas precondiciones no se cumplen, la acción no puede realizarse (lista P), Ejemplo: Para realizar la acción de "abrir una puerta", la precondición podría ser que el agente esté frente a la puerta y que la puerta no esté bloqueada.
- *Efectos negativos (eliminados):* Son las condiciones que dejarán de ser verdaderas después de que la acción sea ejecutada (lista B). Ejemplo: Después de abrir la puerta, un efecto negativo sería que ya no es verdad que la puerta esté cerrada.
- *Efectos positivos (añadidos):* Describen los aspectos del estado que cambiarán cuando se realice la acción, es decir, qué nuevas condiciones se vuelven verdaderas después de ejecutar la acción (lista A). Ejemplo: Al abrir una puerta, un efecto positivo sería que la puerta ahora está abierta. Condiciones que eran falsas antes de que la acción se ejecute y que van a ser verdaderas después que se ejecute.

Ejemplo:

Supongamos que el agente quiere ejecutar la acción "moverse de un punto A a un punto B". operador STRIPS podría definirse de la siguiente manera:

- Acción: Moverse de A a B.
- Precondiciones: El agente está en A. No hay obstáculos entre A y B.
- Efectos negativos: El agente ya no está en A.
- Efectos positivos: El agente está en B.

Aplicación de reglas

Si se plantea el esquema anterior, los pasos para aplicar las técnicas al momento de tratar de resolver un problema de planificación son:

- *Elegir el mejor operador aplicable (según función heurística):* Dado un conjunto de posibles acciones y un estado actual del mundo se tendría que seleccionar de todas las acciones posibles aquellas que sea más beneficiosa para llegar al estado objetivo. Se debe ver cómo se define dado un conjunto de acciones cuál es la mejor para llegar al objetivo. Si se tiene un conjunto heurístico que permita calcular dado el conjunto de acciones cuál es mejor se lo va a aplicar, si no, se tendría que aplicar alguna de las reglas creyendo que todas aportan de la misma manera a la resolución del objetivo.
- *Aplicar operador:* Una vez que se decidió cuál es el operador que se debe aplicar con o sin heurística, se lo aplica. Esta aplicación cambia el estado del mundo.
- *Detectar si se ha llegado a una solución:* Se debe analizar si ese estado del mundo actual es igual al estado objetivo, es decir, saber si se encontró la solución. Se tienen dos alternativas:
    - Se cambió el estado del mundo y es una solución, en cuyo caso ya se terminó
    - Se cambió el estado del mundo y todavía no es una solución, en cuyo caso no se finalizó por lo que se debe repetir el proceso.
- *Repetir:* Cuando se repite el proceso el estado del mundo cambió, por lo que probablemente se tenga que aplicar otro operador diferente. Se vuelve al inicio y se analiza cuál es el nuevo estado y se debe determinar cómo se llega del nuevo estado al estado objetivo.

UNIDAD 4: RAZONAMIENTO EN AMBIENTES DETERMINISTAS

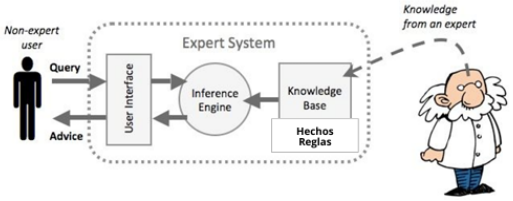
Agentes lógicos

Un agente lógico es un tipo de agente inteligente que toma decisiones y actúa en su entorno basándose en un proceso de razonamiento lógico. Utiliza la lógica formal para representar el conocimiento sobre su entorno, realizar inferencias y planificar sus acciones

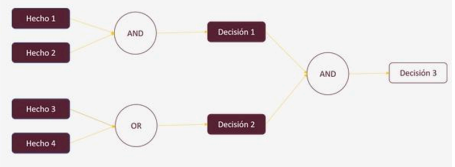
Características

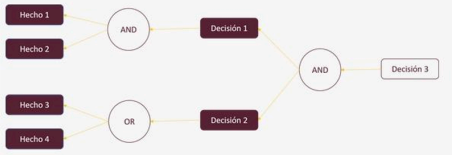
- *Representación del conocimiento:* Representa el conocimiento sobre su entorno en una base de conocimiento (BC) usando algún tipo de lógica formal, como la lógica proposicional o la lógica de predicados. La base de conocimiento contiene hechos (lo que el agente sabe) y reglas lógicas que describen cómo interactúan esos hechos.
- *Razonamiento:* Realiza inferencias a partir de su base de conocimiento. Utiliza reglas lógicas para deducir nuevas informaciones a partir de lo que ya sabe. Puede hacer deducciones (conclusiones lógicas basadas en hechos conocidos) y planificar sus acciones en función de lo que deduce.
- *Percepción y actualización:* Percibe el mundo a través de sus sensores y actualiza su base de conocimiento con nueva información. Puede usar su base de conocimiento actualizada para tomar nuevas decisiones.
- *Acciones basadas en lógica:* Elige sus acciones basándose en un razonamiento profundo. Busca determinar cuál es la mejor acción posible a partir de lo que sabe y lo que puede deducir.

Sistema Experto



Un sistema experto es una aplicación de inteligencia artificial diseñada para simular el conocimiento y el razonamiento de un experto en un campo específico. Estos sistemas funcionan mediante el uso de una base de conocimientos que contiene hechos y reglas, junto con un motor de inferencia que aplica esas reglas para deducir nueva información o tomar decisiones.





- **El encadenamiento hacia adelante** se utiliza en sistemas expertos de producción, donde se trata de inferir nuevas conclusiones a partir de hechos. Por ejemplo, en sistemas de diagnóstico médico, donde cada hecho adicional puede llevar a una nueva conclusión.
- **Encadenamiento hacia atrás** se emplea en sistemas expertos orientados a metas, donde se parte de un objetivo específico (por ejemplo, un

diagnóstico o una solución en particular) y se busca probarlo. Esto es útil en sistemas expertos de recomendación y consulta, como sistemas legales o de diagnóstico.

Agentes basados en conocimiento

El componente principal de un agente basado en conocimiento es su base de conocimiento, una BC es un conjunto de sentencias, cada una de las cuales se expresa en el lenguaje de representación del conocimiento y representa alguna afirmación acerca del mundo.

Debe haber un mecanismo para añadir sentencias nuevas a la base de conocimiento y uno para preguntar qué se sabe en la base de conocimiento. Ambas tareas requieren realizar inferencias, es decir, derivar nuevas sentencias de las antiguas.

El agente mantiene una base de conocimiento que inicialmente contiene algún conocimiento de antecedentes.

Cada vez que el programa del agente es convocado, realiza dos cosas. Primero dice a la BC lo que ha percibido, segundo pregunta a la BC que acción debe ejecutar, para lo cual se debe realizar un razonamiento extensivo acerca del estado actual del mundo, de los efectos de las posibles acciones, etc.

Las sentencias se expresan de acuerdo a la sintaxis del lenguaje de representación, que especifica todas las sentencias que están bien formadas. También se debe definir la semántica, la cual trata del significado de las sentencias. *La semántica del lenguaje define el valor de verdad de cada sentencia respecto a cada mundo posible.*

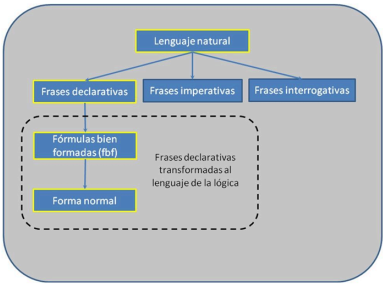
La **semántica** del lenguaje define el valor de verdad de cada sentencia respecto a cada mundo posible.

Los **modelos** son abstracciones matemáticas que nos permiten definir la verdad o falsedad de cada sentencia que sea relevante.

La relación de implicación lógica entre las sentencias cuya notación matemática es: $\alpha \models \beta$ para significar que la sentencia α implica la sentencia β es decir si α es verdadera β también lo debe ser.

Para comprender la implicación y la inferencia consideremos el conjunto de todas las consecuencias de la BC como un pajar y en α como en una aguja. La implicación es como la aguja que se encuentra en el pajar y la inferencia consiste en encontrarla. Si el algoritmo de inferencia i puede derivar α de la BC, escribimos: $BC \vdash_i \alpha$

Que se lee como α se deriva de la BC mediante i. Un algoritmo que deriva sólo sentencias implicadas es sólido. Un algoritmo de inferencia es completo si puede derivar cualquier sentencia que está implicada.



Si una BC es verdadera en el mundo real, entonces cualquier sentencia α que se derive de la BC, mediante un procedimiento de inferencia también será verdadera en el mundo real.

Lógica

La lógica es una disciplina que estudia métodos de formalización del conocimiento y formas válidas de inferencia.

Niveles de abstracción

La lógica proposicional y la lógica de predicados son dos ramas de la lógica formal que se utilizan para representar y razonar sobre hechos y relaciones, pero tienen diferencias clave en cuanto a su expresividad y capacidad.

- Lógica Proposicional: La lógica proposicional (lógica de enunciados) se basa en proposiciones simples que pueden ser verdaderas o falsas. Cada proposición es una afirmación completa, y no se puede descomponer en partes más pequeñas. Utiliza:
    - Predicados: Funciones que describen propiedades o relaciones entre objetos. Ejemplo: Mayor(a, b) puede expresar que "a es mayor que b".
    - Cuantificadores:
        - Cuantificador universal (∀): "Para todo" o "para cada".
        - Cuantificador existencial (∃): "Existe al menos uno".
- Lógica de Predicados: La lógica de predicados (lógica de primer orden) es más expresiva que la lógica proposicional porque no solo trabaja con proposiciones simples, sino que permite representar relaciones entre objetos y cuantificar sobre ellos.

Lógica proposicional

Sintaxis: La sintaxis de la lógica proposicional define las sentencias que se pueden construir. Las sentencias atómicas se componen de un único símbolo proposicional.

Cada uno de estos símbolos representa una proposición que puede ser verdadera o falsa. Utilizaremos letras mayúsculas para estos símbolos: P,Q, R, y siguientes.

Las proposiciones se conectan mediante operadores lógicos como:

¬ (no) Un literal puede ser una sentencia atómica (un literal positivo) o una sentencia atómica negativa (un literal negativo)

^ (y) Una sentencia que tenga como conectiva principal ^ se denomina conjunción, sus componentes son los conjuntores.

∨ (o) Una sentencia que utiliza la conectiva ∨ es una disyunción de los disyuntores (componentes)

→ (implica) Es una sentencia que posee una premisa o antecedente y una conclusión o consecuente.

Semántica: La semántica en lógica proposicional debe especificar cómo obtener el valor de verdad de cualquier sentencia, dado un modelo. Este proceso se realiza de forma recursiva. Todas las sentencias se construyen a partir de las sentencias atómicas y las conectivas lógicas. Hace falta definir el valor de verdad de las sentencias atómicas y cómo calcular el valor de verdad de las sentencias construidas con los conectores lógicos.

Verdadero es verdadero en todos los modelos y Falso es falso en todos los modelos. El valor de verdad de cada símbolo proposicional se debe especificar directamente para cada modelo. Para las sentencias complejas se tienen reglas como la siguiente: Para toda sentencia S y todo modelo M la sentencia ¬S es verdadera en M si y sólo si s es falsa en M.

Lógica proposicional

Tabla de verdad:

P	Q	¬P	P ∧ Q	P ∨ Q	P → Q	(P → Q) ∧ (Q → P)
V	V	F	V	V	V	V
V	F	F	F	V	F	F
F	V	V	F	V	V	F
F	F	V	F	F	V	V

¬P (No P): La negación de P. Su valor de verdad es el opuesto al de P. Si P es V, entonces ¬P es F. Si P es F, entonces ¬P es V.

P ∧ Q (P y Q): La conjunción de P y Q. Es verdadera sólo si ambas proposiciones son verdaderas. Es V solo si P y Q son verdaderas. Es F en los otros casos.

P ∨ Q (P o Q): La disyunción de P y Q. Es falsa sólo si ambas proposiciones son falsas. Es F sólo cuando P y Q son falsas. Es V en los otros casos.

P → Q (P implica Q): La condicional. Es falsa solo cuando P es verdadera y Q es falsa.
Es F solo si P verdadera y Q falsa. Es V en los otros casos.

(P → Q) ∧ (Q → P): La bicondicional. Es verdadera sólo cuando ambas proposiciones tienen el mismo valor de verdad. Es V si ambas son verdaderas o ambas falsas. Es F si tienen valores de verdad diferentes.

P → Q

P	Q	P → Q
V	V	V
V	F	F
F	V	V
F	F	V

Pensemos en la condicional como una promesa o un contrato. P → Q se lee como "Si P, entonces Q". Esta promesa solo se rompe si afirmas P (la promesa) y luego no cumples con Q (la consecuencia).

Cuando P es verdadera: Si afirmas P y luego Q es falsa, se rompió la promesa, por lo que la implicación es falsa.

Cuando P es falsa: Si nunca haces la afirmación inicial (P), no se puede romper la promesa, independientemente de lo que suceda con Q.

Satisfactibilidad: Una sentencia es satisfactoria si es verdadera para algún modelo. Si una sentencia α es verdadera en un *modelo m* entonces m satisface α, o m es un modelo de α. La satisfacibilidad se puede averiguar enumerando los modelos posibles hasta que uno satisface la sentencia.

Lógica proposicional Encadenamiento hacia adelante y hacia atrás

Las bases de conocimiento del mundo real contienen sólo cláusulas de tipo restringido, denominadas Cláusulas de Horn. Una cláusula de Horn es una disyunción de literales, de los cuales, como mucho uno es positivo, esta restricción es muy importante debido a tres razones:

- Las cláusulas de Horn con exactamente un literal positivo se denominan cláusulas positivas. El literal positivo se denomina cabeza y la disyunción de literales negativos cuerpo de la cláusula. Una cláusula positiva que no tiene literales negativos afirma un proposición dada que se denomina hecho. *Las cláusulas positivas forman la base de la programación lógica.*
- Cada cláusula de Horn se puede escribir como una implicación cuya premisa sea una conjunción de literales positivos y cuya conclusión sea un único literal positivo
- La inferencia con cláusulas de Horn se puede realizar mediante algoritmos de encadenamiento hacia adelante y de encadenamiento hacia atrás.
- Averiguar si hay o no implicación con las cláusulas de Horn se puede realizar en un tiempo que es lineal respecto del tamaño de la base de conocimiento.

El encadenamiento hacia adelante es un ejemplo del concepto general de razonamiento dirigido por los datos, es decir, un razonamiento en el que el foco de atención parte de los datos conocidos. Este razonamiento se puede utilizar en un agente para derivar conclusiones a partir de percepciones recibidas, a menudo, sin una petición concreta.

El encadenamiento hacia atrás trabaja a partir de una petición. Si se sabe que la petición *q* es verdadera, entonces no requiere ningún trabajo. El algoritmo encuentra aquellas implicaciones de la base de conocimiento de las que se concluye q. *Si se puede probar que todas las premisas de una de esas implicaciones son verdaderas entonces q es verdadero.*

Lógica de primer orden

La lógica de primer orden representa una buena parte del conocimiento de sentido común. El lenguaje de la lógica de primer orden está construido sobre objetos y relaciones.

Sintaxis y semántica de la lógica de primer orden

Modelos en lógica de primer orden

Los modelos de la lógica de primer orden contienen objetos. El dominio de un modelo es el conjunto de objetos que contiene, a estos objetos se les denomina elementos del dominio.

Sintaxis: Símbolos e interpretaciones: Los elementos sintácticos básicos son los símbolos que representan los objetos, las relaciones y las funciones. Los símbolos se agrupan en tres tipos: símbolos de constante, que representan objetos; símbolos de predicado, que representan relaciones; y símbolos de función que representan funciones.

La semántica debe relacionar las sentencias con los modelos para determinar su valor de verdad. Para que esto ocurra, se necesita de una interpretación que especifique exactamente qué objetos, relaciones y funciones son referenciados mediante símbolos de constante, predicado y de función.

Términos: expresión lógica que se refiere a un objeto. Por lo tanto, los símbolos de constante son términos. Sentencias atómicas: representan hechos. Una sentencia atómica está compuesta por un símbolo de predicado seguido de una lista de términos entre paréntesis.

Una *sentencia atómica es verdadera* en un modelo dado y bajo una interpretación dada, si la relación referenciada por el símbolo de predicado sucede entre los objetos referenciados por los argumentos.

Sustitución: Si hay alguna sustitución Θ que haga que las premisas de la implicación sean idénticas a algunas de las sentencias que ya están en la base de conocimiento, se puede afirmar la conclusión de la implicación, después de aplicar Θ.

Encadenamiento hacia adelante

Las cláusulas positivas de primer orden son disyunciones de literales de las cuales *sólo uno es positivo*. Una cláusula positiva es atómica o es una implicación cuyo antecedente es una conjunción de literales positivos y cuyo consecuente es un único literal positivo. Los literales de primer orden pueden contener variables, cuyo caso se asume que dichas variables están cuantificadas universalmente.

Encadenamiento hacia atrás

Estos algoritmos trabajan hacia atrás desde el objetivo, encadenando a través de las reglas hasta encontrar los hechos conocidos que soportan la demostración.

El algoritmo se invoca con una petición y devuelve el conjunto de todas las sustituciones que satisfacen la petición. La lista de objetivos se ve como una pila a la espera de ser procesada, si todos los objetivos se pueden satisfacer, entonces la rama actual de la demostración tiene éxito.

El algoritmo toma el primer objetivo de la lista y encuentra cada cláusula de la base de conocimiento cuyo literal positivo, o cabeza, se unifica con él.

Cada una de las cláusulas crea una nueva llamada recursiva en la que las premisas o cuerpo, se añaden a la pila de objetivos. Los hechos son cláusulas con cabeza y sin cuerpo, cuando el objetivo se unifica con un hecho conocido, no se añaden más subobjetivos a la pila y el objetivo se resuelve.

Prolog

Los programas en Prolog son conjuntos de cláusulas positivas escritas en una notación diferente a la estándar de la lógica de primer orden. Prolog utiliza las letras mayúsculas en las variables y las minúsculas en las constantes. Las cláusulas están escritas con la cabeza

precediendo al cuerpo, las comas separan los literales en el cuerpo, y el punto indica el final de la sentencia.

Ejemplo: criminal(X) :- americano (X), arma(Y) , vende(X,Y,Z), hostil(Z)

La ejecución de los programas se hace mediante el encadenamiento hacia atrás. El Prolog permite un tipo de negación denominada negación por fallo. *Un objetivo negado no P se considera demostrado si el sistema falla al probar P.*

Resolución

Formas normales conjuntivas en lógica de primer orden

La resolución en primer orden requiere que las sentencias estén en la forma normal conjuntiva (FNC), es decir, una conjunción de cláusulas, donde cada cláusula es una disyunción de literales. La sentencia en FNC será insatisfacible sólo cuando la sentencia original lo sea, así se dispone de una base para hacer demostraciones mediante contradicción con sentencias en FNC.

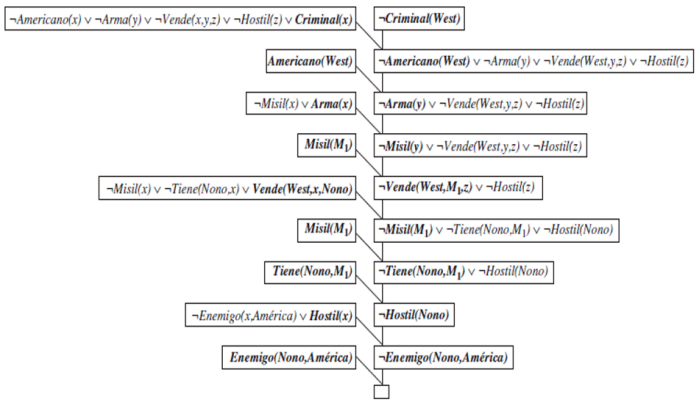
Procedimiento de conversión a la FNC

- Eliminación de las implicaciones
- Anidar las negaciones
- Estandarizar las variables
- Skolenizar: eliminar los cuantificadores existenciales
- Eliminar los cuantificadores universales
- Distribuir la ^ respecto de la ∨

Dos cláusulas que se asumen están con las variables estandarizadas, y así no comparten ninguna variable, se pueden resolver si contienen literales complementarios. *Los literales en LPO son complementarios si uno se unifica con la negación del otro.*

Ejemplo: La resolución demuestra que BC |= α probando que BC^¬α es insatisfacible derivando la cláusula vacía. Las sentencias en FNC son:

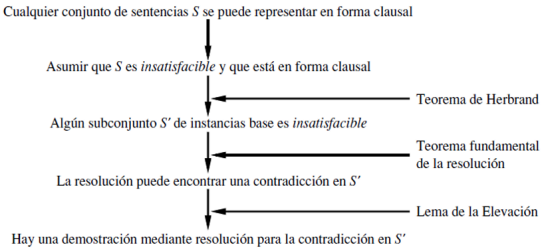
$\neg Americano(x) \vee \neg Arma(y) \vee \neg Vende(x, y, z) \vee \neg Hostil(z) \vee Criminal(x)$
$\neg Misil(x) \vee \neg Tiene(Nono, x) \vee Vende(West, x, Nono)$
$\neg Enemigo(x, América) \vee Hostil(x)$
$\neg Misil(x) \vee Arma(x)$
$Tiene(Nono, M_1).$ $\quad Misil(M_1)$
$Americano(West).$ $\quad Enemigo(Nono, América)$



Lógica de primer orden Completitud de la Resolución:

La resolución es un procedimiento de refutación completo, sí un conjunto de sentencias es insatisfacible, entonces la resolución siempre será capaz de derivar una contradicción. La resolución se puede utilizar para establecer si una sentencia se deduce de un conjunto de sentencias.

Si S es un conjunto de cláusulas insatisfacibles, entonces la aplicación de un número finito de pasos de resolución sobre S nos dará una contradicción.



Formas clausales

- Forma normalizada conjuntiva (FNC). Conjunción de disyunciones. *(P ∨ Q) ∧ (-P ∨ R)*
- Forma normalizada disyuntiva (FND). Disyunción de conjunciones. *(P ∧ Q) V (-P ∧ R)*
- Cláusulas de Horn. Conjunción de disyunciones con no más de un literal positivo (no más de una implicación). *(-P ∨ -Q ∨ R) ∧ (-T V P)*

Conversión a forma clausal - Forma normalizada conjuntiva (FNC).

- Paso 1: Eliminar las implicaciones usando el hecho de que: $A \rightarrow B = -A \vee B$
- Paso 2: Reducir el ámbito de las negaciones a un solo término usando el hecho de que:*-(-P) = P-(A ∧ B) = -A ∨ -B-(A ∨ B) = -A ∧ -B*
- Paso 3: Convertir la fórmula en una conjunción de disyunciones mediante las propiedades asociativa y distributiva. *(A ∧ B) ∨ C = (A ∨ C) ∧ (B ∨ C) (A ∨ B) ∧ C = (A ∧ C) ∨ (B ∧ C) (A ∨ B) ∨ C = A ∨ (B ∨ C), (A ∧ B) ∧ C = A ∧ (B ∧ C)*. Al final, separar en cláusulas eliminando las conjunciones.

El procedimiento de resolución que se usa comúnmente en demostradores automáticos de teoremas y en lógica para agentes inteligentes. El procedimiento para producir una demostración de la proposición P respecto de un conjunto de axiomas F es el siguiente:

- Convertir todas la FBF de F a forma clausal.
- Negar P y convertir a forma clausal. Añadir las cláusulas obtenidas al conjunto.
- Hasta que se encuentre una contradicción o no se pueda seguir avanzando, repetir:
    - Seleccionar dos cláusulas y llamarlas cláusulas padre.
    - Calcular el resolvente
    - Si el resolvente es la cláusula vacía, se encontró una contradicción (P es cierto). Si no, añadirlo al conjunto de cláusulas.

(Si se encuentra una contradicción, significa que lo que se quería probar era cierto. Si no se llega a la contradicción, no se puede sacar conclusiones. P puede ser cierto o falso).

UNIDAD 5: RAZONAMIENTO BAJO INCERTIDUMBRE

Razonamiento bajo incertidumbre

Capacidad de los sistemas de IA para tomar decisiones o hacer inferencias cuando la información disponible es incompleta, ambigua o incierta. Si no se sabe los valores que toma una variable o cuáles son los posibles valores de verdad se está bajo incertidumbre.

En el mundo real, muchas situaciones no proporcionan datos exactos o seguros, lo que hace que el razonamiento tradicional (basado en lógica estricta y reglas deterministas) no sea suficiente. Se plantea un modelo que permite hacer razonamiento bajo incertidumbre o razonar bajo incertidumbre porque no siempre se tiene la certeza de todo en un problema y de debe decidir en base a esa incertidumbre.

Los métodos y técnicas que manejan incertidumbre permiten a los sistemas de IA hacer predicciones, tomar decisiones, o inferir hechos, incluso cuando no tienen datos completos o claros, lo que mejora su aplicabilidad en diversas áreas. Las técnicas que se aplican ayudan a tomar decisiones con una cierta justificación de esa decisión. Saber hasta dónde alcanza la incertidumbre que se tiene y hasta donde alcanza la información cierta que se tiene sirve para tomar las distintas decisiones.

Desde el punto de la IA se debe tener en cuenta la incertidumbre, considerarla en el proceso de toma de decisiones y documentar (qué datos son inciertos y que decisiones se toman sobre esos datos inciertos)

Fuentes de incertidumbre:

La incertidumbre en la IA puede surgir por varias razones:

- *Datos incompletos o faltantes:* No se tiene toda la información relevante.
- Ruido en los datos: Los datos pueden contener errores o ser imprecisos. No se detectaron errores y por lo tanto no se sacaron del set de datos. Esos datos no van a ser reales y por lo tanto se debe meter un mecanismo en el sistema de entrenamiento para que procese ese ruido (minimizandolo, ignorándolo)
- *Ambigüedad:* La información puede tener múltiples interpretaciones. El set de datos puede ser interpretado de forma distinta (lo más leve) o de forma contradictoria (lo más grave) donde el mismo dato puede ser interpretado de diferente manera para dos personas distintas que lo analizan. Un algoritmo de razonamiento no puede tener información contradictoria.
- *Procesos estocásticos:* Muchas situaciones reales están influenciadas por elementos aleatorios o probabilísticos que no se conocen.
- *Conocimiento parcial:* Es posible que no se conozcan todas las reglas o leyes que rigen un sistema, por lo que hay que razonar con la información disponible. En un programa incierto se pueden tener reglas inciertas que alteran el funcionamiento.

Métodos para el razonamiento bajo incertidumbre en IA:

- *Lógica difusa (Fuzzy Logic)*
- *Probabilidades y redes bayesianas:* forma en la que se incluyen dentro del razonamiento las probabilidades. Qué probabilidades hay de que suceda algo y en base a ello decidir.
- *Algoritmos de aprendizaje automático:* Algoritmos donde dentro suyo incluyen incertidumbre.
- *Sistemas expertos con manejo de incertidumbre:* Se modela dentro del sistema lo que no se sabe, y no solamente lo que sí se sabe.

Ejemplos de razonamiento bajo incertidumbre:

- Diagnóstico médico: Un sistema de IA debe determinar la probabilidad de una enfermedad dada una serie de síntomas que pueden ser inciertos o imprecisos.
- Reconocimiento de voz: Los sistemas de reconocimiento de voz deben identificar palabras en medio de ruido o con pronuncias inexactas, 10 que introduce incertidumbre en la interpretación de 10 que se está diciendo.
- Robótica: Un robot debe tomar decisiones en un entorno donde sus sensores pueden ser inexactos o tener información incompleta sobre su entorno.

Lógica difusa

Cuando hablamos de lógica difusa, el adjetivo “difuso” se debe a que en esta lógica, los valores de verdad son no-deterministas y tienen, por lo general, una connotación de incertidumbre.

La lógica difusa es una extensión de la lógica clásica que permite manejar incertidumbre y ambigüedad. Introducida por Lotfi Zadeh en 1965. En lugar de valores binarios (0 y 1), permite grados de verdad que varían entre 0 y 1, donde 0 es totalmente certero que es falso, 1 totalmente con confianza que es verdadero y resto de los valores tienen alguna probabilidad de confianza que sea verdadero o falso.

Ejemplo: "Templado" puede ser un grado entre "frío" y "caliente". Puede ser que el templado no esté a la mitad de la escala, sino que un poco más arriba, pero depende de los datos que se tengan. Templado no va a ser ni 0 ni 1, sino un valor intermedio.

En la lógica clásica a su vez se tienen un conjunto de operadores que permiten combinar lo que se conoce en expresiones lógicas más complejas, como son los operadores AND, OR, NOT. Aun cuando se combinan expresiones simples en expresiones más complejas, el resultado de esas expresiones complejas, el resultado va a ser binario. Si en la lógica binaria se tienen valores de verdad verdadero - falso, se tienen expresiones simples que se evalúan en un valor verdadero - falso, se tienen expresiones compuestas complejas y operadores que se terminan evaluando en verdadero - falso, entonces si en la lógica difusa los valores van a ser entre 0 y 1, se puede

trasladar la lógica binaria a un entorno difuso en el cual puede tomar varios valores. Los operadores lógicos difusos deberían entregar como resultado algún valor entre 0 y 1.

Estructura

La lógica difusa tiene como base los conjuntos difusos y posee un sistema de inferencia basado en reglas de producción de la forma "si antecedente entonces consecuente", donde los valores lingüísticos del antecedente y el consecuente están definidos por conjuntos difusos.

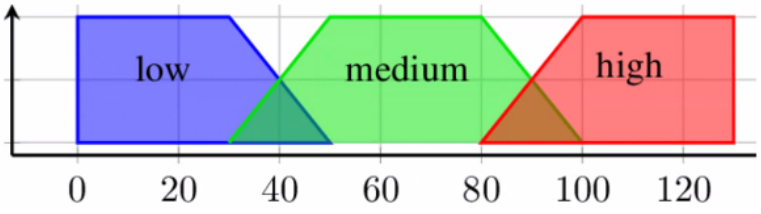
Aplicación de los sistemas basados en lógica: Control numérico, clasificación, soporte de decisión, visión computarizada, sistemas expertos.

Un sistema de inferencia difusa interpreta los valores de un vector de entrada y basado en un conjunto de reglas, asigna valores al vector de salida.

$$Caso\ general:\ \ Entrada\ -\ interpretación\ --{>}\ Salida\ -\ Asignación$$

La matemática difusa

- Las variables pueden tomar valores intermedios entre 0 y 1.
- Se representan con conjuntos difusos. Esos conjuntos vana a ser valores que abarcan rangos que se definen de acuerdo al problema que se tenga, pero si o si son rangos, no valores puntuales.
- Una función de membresía es una función que cuando se le asigna un valor a la variable x, devuelve un valor entre 0 y 1 que representa el grado de pertenencia a un conjunto difuso.

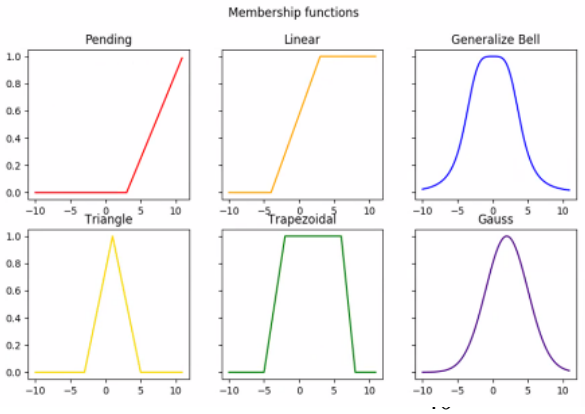


Cuando a la variable se la evalúa de acuerdo a algún criterio en base a un cierto valor da un grado de membresía que dice que tan probable es que la variable pertenezca a cierto conjunto.

En los cruces entre clases, cuando se evalúa el método da una posibilidad que esté en dos conjuntos separados y en cuanto porcentaje está en cada uno, ahí está la incertidumbre, lo difuso. Se debe determinar si pertenece a un conjunto, si pertenece a otro o si pertenece a los dos y en ese caso determinar cuanto pertenece al primero y cuánto al segundo.

Funciones de membresía

- *Pending - pendiente:* Se va a tener un valor a partir del cual la variable va a comenzar a pertenecer a cierto conjunto, a medida que se aumenta el valor de x va a pertenecer más al conjunto que se está analizando.
- *linear:* Se tiene un valor en donde a partir de él empieza a pertenecer y avanzando en x se tiene otro



valor que ya pertenece totalmente de ahí en adelante, cuando llega a dicho punto por más que siga avanzando en el punto x va a pertenecer totalmente al conjunto

- *Triangular:* En un punto empieza a pertenecer hasta llegar a un valor máximo y cuando pasa ese máximo sigue perteneciendo pero en menor medida hasta que llega a un punto que no pertenece más.
- *Trapezoidal:* sería una generalización de la lineal, en cierto punto comienza a pertenecer al conjunto, llega a un máximo en cierto valor, permanece en el máximo durante un cierto tiempo y a partir del siguiente punto máximo comienza a pertenecer cada vez menos hasta que llega a un valor 0 y luego no pertenece más al conjunto.
- *Gauss y Generalize Ball:* Son generalizaciones y para distribuciones continuas, no se tiene un punto concreto donde empieza a pertenecer, sino que arranca en 0, crece de a poco hasta que consigue alcanzar un máximo, luego sigue avanzando y empieza a decrecer de nuevo. Depende de las distribuciones de variables que se tenga.

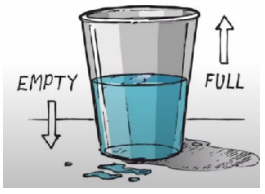
Los valores en y van entre 0 y 1 que significa el grado de pertenencia que se tiene al conjunto.

En el caso de la temperatura, si se tuviera que definir un tipo de función de membresía se podría utilizar la función pendiente porque si se tiene una temperatura es baja y comienza a subir, sube hasta que en un determinado momento deja de ser baja y luego más adelante no vuelve a ser baja. Lo mismo ocurre con la temperatura alta, donde cierta temperatura comienza a subir hasta que es alta y llega a su máximo.

Con la temperatura media podría ser una función trapezoidal o triangular, porque se tiene una temperatura que es baja hasta cierto punto, cuando llega a ese punto comienza a subir, es decir comienza a pertenecer cada vez más al conjunto de la temperatura media, hasta que llega a un máximo y luego cuando la temperatura siga subiendo comienza a bajar la pertenencia a la temperatura media porque se va a ir aproximando cada vez más a la temperatura alta.

<u>Lógica difusa vs Lógica booleana (crisp)</u>

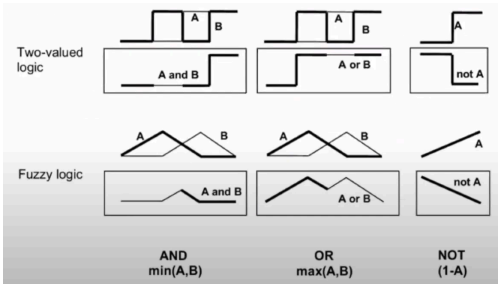
- Lógica booleana: Verdad absoluta (0 ó 1).
- Lógica difusa: Grados de verdad, permitiendo evaluar situaciones ambiguas o graduales.



- Ejemplo: En lógica booleana, un vaso puede estar lleno (1) o vacío (O); en lógica difusa, puede estar "parcialmente lleno" (0.5). En este caso los conjuntos difusos son “parcialmente lleno” y “parcialmente vacío” que dividen la capacidad del vaso en dos. En un problema la definición de los conjuntos es importante porque define lo que se está tratando de obtener.

Si se toman conjunto que son excluyentes y extremos se está volviendo a lógica booleana y se pueden tratar como tales. Los problemas binarios son un caso especial de los problemas de lógica difusa. No necesariamente se dejan de lado los problemas binarios.

Operadores



- AND (mínimo): Elige el menor valor entre dos entradas.
- OR (máximo): Elige el mayor valor entre dos entradas.
- NOT (complemento): Calcula el complemento de una entrada (1 -valor).

En la lógica difusa se tiene una función que tiene rangos/pendientes, se tiene una función que baja y sube de forma gradual. La pendiente que se tiene es la pertenencia al conjunto. Si se tienen dos operaciones como la pertenencia al conjunto A y al conjunto B y se tiene que unirlas con el equivalente difuso a un AND, se toma de los dos valores al menor. Para tomar el equivalente a un OR se toma de los dos valores el mayor y para tener el equivalente a un NOT (se evalúa con una sola función) se toma el complemento.

Si se tiene una función evaluada con NOT, a medida que vaya aumentando el grado de pertenencia, el NOT disminuye el grado de pertenencia.

Reglas difusas

Para tomar decisiones tenemos que basarnos en los valores difusos, los cuales van a decir "si pasa tal cosa hace tal otra".

- Se utilizan reglas IF-THEN para tomar decisiones basadas en condiciones difusas.
- IF temperatura es "alta" AND humedad es "alta" THEN ventilador = "rápido"

Estas reglas permiten modelar el comportamiento de sistemas complejos de manera sencilla. Se debe definir el rango en el que está cada conjunto para realizar las evaluaciones.

Una regla difusa toma la forma:

Si X es A luego Y es B. Donde A y B son valores lingüísticos definidos por conjuntos difusos sobre los rangos X e Y respectivamente. La primera parte de la regla es el antecedente o premisa, mientras que la parte final es el consecuente.

El Antecedente (servicio), es una interpretación que retorna un número entre 0 y 1, mientras que el Consecuente es una sentencia que asigna el conjunto fuzzy completo B a la variable de salida Y.

Etapas para la aplicación de las reglas de Lógica Difusa

- **Paso I - Fuzzificación de las entradas:** se deben resolver todas las sentencias en el antecedente en función de su grado de membresía entre 0 y 1. Convertir variables precisas (crisp) en variables difusas (fuzzy, entre 0 y 1).
- **Paso II - Aplicación de los operadores difusos:** si existen múltiples partes en el antecedente se aplican los operadores fuzzy y se resuelve el antecedente como un número entre 0 y 1. Si existen múltiples partes en el antecedente, se aplican operadores fuzzy (como AND, OR) para resolver el antecedente como un valor entre 0 y 1.

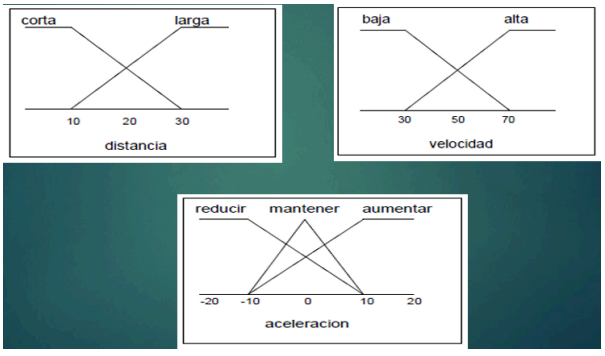
- **Paso III - Aplicación de la implicación:** el consecuente de una regla fuzzy asigna un conjunto fuzzy completo a la salida. Conjunto de reglas IF-THEN. Las reglas dicen que cuando se da tal condición se va a obtener cierto resultado. Se llegan a conclusiones a partir de premisas.
- **Paso IV - Agregación:** la salida de cada regla se "suma". Proceso de unificación de las salidas para cada regla. Evalúa las reglas y combina los resultados. Las reglas pueden cumplirse de forma simple (o se cumple ninguna o se cumple una) o se pueden cumplir varias reglas, y si se cumplen varias reglas hay que determinar cómo se combinan dichas reglas porque se dio por hecho que pasan varias cosas simultáneamente.
    - La entrada: Es la lista de salidas truncadas
    - La salida: Es un conjunto difuso para cada variable de salida
    - Consiste en tomar un único conjunto de salida para todo el sistema a partir de los conjuntos de salida de cada regla.
- **Paso V - Defuzificación:** para obtener como salida un número se aplica algún método de defuzificación. Convertir resultados difusos en valores precisos. Proceso para pasar los valores entre 0 y 1 con los que se trabajó el problema de vuelta a las variables reales de vuelta.

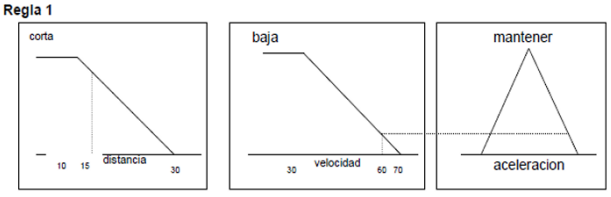


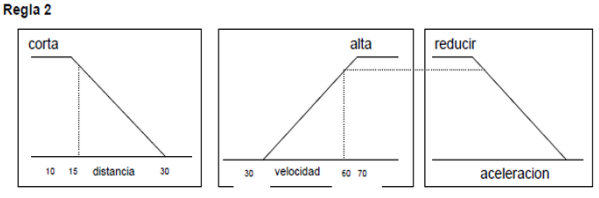


Lógica de conducción de un automóvil considerando la velocidad y la distancia entre autos

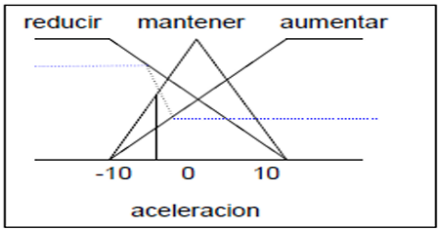
- Regla 1: Si la distancia entre autos es corta y la velocidad es baja mantener la velocidad
- Regla 2: Si la distancia entre autos es corta y la velocidad es alta reducir la velocidad
- Regla 3: Si la distancia entre autos es larga y la velocidad es baja aumentar la velocidad
- Regla 4: Si la distancia entre autos es larga y la velocidad es alta mantener la velocidad





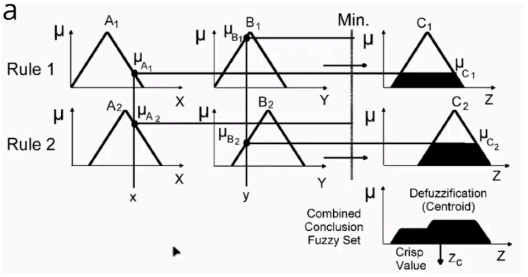


Por ejemplo si la distancia es de 15 metros y la velocidad es de 60 km el resultado es:

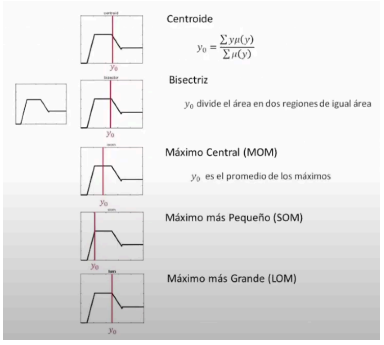


Inferencia difusa

Consiste en la evaluación de las reglas difusas y el uso de operadores para combinar los grados de pertenencia. Un sistema de inferencia común es **Mamdani.**



Mamdani es un tipo de forma de evaluar las reglas difusas donde se tienen distintas distribuciones que dan distintas aplicaciones de las distintas reglas. Esta aplicación da una figura que es la fuzzificación de la combinación (evaluación simultánea de ambas funciones). Para saber qué decisión tomar se tendría que tomar una decisión de a donde se va a parar en el conjunto de salida en base a las técnicas de defuzzificación.



Defuzzificación

- Convertir el conjunto difuso obtenido en un valor numérico preciso. Valores entre 0 y 1 a las variables originales.
- Métodos comunes:
    - Centroide: Calcula el centro de gravedad del área bajo la curva.
    - Promedio ponderado: Toma la media de los máximos.

Modelos bayesianos

Razonamiento bajo incertidumbre

- Grados de veracidad
    - Lógica difusa
- Grados de creencia
    - Teoría de la probabilidad

Sistemas basados en la teoría de la probabilidad

Se define un conjunto de variables que se usan para modelar el mundo. Tipos:

- Discretas: X
- Continuas: Fiebre

Se definen los valores que puede tomar cada variable (dominio). Ejemplo: ⟨x1 , x2 , x3 ⟩ ó ⟨cierto, falso⟩

Probabilidad a priori o incondicional: Es el grado de creencia que se le otorga a una proposición en **ausencia** de cualquier otra información. Podemos pensar en una proposición como una declaración sobre la ocurrencia de un evento. Ejemplo: P(X = x1) = 0,3 o P(x1) = 0,3; P(Test = positivo) = 0,3 o P(positivo) = 0,3

Probabilidad condicional o a posteriori: Es la probabilidad de que ocurra un evento a dado que ocurre otro evento b (evidencia). No implica causa-efecto. Ejemplo: P(X = x1 |Y = y1) = 0,7 P(X|Y) = 0,7; P(Test = positivo|Enfermedad = presente) = 0,7 o P(positivo|presente) = 0,7

- Probabilidad condicional o a posteriori: $P(a \mid b) = \frac{P(a \wedge b)}{P(b)}$ $P(a \mid b)\, P(b) = P(a \wedge b)$
- Regla de Bayes $P(b \mid a) = \frac{P(a \mid b)\, P(b)}{P(a)}$

  $P(a \mid b)$ es la probabilidad a posteriori de que la hipótesis a sea verdadera dado que se ha observado la evidencia b

  $P(b \mid a)$ es la probabilidad de observar la evidencia b si la hipótesis a es verdadera

  $P(a)$ es la probabilidad a priori de que la hipótesis a sea verdadera antes de observar la evidencia b

  $P(b)$ es la probabilidad total de que ocurra la evidencia b

¿Cómo se calcula P(b)? Se calcula aplicando el teorema de la probabilidad total: Si b puede ocurrir bajo diferentes condiciones A_1, A_2,.....,A_n entonces la probabilidad total de b se calcula como la sumatoria de las probabilidades de b dado A_i multiplicada por la probabilidad de A_i

$P(b) = \Sigma\, P(b / A_i)\, .\, P(A_i)$ Entonces la probabilidad total de que ocurra b es la suma de las probabilidades de que ocurra b bajo cada evento A_i ponderada por la probabilidad de que ocurra cada A_i

Ejemplo de Juan y la radiografía de tórax

$$P(Enf = verdadero|Rad = positivo)$$

$$= \frac{P(Rad = positivo|Enf = verdadero)P(Enf = verdadero)}{P(Rad = positivo)}$$

$$= \frac{P(Rad = positivo|Enf = verdadero)P(Enf = verdadero)}{P(Rad = pos|Enf = ver)P(Enf = ver) + P(Rad = pos|Enf = fal)P(Enf = fal)}$$

$$= \frac{0,6 \times 0,001}{0,6 \times 0,001 + 0,02 \times 0,999}$$

$$= 0.029$$

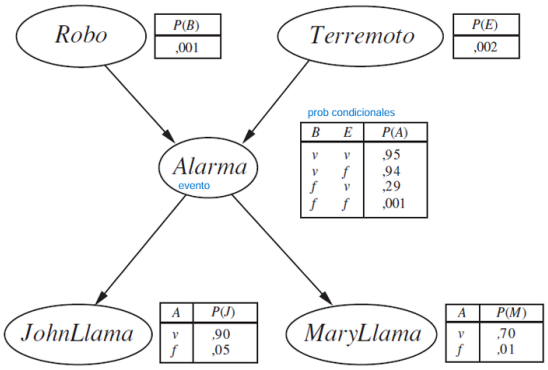
Redes bayesianas: son estructuras de datos que representan las dependencias entre variables. Muestran una descripción compacta de cualquier distribución de probabilidad conjunta completa. Son grafos dirigidos y cada nodo contiene información probabilística.

Especificación completa:

- Un conjunto de variables aleatorias forma los nodos

- Un conjunto de arcos dirigidos conecta pares de nodos
- Si hay un arco de X a Y, X es padre de Y (x depende de y)
- Cada nodo Xi tiene una distribución de probabilidad condicionada por el padre $P(X_i \mid Padres(X_i))$
- El grafo no tiene ciclos dirigidos, es un grafo acíclico dirigido (GAD)

El significado intuitivo de un arco de X a Y, es que X tiene una influencia directa sobre Y.



- Variables: Pregunta - Evidencias - Ocultas
- Usando las distribuciones conjuntas totales: Se identifican los sucesos atómicos en los que la proposición se cumple y se suman sus probabilidades. No es inferencia en una red bayesiana. No es eficiente.
- Con la regla de Bayes
- Inferencia automática: Puede ser Exacta (por enumeración (no se puede hacer a gran escala)) o Aproximada.
    - Enumeración: Una probabilidad conjunta se puede calcular como un producto de probabilidades condicionales:

      $P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid padres(X_i))$ probabilidad que se de cada uno de los x

      dado que se dio su padre.

<u>Consideraciones para redes grandes:</u> La representación de las probabilidades conjuntas totales tendría un tamaño inmanejable cuando el problema se vuelve grande. Los mecanismos de inferencia automáticos se deben enfocar en la eficiencia. No es posible crear estas redes de forma manual. Tanto el conocimiento del dominio para definir la topología, como las probabilidades conocidas, no son suficientes. En estos casos, se deben utilizar métodos automáticos para "aprender" las redes.

Sumando las probabilidades conjuntas de los casos donde se cumplen las proposiciones se puede calcular la probabilidad buscada.

$$P(X|e) = \alpha P(X, e) = \alpha \sum_{y} P(X, e, y)$$

Donde X es la variable pregunta, e valores observados e y las variables no observadas, α es la constante de normalización igual a $\frac{1}{P(e)}$. $P(X, e, y)$ es un subconjunto de las probabilidades de la distribución conjunta completa.

Lógica bayesiana

Capacidad de los sistemas de IA para tomar decisiones o hacer inferencias cuando la información disponible es incompleta, ambigua o incierta.

Grado de veracidad: Representa cuán probable o verdadero es un hecho o afirmación en un contexto específico. Se asocia a una medida numérica que indica la confianza en que un enunciado sea verdadero, generalmente en un rango de 0 a 1, donde 0 representa falsedad absoluta y 1 representa verdad absoluta. *Lógica difusa.*

Grado de creencia: Es subjetivo. Refleja la confianza subjetiva que una persona o sistema tiene en una información o hecho. No se basa únicamente en información objetiva, sino también en factores subjetivos, como experiencias previas o el contexto específico en que se evalúa la afirmación. *Teoría de la probabilidad.* Se suponen o creen cosas.

Se determina como se toman decisiones, como se plantea algún algoritmo, en base a las probabilidades conicidad y en base a lo que se va aprendiendo (hechos que van sucediendo). A medida que van pasando cosas, se va aprendiendo cada vez más de las probabilidades que se tienen. Una cosa es determinar la probabilidad de ocurrencia de un hecho sin tener ningún tipo de información previa y otra es teniendo información previa que puede ayudar a afinar esa probabilidad. A medida que más se va sabiendo del problema que se está tratando de modelar, la probabilidad de ocurrencia de algo se va a ir modificando para mejor. La información que se adquiere ayuda a "pulir" la probabilidad de ocurrencia.

Sistemas basados en la teoría de la probabilidad

- La **teoría de la probabilidad** es una rama de las matemáticas que estudia la ocurrencia de eventos aleatorios e independientes entre sí y permite cuantificar la incertidumbre sobre estos eventos. Se utiliza para modelar y analizar situaciones en las que el resultado no es seguro, como en juegos de azar, pronósticos del tiempo, procesos financieros, y muchos problemas en ciencia e ingeniería.
- Los **sistemas basados en la teoría de la probabilidad** son modelos computacionales que utilizan principios probabilísticos para manejar la incertidumbre y tomar decisiones en entornos inciertos o con datos incompletos. Estos sistemas emplean técnicas probabilísticas para modelar, analizar, y predecir comportamientos o eventos en situaciones donde no es posible tener certeza absoluta sobre los eventos.

En la teoría bayesiana se tienen en cuenta probabilidades dependientes, la ocurrencia de un evento altera la probabilidad de ocurrencia de otro evento. En este caso, sí se puede establecer una relación entre eventos.

Características

- *Manejo de incertidumbre:* Los sistemas probabilísticos no requieren datos exactos, sino que trabajan con distribuciones de probabilidad para representar el nivel de certeza de los eventos o estados.
- *Inferencia probabilística:* Permiten hacer inferencias sobre eventos futuros o desconocidos utilizando datos históricos o información parcial. Esto es fundamental en aplicaciones como el diagnóstico médico, donde no siempre se dispone de información completa. se debe tener un credo de certeza de lo que se está infiriendo es verdadero.
- *Modelo de eventos dependientes:* Utilizan probabilidades condicionales para modelar relaciones entre eventos o variables, lo cual es importante en aplicaciones como la predicción de fallos en sistemas mecánicos o electrónicos. La probabilidad de ocurrencia de un evento altera la probabilidad de ocurrencia de otro.
- *Actualización dinámica:* Algunos sistemas probabilísticos pueden actualizar sus modelos a medida que reciben nueva información, utilizando métodos como el teorema de Bayes. Esto es común en sistemas de recomendación o en motores de búsqueda personalizados. El sistema aprende con la experiencia y ajusta la probabilidad de ocurrencia con la experiencia.

Ejemplos

- *Redes bayesianas:* Estas redes son grafos que modelan relaciones de dependencia entre variables y permiten calcular probabilidades condicionales. Se usan en diagnóstico médico, sistemas expertos, y procesamiento de lenguaje natural.
- *Procesos de Markov:* Modelan sistemas en los que el estado futuro depende solo del estado actual, sin necesidad de historial previo. Son útiles en el modelado de secuencias, como el reconocimiento de patrones y el análisis de datos de series temporales.
- *Filtros de Kalman:* Son algoritmos para estimar el estado de un sistema dinámico a lo largo del tiempo. Se usan en aplicaciones como la navegación (GPS), la robótica, y el control de sistemas autónomos.
- *Modelos de predicción y recomendación:* Los sistemas de recomendación, como los que usa Netflix o Amazon, utilizan algoritmos probabilísticos para hacer recomendaciones personalizadas a partir de datos de comportamiento y preferencia de los usuarios.

Probabilidad a priori o incondicional

Es el grado de creencia que se le otorga a una proposición en ausencia de cualquier otra información. Es la probabilidad de un evento calculada sin considerar ninguna información adicional o evento previo. No se tiene información previa, es la probabilidad que se asigna sin saber nada antes. Es la probabilidad de que ocurra un evento basado únicamente en información general o histórica, sin condicionarla a otro evento. Que no dependa de otro evento no significa

que no se tiene información histórica. Es un dato estadístico. Que sea una probabilidad incondicional significa que no depende de ningún otro elemento.

Ejemplo: Supongamos que en un estudio de mercado, la probabilidad de que una persona compre un determinado producto es del 10% o 0.1. Este valor es una probabilidad a priori, ya que se basa en datos generales previos sin considerar características o condiciones adicionales, como edad o ingresos de los compradores.

$$P(compra = positivo) = 0{,}1 \text{ ó } P(positivo) = 0{,}1$$

Probabilidad condicional o a posteriori

Es la probabilidad de que ocurra un evento a dado que ocurre otro evento b (evidencia). Suponemos que el evento b está relacionado con a de forma que la probabilidad de b altera la probabilidad de a. Puede pasar que cuando se haga la estimación los eventos sean independientes por eso es importante hacer un análisis para determinar la relación entre los dos eventos y no correr el riesgo asumir una relación entre eventos que en realidad son independientes. Es la probabilidad de que ocurra un evento dado que ya ha ocurrido otro evento relacionado. Para calcular la probabilidad condicional en el caso que si se tienen eventos relacionados, *se utiliza el teorema de Bayes.*

Ejemplo: En un hospital, se estudia la probabilidad de que una persona tenga una enfermedad en función de la presencia de ciertos síntomas. Se sabe que:

1. El 2% de las personas en la población tienen la enfermedad. (Probabilidad a priori)
2. Cuando una persona tiene la enfermedad, hay un 80% de probabilidad de que presente los síntomas específicos asociados a esta. (Se está asumiendo una relación entre tener la enfermedad y tenga síntomas)
3. Cuando una persona no tiene la enfermedad, hay un 5% de probabilidad de que presente los mismos síntomas (tasa de falsos positivos).

Se quiere calcular la probabilidad de que una persona tenga la enfermedad dado que presenta síntomas específicos.

Pasos:

1. **Definir los eventos**
    a. E: La persona tiene la enfermedad. (E=presente)
    b. S: La persona presenta síntomas específicos. (S=positivo)

    Buscamos la probabilidad de que una persona tenga la enfermedad dado que presenta los síntomas: P(E|S). Donde:
    - P(): Probabilidad
    - E: Evento que se está buscando
    - S: Probabilidad conocida
    - |: se lee “dado que” o “en base a que”. Establece condicionalidad.

2. **Extraer las probabilidades dadas.** De acuerdo al enunciado, tenemos:
    a. La probabilidad de que una persona tenga la enfermedad (a priori): P(E) = 0.02 (2%).
    b. La probabilidad de que una persona no tenga la enfermedad: P(¬E) = 1 - P(E) = 0.98. Complemento de la probabilidad.
    c. La probabilidad de presentar síntomas si la persona tiene la enfermedad: P(S|E) = 0.8 (80%). Cuando se da E, S se produce un 80% de las veces. Evento condicional.
    d. La probabilidad de presentar síntomas sin tener la enfermedad: P(S|¬E) = 0.05 (5%).

3. Calcular la probabilidad total de presentar síntomas

Para calcular P(E|S), la probabilidad de que una persona tenga la enfermedad dado que presenta los síntomas, usamos el teorema de Bayes:

$$P(E|S) = \frac{P(S|E) \cdot P(E)}{P(S)}$$

donde P(S) es la probabilidad total de presentar síntomas, no se conoce, se debe averiguar, que se calcula con:

$$P(S) = P(S|E) \cdot P(E) + P(S|\neg E) \cdot P(\neg E)$$

$$P(S) = (0.8 \cdot 0.02) + (0.05 \cdot 0.98)$$

$$P(S) = 0.016 + 0.049 = 0.065$$

Probabilidad total de que tenga síntomas.

El evento siempre es la segunda parte del evento de probabilidad. Lo que se sabe es lo que está en la segunda parte de la ecuación y lo que se intenta averiguar se coloca en la primera parte de la ecuación. Multiplicar dos probabilidades (P(S|E).P(E)) se está ponderando la probabilidad de que tenga síntomas en base a la probabilidad de que esté enferma (si la probabilidad de estar enferma aumenta, la probabilidad de tener síntomas también aumenta.

4. Aplicar el Teorema de Bayes para calcular P(E|S)

$$P(E|S) = \frac{P(S|E) \cdot P(E)}{P(S)}$$

$$P(E|S) = \frac{0.8 \cdot 0.02}{0.065}$$

$$P(E|S) = \frac{0.016}{0.065} \approx 0.246$$

Resultado

La probabilidad de que una persona tenga la enfermedad dado que presenta los síntomas es aproximadamente 0.246, o el 24.6%. (Probabilidad a posteriori)

En base a la información que se conoce ahora, se altera la probabilidad a priori original independiente y se la hace dependiente. Cuando la probabilidad dependiente obtiene información nueva se altera la probabilidad a priori.

Redes Bayesianas

Las redes bayesianas (o redes de creencias bayesianas) son modelos probabilísticos que representan relaciones de dependencia entre un conjunto de variables mediante un grafo acíclico dirigido. Cada nodo en la red representa una variable, y las aristas entre los nodos indican relaciones de dependencia probabilística entre ellas.

Las redes bayesianas se utilizan para realizar inferencias, diagnosticar problemas y tomar decisiones en condiciones de incertidumbre. Cuando se tiene un conjunto de variables relacionadas entre sí en distintos niveles, que no puede utilizar un teorema de bayes simple que tiene una relación simple, se debe armar una red, la cual deja calcular las probabilidades finales viendo el impacto de las probabilidades parciales dentro de la red. *Las redes bayesianas son una generalización del teorema de bayes.*

Ejemplo: Diagnóstico de Fiebre y Gripe

Supongamos que tenemos una red bayesiana con tres variables:

- F: Fiebre (¿tiene fiebre o no?)
- G: Gripe (¿tiene gripe o no?)
- D: Dolor de cabeza (¿tiene dolor de cabeza o no?)

La red bayesiana tiene las siguientes dependencias:

- Tener gripe (G) influye en tener fiebre (F).
- Tener fiebre (F) y gripe (G) también influye en tener dolor de cabeza (D).

Definición de las probabilidades: Dadas las dependencias entre las variables, definimos las probabilidades condicionales:

Probabilidad a priori de tener gripe: $P(G)=0.1$

Probabilidad de tener fiebre dado que se tiene gripe: $P(F|G)=0.8$

Probabilidad de tener fiebre sin tener gripe: $P(F|\neg G)=0.2$

Dadas las dependencias entre las variables, definimos las probabilidades condicionales:

1. Probabilidad a priori de tener gripe: $P(G) = 0.1$
2. Probabilidad de tener fiebre dado que se tiene gripe: $P(F|G) = 0.8$
3. Probabilidad de tener fiebre sin tener gripe: $P(F|\neg G) = 0.2$
4. Probabilidad de tener dolor de cabeza dado fiebre y gripe:

   $P(D|F,G)=0.9$ $P(D|F,\neg G)=0.6$ $P(D|\neg F,G)=0.4$ $P(D|\neg F,\neg G)=0.1$

Ahora, no se tiene un evento condicional a otro, sino que se tiene un evento condicional a un conjunto de otros elementos.

Objetivo de la Inferencia

Queremos calcular la probabilidad de que una persona tenga gripe dado que tiene fiebre y dolor de cabeza, es decir, queremos inferir $P(G|F,D)$.

Resolución mediante Inferencia Exacta por Enumeración. Para resolver P(G|F,D), usamos el teorema de Bayes:

$$P(G|F,D) = \frac{P(F,D|G) \cdot P(G)}{P(F,D)}$$

Queremos calcular la probabilidad de que una persona tenga gripe dado que tiene fiebre y dolor de cabeza, es decir, queremos inferir P(G|F,D).

Resolución mediante Inferencia Exacta por Enumeración. Para resolver P(G|F,D), usamos el teorema de Bayes:

$$P(G|F,D) = \frac{P(F,D|G) \cdot P(G)}{P(F,D)} \qquad P(F,D) = P(F,D|G) \cdot P(G) + P(F,D|\neg G) \cdot P(\neg G)$$

Pasos

1. **Calcular P(F,D|G).** Utilizamos la regla de multiplicación para expandir P(F,D|G) (caso que se tenga gripe)

   $$P(F,D|G) = P(D|F,G) \cdot P(F|G) \qquad P(F,D|G) = 0.9 \cdot 0.8 = 0.72$$

2. **Calcular P(F,D|¬G).** De manera similar, expandimos P(F,D|¬G)(caso que no se tenga gripe):

   $$P(F,D|\neg G) = P(D|F,\neg G) \cdot P(F|\neg G) \qquad P(F,D|\neg G) = 0.6 \cdot 0.2 = 0.12$$

3. **Calcular P(F,D).** Ahora podemos calcular P(F,D) utilizando la probabilidad total que tenga fiebre y dolor de cabeza (si tiene o no gripe):

   $$P(F,D) = P(F,D|G) \cdot P(G) + P(F,D|\neg G) \cdot P(\neg G)$$

   $$P(F,D) = (0.72 \cdot 0.1) + (0.12 \cdot 0.9) = 0.072 + 0.108 = 0.18$$

4. **Calcular P(G|F,D)P(G|F, D)P(G|F,D).** Finalmente, usamos el teorema de Bayes para obtener P(G|F,D)

   $$P(G|F,D) = \frac{P(F,D|G) \cdot P(G)}{P(F,D)} \qquad P(G|F,D) = \frac{0.72 \cdot 0.1}{0.18} = \frac{0.072}{0.18} \approx 0.4$$

P(F,D): probabilidad total de que tenga fiebre y dolor de cabeza, sin importar si está enfermo o no. Es una probabilidad compuesta (la probabilidad compuesta se refiere a la probabilidad de que dos o más eventos ocurran juntos, combina las probabilidades de eventos individuales para determinar la probabilidad general de un resultado específico).

Resultado: La probabilidad de que una persona tenga gripe dado que tiene fiebre y dolor de cabeza es aproximadamente 0.4, o 40%. Hay un 40% de probabilidad de que si se tiene fiebre y dolor de cabeza se tenga gripe.